



**System calls**, Processes, Threads, Inter-process communication, Concurrency and synchronization. Deadlock. CPU scheduling. Memory management and Virtual memory. File systems. Disks is also under this

**Mark Distribution in Previous GATE**

| Year          | 2026 - 1 | 2026 - 2 | 2025 - 1 | 2025 - 2 | 2024 - 1 | 2024 - 2 | 2023 | 2022 | 2021 - 1 | 2021 - 2 | Minimum |
|---------------|----------|----------|----------|----------|----------|----------|------|------|----------|----------|---------|
| 1 Mark Count  | 2        | 1        | 2        | 1        | 2        | 2        | 3    | 2    | 4        | 2        | 1       |
| 2 Marks Count | 2        | 3        | 3        | 3        | 4        | 4        | 3    | 4    | 1        | 3        | 1       |
| Total Marks   | 6        | 7        | 8        | 7        | 10       | 10       | 9    | 10   | 6        | 8        | 6       |

Welcome to the Operating System (OS) chapter of your GATE Computer Science preparation! This section is designed as a comprehensive, exam-focused reference to help you master the core concepts, formulas, and problem-solving techniques essential for excelling in the GATE exam. Operating Systems form the bedrock of computing, managing hardware and software resources to provide a stable and efficient environment for applications. For GATE CS, OS is a high-scoring subject, typically carrying a weightage of 8-12 marks. Questions range from conceptual understanding of OS principles, numerical problems on scheduling, memory management, and disk I/O, to analytical questions on deadlock and synchronization. A strong grasp of this subject is crucial not only for GATE but also for a fundamental understanding of computer science.

## Topic-wise Key Concepts

### Bankers Algorithm

The Banker's Algorithm is a deadlock avoidance algorithm that checks for a safe state before granting a resource request. It ensures that the system can always find a sequence of processes that can complete their execution without leading to a deadlock.

#### • Important Formulas/Theorems:

1. **Need Matrix:** For each process  $P_i$  and resource type  $R_j$ ,  $Need[i][j] = Max[i][j] - Allocation[i][j]$ . This represents the remaining resources  $P_i$  may still request.

2. **Safety Algorithm:**

- Initialize  $Work = Available$  and  $Finish[i] = false$  for all processes.
- Find an  $i$  such that  $Finish[i] == false$  and  $Need[i] \leq Work$ .
- If such an  $i$  exists, then  $Work = Work + Allocation[i]$  and  $Finish[i] = true$ . Go to step 2.
- If no such  $i$  exists, the system is in a safe state if  $Finish[i] == true$  for all  $i$ .

3. **Resource-Request Algorithm:**

- If  $Request_i \leq Need_i$ , proceed to step 2. Otherwise, error.
- If  $Request_i \leq Available$ , proceed to step 3. Otherwise,  $P_i$  must wait.
- Pretend to allocate resources:  $Available = Available - Request_i$ ,  $Allocation_i = Allocation_i + Request_i$ ,  $Need_i = Need_i - Request_i$ .
- Run Safety Algorithm. If safe, grant request. If unsafe, revert allocation and  $P_i$  waits.

#### • Key Properties:

- Ensures deadlock avoidance by maintaining the system in a safe state.
- Requires prior knowledge of maximum resource needs for each process.
- Less conservative than deadlock prevention, allowing higher resource utilization.

#### • Common Pitfalls:

- Confusing safe state with deadlock-free state (safe implies deadlock-free, but unsafe doesn't necessarily imply deadlock).
- Incorrectly calculating the Need matrix or updating Available/Allocation vectors.
- Not checking all conditions in the Safety Algorithm or Resource-Request Algorithm.

#### • Standard Problem-Solving Techniques:

- Systematically trace the Safety Algorithm step-by-step, updating  $Work$  and  $Finish$  arrays.
- For resource requests, first check against  $Need$  and then against  $Available$ , then temporarily allocate and run the Safety Algorithm.

### Best Fit

Best Fit is a dynamic memory allocation algorithm that allocates the smallest available memory hole (partition) that is large enough to satisfy a request. Its goal is to minimize the amount of internal fragmentation.

- **Key Properties:**

- Tends to produce the smallest leftover hole, which might be useful for future small requests.
- Suffers from external fragmentation, where total free memory is sufficient but not contiguous.
- Requires searching the entire list of free partitions to find the best fit, making it slower than First Fit.

- **Common Pitfalls:**

- Confusing it with First Fit or Worst Fit.
- Incorrectly identifying the "smallest" suitable hole, especially when multiple holes have the same size.

- **Standard Problem-Solving Techniques:**

- Maintain a sorted list of free memory blocks (or iterate through all) and select the one with the smallest size greater than or equal to the request.
- Draw memory maps to visualize allocation and fragmentation.

## Context Switch

A context switch is the mechanism by which the CPU saves the state of the currently executing process or thread and restores the state of another process or thread. This allows multiple processes/threads to share a single CPU.

- **Key Properties:**

- Involves saving the Process Control Block (PCB) of the current process and loading the PCB of the next process.
- An overhead operation, as the CPU is not performing useful work during the switch.
- Latency is the time taken for a context switch.
- More frequent context switches (e.g., small time quantum in Round Robin) increase overhead.

- **Common Pitfalls:**

- Underestimating the overhead of context switching in performance calculations.
- Not understanding \*what\* exactly is saved (CPU registers, program counter, stack pointer, etc.).

## DMA (Direct Memory Access)

DMA is a feature of computer systems that allows certain hardware subsystems (like disk drive controllers or network cards) to access main system memory (RAM) independently of the central processing unit (CPU). This significantly improves I/O performance by offloading data transfer from the CPU.

- **Key Properties:**

- Reduces CPU overhead for large data transfers, allowing the CPU to perform other tasks.
- DMA controller manages the transfer, initiating an interrupt only upon completion.
- "Cycle stealing" occurs when the DMA controller temporarily takes control of the system bus from the CPU to transfer data.
- Requires careful setup by the CPU (source, destination, size).

- **Common Pitfalls:**

- Thinking DMA eliminates CPU involvement entirely (CPU still initiates and handles completion).
- Not understanding the concept of cycle stealing and its impact on CPU performance.

## Deadlock Prevention Avoidance Detection

These are strategies to deal with deadlocks. Prevention aims to ensure that at least one of the four necessary conditions for deadlock never holds. Avoidance dynamically grants resources only if the system remains in a safe state. Detection allows deadlocks to occur and then finds and resolves them.

- **Important Formulas/Theorems:**

1. **Necessary Conditions for Deadlock (Coffman Conditions):**

- $C_1$ : **Mutual Exclusion:** At least one resource must be held in a non-sharable mode.
- $C_2$ : **Hold and Wait:** A process holding at least one resource is waiting to acquire additional resources held by other processes.
- $C_3$ : **No Preemption:** Resources cannot be preempted; they can only be released voluntarily by the process holding them.
- $C_4$ : **Circular Wait:** A set of processes  $\{P_0, P_1, \dots, P_n\}$  exists such that  $P_0$  is waiting for a resource held by  $P_1$ ,  $P_1$  is waiting for a resource held by  $P_2$ , ...,  $P_{n-1}$  is waiting for a resource held by  $P_n$ , and  $P_n$  is waiting for a resource held by  $P_0$ .

2. **Deadlock Prevention:** Break one or more of the four conditions.

- Break  $C_1$ : Spooling (for printers), virtualizing resources.
- Break  $C_2$ : Request all resources at once, or release all resources before requesting new ones.
- Break  $C_3$ : Allow preemption of resources (e.g., CPU, memory).

- Break  $C_4$ : Impose a total ordering of resource types; processes must request resources in increasing order.
- 3. **Deadlock Avoidance:** Banker's Algorithm (for multiple instances of resources), Resource-Allocation Graph Algorithm (for single instances).
- 4. **Deadlock Detection:**
  - If resources have single instances, use a Wait-For Graph. A cycle implies deadlock.
  - If resources have multiple instances, use an algorithm similar to Banker's Safety Algorithm.
- **Key Properties:**
  - Prevention is conservative, often leading to low resource utilization.
  - Avoidance is more flexible but requires future knowledge of resource requests.
  - Detection allows higher concurrency but incurs overhead for detection and recovery.
- **Common Pitfalls:**
  - Confusing which condition each prevention strategy addresses.
  - Incorrectly applying RAG for multiple instance resources (a cycle in RAG with multiple instances does not necessarily imply deadlock).
  - Not understanding the trade-offs between the three strategies.
- **Standard Problem-Solving Techniques:**
  - For prevention, identify which condition is being broken.
  - For avoidance, apply Banker's Algorithm.
  - For detection, draw the RAG or Wait-For Graph and look for cycles.

## Demand Paging

Demand Paging is a virtual memory technique where pages are loaded into main memory only when they are referenced (demanded) by the CPU. This allows for larger virtual address spaces than physical memory and reduces I/O overhead by not loading unused pages.

- **Important Formulas/Theorems:**
  1. **Effective Access Time (EAT):**

$$EAT = (1 - p) \times MemoryAccessTime + p \times (PageFaultOverhead + MemoryAccessTime)$$

Where  $p$  is the page fault rate. *PageFaultOverhead* includes time for interrupt, saving process state, reading page from disk, updating page table, restoring process state.

- **Key Properties:**
  - Leverages locality of reference to achieve good performance.
  - Reduces physical memory requirements for processes.
  - Introduces page faults, which are expensive operations.
  - Can lead to thrashing if the page fault rate becomes too high.
- **Common Pitfalls:**
  - Incorrectly calculating EAT, especially the components of page fault overhead.
  - Misunderstanding the concept of locality of reference.
  - Confusing demand paging with simple paging.
- **Standard Problem-Solving Techniques:**
  - Calculate EAT by plugging in given values for page fault rate, memory access time, and page fault overhead.
  - Analyze scenarios for thrashing.

## Disk

A disk (hard disk drive) is a non-volatile storage device that stores data on rotating platters. It is characterized by its physical structure and the time it takes to access data.

- **Important Formulas/Theorems:**
  1. **Disk Access Time:**

$$AccessTime = SeekTime + RotationalLatency + TransferTime$$

- **Seek Time:** Time taken for the disk arm to move the read/write heads to the correct track.
- **Rotational Latency:** Time taken for the desired sector to rotate under the read/write head (average is half a rotation).
- **Transfer Time:** Time taken to transfer the actual data.

$$TransferTime = \frac{NumberOfSectors}{SectorsPerTrack} \times RotationTime$$

- **Key Properties:**

- Data is stored in sectors, which are grouped into tracks, and tracks are stacked into cylinders.
- Seek time is typically the most significant component of disk access time.
- Mechanical components make disk access orders of magnitude slower than RAM access.

- **Common Pitfalls:**

- Incorrectly calculating average rotational latency (usually half a rotation).
- Forgetting to include all three components of access time.

- **Standard Problem-Solving Techniques:**

- Break down the problem into calculating each component (seek, rotational, transfer) and then sum them up.

## Disk Scheduling

Disk scheduling algorithms aim to minimize the total head movement of the disk arm, thereby reducing the average disk access time and improving I/O throughput. They reorder pending disk I/O requests.

- **Important Formulas/Theorems:**

1. **Total Head Movement:** Sum of absolute differences between consecutive track numbers accessed.

- **Algorithms:**

- **FCFS (First-Come, First-Served):** Processes requests in the order they arrive. Simple but inefficient.
- **SSTF (Shortest Seek Time First):** Selects the request with the minimum seek time from the current head position. Can lead to starvation.
- **SCAN (Elevator Algorithm):** The disk arm moves from one end of the disk to the other, servicing requests along the way. When it reaches an end, it reverses direction.
- **C-SCAN (Circular SCAN):** Similar to SCAN, but when the arm reaches one end, it immediately returns to the other end without servicing requests on the return trip. Provides more uniform wait times.
- **LOOK:** Similar to SCAN, but the arm only goes as far as the last request in each direction, then reverses.
- **C-LOOK:** Similar to C-SCAN, but the arm only goes as far as the last request in each direction, then returns to the first request in the other direction without going to the absolute end.

- **Key Properties:**

- SSTF is optimal for average seek time but can cause starvation.
- SCAN/C-SCAN/LOOK/C-LOOK provide better fairness and prevent starvation.
- C-SCAN/C-LOOK are preferred for heavy loads as they provide more uniform service.

- **Common Pitfalls:**

- Incorrectly calculating head movement, especially for SCAN/C-SCAN/LOOK/C-LOOK where direction matters.
- Forgetting to include the initial head position in the calculation.
- Confusing SCAN with C-SCAN or LOOK with C-LOOK.

- **Standard Problem-Solving Techniques:**

- Draw a number line representing tracks and mark request positions. Trace the head movement for each algorithm.
- Carefully track the current head position and the direction of movement.

## File System

A file system is an OS component that controls how data is stored and retrieved. It organizes files into a hierarchical structure, manages file attributes, and provides mechanisms for accessing and protecting files.

- **Key Properties:**

- **File Attributes:** Name, type, location, size, protection, time/date, user ID.
- **File Operations:** Create, write, read, reposition, delete, truncate.
- **Directory Structure:** Single-level, two-level, tree-structured, acyclic-graph, general graph.
- **Allocation Methods:** Contiguous, Linked, Indexed.
- **Free Space Management:** Bit vector, linked list, grouping, counting.

- **Common Pitfalls:**

- Confusing file allocation methods (e.g., linked vs. indexed).
- Not understanding the trade-offs of different directory structures (e.g., sharing, path length).

## Fork System Call

The fork() system call creates a new process (child process) that is a duplicate of the calling process (parent process). Both processes then execute concurrently from the point of the fork() call.

- **Key Properties:**

- The child process receives a copy of the parent's address space, including code, data, and stack segments (often implemented using copy-on-write).
- The child process gets a unique Process ID (PID).
- `fork()` returns 0 to the child process, the child's PID to the parent process, and -1 on failure.
- Child process inherits open file descriptors, signals, etc.
- **Common Pitfalls:**
  - Incorrectly predicting the output of programs involving multiple `fork()` calls due to confusion about which process is executing which part of the code.
  - Not understanding the return values of `fork()` in parent vs. child.
  - Assuming parent and child share memory directly (they get copies, though copy-on-write optimizes this).
- **Standard Problem-Solving Techniques:**
  - Trace the execution path for both parent and child processes separately after each `fork()`.
  - Keep track of the number of processes created. For  $N$  `fork()` calls in a sequence,  $2^N$  processes are created. If `fork()` is in a loop, it's more complex.

## IO Handling

I/O handling refers to the mechanisms and techniques used by the operating system to manage data transfer between the CPU/memory and peripheral devices. Efficient I/O handling is crucial for system performance.

- **Key Properties:**
  - **Polling:** CPU repeatedly checks the status of an I/O device. Simple but inefficient, wastes CPU cycles.
  - **Interrupt-driven I/O:** Device notifies the CPU via an interrupt when I/O is complete or ready. More efficient for sporadic I/O.
  - **DMA (Direct Memory Access):** Device controller transfers data directly to/from memory without CPU intervention. Most efficient for large data transfers.
  - **Buffering:** Using temporary memory areas to hold data during transfer, smoothing out speed differences.
  - **Spooling:** Holding data for a device (e.g., printer) in a buffer until the device is ready.
- **Common Pitfalls:**
  - Confusing the efficiency and use cases of polling vs. interrupts vs. DMA.
  - Not understanding the role of device controllers.

## Input Output

Input/Output (I/O) refers to the communication between an information processing system (like a computer) and the outside world, possibly a human or another information processing system. It involves the transfer of data to and from peripheral devices.

- **Key Properties:**
  - Can be synchronous (CPU waits for I/O completion) or asynchronous (CPU continues processing while I/O operates in background).
  - Managed by device controllers and device drivers.
  - Performance is often a bottleneck in overall system performance.

## Inter Process Communication (IPC)

IPC refers to mechanisms provided by the operating system that allow independent processes to communicate and synchronize their actions. This is essential for cooperative processes and distributed systems.

- **Key Properties:**
  - **Shared Memory:** Processes share a region of memory for direct data exchange. Fastest IPC, but requires explicit synchronization.
  - **Message Passing:** Processes communicate by sending and receiving messages. Simpler to implement for small data transfers, easier to use in distributed systems.
  - **Pipes:** Unidirectional (ordinary pipes) or bidirectional (named pipes/FIFOs) communication channels.
  - **Sockets:** For network communication, allowing processes on different machines to communicate.
  - **Semaphores/Mutexes/Monitors:** Primarily for synchronization, but can indirectly facilitate communication by coordinating access to shared resources.
- **Common Pitfalls:**
  - Confusing shared memory with message passing (shared memory requires explicit synchronization, message passing often has built-in synchronization).
  - Not understanding the difference between pipes and named pipes.

## Interrupts

An interrupt is a signal to the CPU from a hardware device or a software program that indicates an event has occurred and requires immediate attention. It causes the CPU to temporarily suspend its current task and execute a special routine called an interrupt handler.

- **Key Properties:**

- **Hardware Interrupts:** Generated by I/O devices (e.g., disk completion, keyboard input).
- **Software Interrupts (Traps/Exceptions):** Generated by programs (e.g., system calls, division by zero, page fault).
- **Interrupt Vector:** A table containing the addresses of interrupt service routines (ISRs) for various interrupt types.
- **Maskable Interrupts:** Can be temporarily ignored by the CPU.
- **Non-maskable Interrupts (NMI):** Critical events that cannot be ignored (e.g., memory parity error).
- Involves saving the CPU's context (registers, PC) before executing the ISR and restoring it afterward.

- **Common Pitfalls:**

- Confusing hardware interrupts with software interrupts (traps).
- Not understanding the role of the interrupt vector.
- Forgetting the context saving/restoring overhead.

## Least Recently Used (LRU)

LRU is a page replacement algorithm that replaces the page that has not been used for the longest period of time. It is based on the principle of temporal locality, assuming that pages recently used are likely to be used again soon.

- **Key Properties:**

- Generally performs well, approximating the optimal algorithm.
- Does not suffer from Belady's anomaly (increasing page faults with more frames).
- Difficult to implement efficiently in hardware, as it requires tracking the exact usage time of each page.
- Approximations like Second Chance or Clock algorithm are often used in practice.

- **Common Pitfalls:**

- Incorrectly identifying the "least recently used" page, especially in complex sequences.
- Confusing it with FIFO or LFU.

- **Standard Problem-Solving Techniques:**

- Maintain a "time stamp" or "counter" for each page, or conceptually keep a stack where the most recently used page is at the top. When a page is referenced, move it to the top. When a page fault occurs, remove the page at the bottom.
- Trace the page references and page frames step-by-step.

## Linked Allocation

Linked allocation is a file allocation method where each file is stored as a linked list of disk blocks. Each block contains a pointer to the next block in the file.

- **Key Properties:**

- No external fragmentation, as any free block can be used.
- Files can grow dynamically.
- Sequential access is efficient, but direct access (random access) is very slow due to pointer traversal.
- Pointer overhead: a portion of each disk block is used for the pointer, reducing data storage capacity.
- Reliability issue: a lost or damaged pointer can lead to the loss of the rest of the file.

- **Common Pitfalls:**

- Confusing it with contiguous or indexed allocation.
- Overlooking the performance impact on random access or the pointer overhead.

## Memory Management

Memory management is the operating system's function of handling primary memory. It allocates memory to processes, protects processes from each other, and provides an abstraction of a large, uniform memory space.

- **Key Properties:**

- **Relocation:** Mapping logical addresses to physical addresses.
- **Protection:** Preventing processes from accessing each other's memory.

- **Sharing:** Allowing processes to share memory regions.
- **Logical Organization:** Segmentation, paging.
- **Physical Organization:** Contiguous allocation, non-contiguous allocation.
- **Techniques:** Paging, segmentation, virtual memory.
- **Common Pitfalls:**
  - Confusing internal vs. external fragmentation.
  - Not understanding the difference between logical and physical addresses.

## Multilevel Paging

Multilevel paging (or hierarchical paging) is a technique used in virtual memory systems to reduce the size of the page table. Instead of a single, large page table, it uses a tree-like structure of page tables, where the outer page table points to inner page tables, which then point to physical frames.

- **Important Formulas/Theorems:**

1. **Effective Access Time (EAT) for N-level Paging:**

$$EAT = (N + 1) \times MemoryAccessTime$$

(Assuming no TLB and page tables are in memory).

2. **Number of Bits for Page Number:** If a logical address is  $m$  bits and page size is  $2^k$  bytes, then page offset is  $k$  bits, and page number is  $(m - k)$  bits.
  3. **Number of Entries in Page Table:**  $2^{\text{page number bits}}$ .
  4. **Size of Page Table:**  $2^{\text{page number bits}} \times \text{size of page table entry}$ .
- **Key Properties:**
    - Reduces the amount of physical memory required for page tables, especially for sparse address spaces.
    - Increases memory access time because multiple memory accesses are needed to find a physical address (one for each level of page table).
    - Can be combined with TLB to mitigate increased access time.
  - **Common Pitfalls:**
    - Incorrectly calculating the number of bits for each level of the page table.
    - Forgetting to account for all memory accesses when calculating EAT without TLB.
  - **Standard Problem-Solving Techniques:**
    - Break down the virtual address into segments corresponding to each level of the page table and the page offset.
    - Calculate the number of page table entries and the size of each page table level.

## OS Protection

OS protection mechanisms ensure that system resources (CPU, memory, files, I/O devices) are used only in ways authorized by the operating system. This prevents malicious or erroneous programs from harming the system or other programs.

- **Key Properties:**
  - **Dual-Mode Operation:** User mode (restricted privileges) and Kernel mode (full privileges). A mode bit indicates the current mode.
  - **Privileged Instructions:** Instructions that can only be executed in kernel mode (e.g., I/O instructions, setting timer).
  - **System Calls:** User programs request OS services, transitioning from user to kernel mode.
  - **Memory Protection:** Base and limit registers, paging, segmentation.
  - **Access Control Lists (ACLs):** For files and other resources, specifying who can access what and how.
  - **Capabilities:** Tokens that grant specific access rights to a resource.
- **Common Pitfalls:**
  - Confusing the purpose of dual-mode operation with memory protection.
  - Not understanding how system calls facilitate a controlled transition to kernel mode.

## Page Replacement

Page replacement algorithms decide which page to remove from main memory when a page fault occurs and no free frames are available. The goal is to minimize the number of page faults.

- **Important Formulas/Theorems:**

1. **Hit Ratio:**  $\frac{\text{Number of Page Hits}}{\text{Total Page References}}$
  2. **Miss Ratio (Page Fault Rate):**  $\frac{\text{Number of Page Faults}}{\text{Total Page References}}$
  3.  $\text{HitRatio} + \text{MissRatio} = 1$
- **Algorithms:**
    - **FIFO (First-In, First-Out):** Replaces the page that has been in memory the longest. Simple but can suffer from Belady's anomaly.
    - **Optimal (OPT/MIN):** Replaces the page that will not be used for the longest period of time in the future. Impossible to implement in practice, used as a benchmark.
    - **LRU (Least Recently Used):** Replaces the page that has not been used for the longest time. Generally good performance, does not suffer from Belady's anomaly, but complex to implement.
    - **LFU (Least Frequently Used):** Replaces the page with the smallest count of references.
    - **MFU (Most Frequently Used):** Replaces the page with the largest count of references (less common).
    - **Second Chance (Clock):** A FIFO variant that gives a page a "second chance" if its reference bit is set.
  - **Key Properties:**
    - Belady's anomaly: For some algorithms (like FIFO), increasing the number of available page frames can increase the number of page faults.
    - Optimal algorithm provides the theoretical minimum number of page faults.
  - **Common Pitfalls:**
    - Incorrectly applying the rules for each algorithm, especially for LRU and Optimal.
    - Forgetting to count the initial page loads as page faults.
    - Misinterpreting Belady's anomaly.
  - **Standard Problem-Solving Techniques:**
    - Trace the page reference string through the page frames for each algorithm, step-by-step, marking hits and faults.
    - Keep track of the state of each page (e.g., arrival time for FIFO, last used time for LRU, future use for Optimal).

## Precedence Graph

A precedence graph (or process graph) is a directed acyclic graph (DAG) used to represent the execution order and dependencies among a set of tasks or processes. An edge from task A to task B means task A must complete before task B can start.

- **Key Properties:**
  - Nodes represent tasks/processes, edges represent dependencies.
  - Helps in identifying potential parallelism and critical paths.
  - Used in scheduling to ensure correct execution order.
  - Must be acyclic to represent a valid execution sequence.
- **Common Pitfalls:**
  - Drawing cycles, which implies an impossible dependency.
  - Misinterpreting the meaning of an edge (A → B means A *must* finish before B starts, not just that B follows A).

## Process

A process is a program in execution. It is an active entity, unlike a program, which is a passive entity. Each process has its own address space, resources, and execution context.

- **Key Properties:**
  - **Process Control Block (PCB):** Contains all information about a process (state, PC, registers, memory limits, open files, etc.).
  - **Process States:**
    - **New:** Process being created.
    - **Ready:** Waiting to be assigned to a processor.
    - **Running:** Instructions are being executed.
    - **Waiting (Blocked):** Waiting for some event (e.g., I/O completion, signal).
    - **Terminated:** Process has finished execution.
  - Each process has its own virtual address space, including code, data, and stack segments.
  - Processes are typically heavy-weight compared to threads.
- **Common Pitfalls:**
  - Confusing a process with a program or a thread.
  - Incorrectly identifying transitions between process states.



## Process Scheduling

Process scheduling is the activity of the operating system that selects which process to run next from the ready queue and allocates the CPU to it. The goal is to optimize various performance metrics.

- **Important Formulas/Theorems:**

1. **Turnaround Time (TAT):**  $CompletionTime - ArrivalTime$
2. **Waiting Time (WT):**  $TurnaroundTime - BurstTime$
3. **Response Time (RT):**  $FirstResponseTime - ArrivalTime$
4. **Throughput:** Number of processes completed per unit time.
5. **CPU Utilization:** Percentage of time the CPU is busy.

- **Algorithms:**

- **FCFS (First-Come, First-Served):** Non-preemptive. Simple, but can have high average waiting time (convoy effect).
- **SJF (Shortest Job First):** Non-preemptive. Optimal for minimum average waiting time. Requires knowing future burst times.
- **SRTF (Shortest Remaining Time First):** Preemptive version of SJF. Optimal for minimum average waiting time.
- **Priority Scheduling:** Can be preemptive or non-preemptive. Processes with higher priority run first. Can lead to starvation (low-priority processes never run).
- **Round Robin (RR):** Preemptive. Each process gets a small unit of CPU time (time quantum). Fair, good for interactive systems.

- **Key Properties:**

- **Preemptive:** CPU can be taken away from a running process.
- **Non-preemptive:** CPU is held until process completes or blocks.
- **Starvation:** A process may wait indefinitely.
- **Aging:** Gradually increasing the priority of processes that wait for a long time to prevent starvation.

- **Common Pitfalls:**

- Incorrectly drawing Gantt charts, especially for preemptive algorithms.
- Mistakes in calculating TAT, WT, or RT.
- Confusing preemptive vs. non-preemptive versions of algorithms.
- Not handling arrival times correctly.

- **Standard Problem-Solving Techniques:**

- Draw a Gantt chart for each algorithm, carefully marking arrival, start, and completion times.
- Systematically calculate TAT, WT, and RT for each process and then their averages.
- For preemptive algorithms, check for preemption at every arrival or completion event.

## Process Synchronization

Process synchronization refers to the coordination of multiple processes to ensure that they access shared resources or critical sections in a controlled manner, preventing race conditions and maintaining data consistency.

- **Important Formulas/Theorems:**

1. **Critical Section Problem Requirements:**

- **Mutual Exclusion:** Only one process can be in its critical section at any time.
- **Progress:** If no process is in its critical section and some processes want to enter, only those not in their remainder section can participate in the decision, and this decision cannot be postponed indefinitely.
- **Bounded Waiting:** There is a limit on the number of times other processes can enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

- **Mechanisms:**

- **Semaphores:** Integer variables accessed only through atomic wait() (P) and signal() (V) operations. Can be binary (mutex) or counting.
- **Mutex Locks:** A simpler binary semaphore, used for mutual exclusion.
- **Monitors:** High-level synchronization construct that encapsulates shared data and procedures that operate on the data, ensuring mutual exclusion and providing condition variables for waiting.
- **Hardware Solutions:** TestAndSet, Swap instructions.

- **Key Properties:**

- **Race Condition:** When multiple processes access and modify shared data concurrently, and the final result depends on the order of execution.
- **Deadlock:** Can occur with improper use of synchronization primitives (e.g., two processes waiting for each other's resources).

- **Common Pitfalls:**

- Incorrectly using wait() and signal() operations, leading to deadlock or starvation.
- Not ensuring atomicity of synchronization primitives.
- Confusing mutexes with counting semaphores.
- **Standard Problem-Solving Techniques:**
  - Trace the execution flow of multiple processes accessing shared resources, observing the values of semaphores or mutexes.
  - Identify potential race conditions and propose synchronization solutions.
  - Analyze classic synchronization problems (Producer-Consumer, Readers-Writers, Dining Philosophers).

## Resource Allocation

Resource allocation is the process by which the operating system assigns available resources (CPU, memory, I/O devices, files) to processes. This is a fundamental task of the OS, impacting system performance and stability.

- **Key Properties:**
  - Resources can be preemptable (e.g., CPU) or non-preemptable (e.g., printer).
  - Proper resource allocation is critical to prevent deadlocks and ensure fairness.
  - Involves policies for granting requests and reclaiming resources.

## Resource Allocation Graph (RAG)

A Resource Allocation Graph is a directed graph used to depict the state of a system regarding resources and processes. It helps in visualizing resource allocation and detecting deadlocks.

- **Key Properties:**
  - **Nodes:** Processes (circles) and Resource types (rectangles).
  - **Edges:**
    - **Request Edge:** From process to resource type (e.g.,  $P_i \rightarrow R_j$ ), indicating  $P_i$  is requesting an instance of  $R_j$ .
    - **Assignment Edge:** From resource type to process (e.g.,  $R_j \rightarrow P_i$ ), indicating an instance of  $R_j$  has been allocated to  $P_i$ .
  - If the RAG contains a cycle, and each resource type has only one instance, then a deadlock exists.
  - If the RAG contains a cycle, and some resource types have multiple instances, a deadlock *may* exist (further analysis needed).
- **Common Pitfalls:**
  - Incorrectly drawing request vs. assignment edges.
  - Assuming a cycle always implies deadlock, especially for multiple-instance resources.
  - Not understanding the difference between RAG and Wait-For Graph.
- **Standard Problem-Solving Techniques:**
  - Draw the RAG based on the given process and resource states.
  - Identify cycles in the graph.
  - If cycles exist, determine if they lead to deadlock based on resource instance counts.

## Round Robin Scheduling

Round Robin (RR) is a preemptive process scheduling algorithm designed for time-sharing systems. Each process is given a small unit of CPU time, called a time quantum (or time slice). When the quantum expires, the process is preempted and added to the end of the ready queue.

- **Important Formulas/Theorems:**
  1. **Turnaround Time (TAT):**  $CompletionTime - ArrivalTime$
  2. **Waiting Time (WT):**  $TurnaroundTime - BurstTime$
  3. **Number of Context Switches:** Depends on the number of processes and the time quantum.
- **Key Properties:**
  - Fair scheduling, as every process gets a chance to run.
  - Good for interactive systems, providing quick response times.
  - Performance heavily depends on the time quantum:
    - Small quantum: High context switch overhead, good response time.
    - Large quantum: Approaches FCFS, lower overhead, potentially poor response time.
- **Common Pitfalls:**
  - Incorrectly handling the time quantum and preemption points.
  - Mistakes in calculating TAT or WT, especially when processes arrive at different times.

- Forgetting to account for context switch overhead if specified in the problem.
- **Standard Problem-Solving Techniques:**
  - Draw a Gantt chart, carefully allocating CPU time in quantum chunks and moving processes to the end of the ready queue upon preemption.
  - Keep track of remaining burst times for each process.

## Semaphore

A semaphore is a synchronization primitive, a protected integer variable that can only be accessed and modified by two atomic operations: `wait()` (also known as P or down) and `signal()` (also known as V or up).

### • Important Formulas/Theorems:

#### 1. **wait(S) operation:**

```
wait(S) {
    while (S <= 0); // busy wait
    S--;
}
```

(Alternatively, block the process if  $S \leq 0$ ).

#### 2. **signal(S) operation:**

```
signal(S) {
    S++;
}
```

(Alternatively, wake up a blocked process if any).

### • Key Properties:

- **Counting Semaphore:** Can range over an unrestricted domain. Used to control access to a resource with multiple instances.
- **Binary Semaphore (Mutex):** Can only be 0 or 1. Used for mutual exclusion (like a lock).
- Can be used for both mutual exclusion and process synchronization (ordering of events).
- Improper use can lead to deadlock or starvation.

### • Common Pitfalls:

- Swapping the order of `wait()` and `signal()` operations, leading to incorrect synchronization.
- Using busy-waiting semaphores in a single-processor system (wastes CPU cycles).
- Not initializing semaphores correctly.

### • Standard Problem-Solving Techniques:

- Trace the values of semaphores as processes execute `wait()` and `signal()` operations.
- Identify critical sections and shared resources to determine appropriate semaphore usage.
- Solve classic synchronization problems using semaphores.

## SRTF (Shortest Remaining Time First)

SRTF is a preemptive version of the Shortest Job First (SJF) scheduling algorithm. The CPU is allocated to the process with the smallest remaining burst time. If a new process arrives with a shorter burst time than the currently running process's remaining time, the current process is preempted.

### • Important Formulas/Theorems:

1. **Turnaround Time (TAT):**  $CompletionTime - ArrivalTime$
2. **Waiting Time (WT):**  $TurnaroundTime - BurstTime$

### • Key Properties:

- Optimal for minimizing average waiting time.
- Requires knowing the remaining burst time of all processes, which is difficult in practice.
- High overhead due to frequent context switches, especially with many short jobs or frequent arrivals.
- Can suffer from starvation if a continuous stream of short jobs keeps arriving.

### • Common Pitfalls:

- Incorrectly determining when preemption occurs (at every arrival or when a shorter job becomes ready).
- Mistakes in calculating remaining burst times.
- Overlooking processes that arrive later but have very short burst times.

### • Standard Problem-Solving Techniques:

- Create a Gantt chart, carefully tracking the remaining burst time of all ready processes at each time unit.
- At each time unit, check for new arrivals and select the process with the minimum remaining burst time.

## System Calls

System calls provide an interface between a running program and the operating system. They allow user-level programs to request services from the OS kernel, such as file I/O, process creation, memory allocation, and device access.

- **Key Properties:**

- Transition from user mode to kernel mode (privileged mode).
- Typically invoked through a software interrupt (trap).
- Provide a controlled and secure way for user programs to access protected resources.
- Examples: `fork()`, `exec()`, `read()`, `write()`, `open()`, `close()`, `wait()`, `exit()`.

- **Common Pitfalls:**

- Confusing system calls with library calls (library calls are user-level functions that may or may not invoke system calls).
- Not understanding the mode switch involved.

## Threads

A thread (or lightweight process) is a basic unit of CPU utilization within a process. Multiple threads within the same process share the same code section, data section, and other OS resources (like open files, signals), but each has its own program counter, register set, and stack.

- **Key Properties:**

- **Faster Context Switching:** Switching between threads in the same process is faster than switching between processes.
- **Resource Sharing:** Threads within a process share memory and resources, facilitating communication.
- **Concurrency:** Multiple threads can execute concurrently, improving application responsiveness and throughput on multi-core systems.
- **User-level Threads:** Managed by a user-level library, faster to create/manage, but if one thread blocks, the entire process blocks.
- **Kernel-level Threads:** Managed by the OS kernel, slower to create/manage, but if one thread blocks, others can still run.
- **Many-to-One, One-to-One, Many-to-Many models:** Different mappings between user and kernel threads.

- **Common Pitfalls:**

- Confusing threads with processes (threads share memory, processes do not).
- Forgetting that synchronization is still required for shared data among threads.
- Misunderstanding the implications of user-level vs. kernel-level threads.

## Translation Lookaside Buffer (TLB)

The TLB is a small, fast hardware cache that stores recent virtual-to-physical address translations. It is used to speed up memory access in paged virtual memory systems by avoiding multiple memory accesses to the page table.

- **Important Formulas/Theorems:**

1. **Effective Access Time (EAT) with TLB:**

$$EAT = TLB\_Hit\_Ratio \times (TLB\_Access\_Time + Memory\_Access\_Time) + (1 - TLB\_Hit\_Ratio)$$

Where *Page\_Table\_Access\_Time* is the time to access the page table in main memory. If the page table is also in memory, this is another *Memory\_Access\_Time*.

- **Key Properties:**

- Significantly reduces the average memory access time if the hit ratio is high.
- A TLB miss requires a page table walk (accessing the page table in main memory).
- TLB entries are flushed on context switches between processes (though some TLBs support ASIDs to avoid this).

- **Common Pitfalls:**

- Incorrectly calculating EAT, especially the components for a TLB miss (TLB access + memory access for page table + memory access for data).
- Forgetting to include TLB access time even on a hit.

- **Standard Problem-Solving Techniques:**

- Carefully plug values into the EAT formula, ensuring all access times are accounted for.
- Understand how a TLB miss impacts the total access time.

## Virtual Memory

Virtual memory is a memory management technique that provides an application with an illusion of a contiguous, large address space, even if the physical memory is fragmented or smaller than the virtual address space. It separates logical memory from physical memory.

- **Key Properties:**

- Allows programs to be larger than physical memory.
- Enables efficient sharing of memory among multiple processes.
- Provides memory protection.
- Commonly implemented using paging (demand paging) and swapping.
- Introduces the concept of page faults.

- **Important Formulas/Theorems:**

1. **Effective Access Time (EAT):** (See Demand Paging and TLB sections for detailed formulas).

- **Common Pitfalls:**

- Confusing virtual memory with physical memory.
- Not understanding the role of the MMU (Memory Management Unit) in address translation.
- Misinterpreting the benefits and overheads of virtual memory.

## Quick Formula Reference

- **Banker's Algorithm:**

- Need Matrix:  $Need[i][j] = Max[i][j] - Allocation[i][j]$

- **Demand Paging / Virtual Memory EAT (without TLB):**

$$EAT = (1 - p) \times MemoryAccessTime + p \times (PageFaultOverhead + MemoryAccessTime)$$

where  $p$  is page fault rate.

- **Disk Access Time:**

$$AccessTime = SeekTime + RotationalLatency + TransferTime$$

$$AverageRotationalLatency = \frac{1}{2} \times RotationTime$$

$$TransferTime = \frac{NumberOfSectors}{SectorsPerTrack} \times RotationTime$$

- **Multilevel Paging EAT (without TLB):**

$$EAT = (N + 1) \times MemoryAccessTime$$

for  $N$ -level paging, assuming page tables are in memory.

- **Page Replacement:**

- Hit Ratio:  $\frac{\text{Number of Page Hits}}{\text{Total Page References}}$
- Miss Ratio (Page Fault Rate):  $\frac{\text{Number of Page Faults}}{\text{Total Page References}}$
- $HitRatio + MissRatio = 1$

- **Process Scheduling Metrics:**

- Turnaround Time (TAT):  $CompletionTime - ArrivalTime$
- Waiting Time (WT):  $TurnaroundTime - BurstTime$
- Response Time (RT):  $FirstResponseTime - ArrivalTime$

- **TLB EAT:**

$$EAT = TLB\_Hit\_Ratio \times (TLB\_Access\_Time + Memory\_Access\_Time) + (1 - TLB\_Hit\_Ratio) \times \dots$$

## Important Tips for GATE

1. **Master Numerical Problems:** A significant portion of OS questions in GATE are numerical, especially from Process Scheduling (FCFS, SJF, SRTF, RR), Memory Management (Paging, Page Replacement, TLB EAT), and Disk Scheduling. Practice drawing Gantt charts and calculating metrics meticulously.
2. **Understand Preemption:** Pay close attention to whether a scheduling algorithm is preemptive or non-preemptive.

This is a common source of error in Gantt chart construction. For preemptive algorithms, always check for new arrivals at each time unit.

3. **Trace Algorithms Systematically:** For complex algorithms like Banker's, Page Replacement, or Disk Scheduling, trace the steps methodically. Draw diagrams (RAG, memory maps, Gantt charts) to visualize the state changes.
4. **Differentiate Similar Concepts:** Be clear on the distinctions between processes and threads, mutexes and semaphores, internal and external fragmentation, logical and physical addresses, and different deadlock strategies (prevention, avoidance, detection).
5. **Read Questions Carefully:** Look for keywords like "average," "minimum," "maximum," "at least," "at most," "preemptive," "non-preemptive," and specific units (ms, ns). A small detail can change the entire answer.
6. **Focus on Trade-offs:** Many OS concepts involve trade-offs (e.g., performance vs. fairness in scheduling, memory usage vs. access time in paging). Understand why certain design choices are made and their implications.
7. **Review Formulas Regularly:** Keep the key formulas for EAT, scheduling metrics, and disk access time handy and review them frequently. Understand the components of each formula.
8. **Practice with GATE Previous Year Questions:** This will help you understand the common question patterns, difficulty levels, and time management strategies specific to the GATE exam.

## 5.1

## Bankers Algorithm (2)

 **Practice Test:** Test 1 (8Q)

### 5.1.1 Bankers Algorithm: GATE CSE 2018 | Question: 39



In a system, there are three types of resources:  $E$ ,  $F$  and  $G$ . Four processes  $P_0$ ,  $P_1$ ,  $P_2$  and  $P_3$  execute concurrently. At the outset, the processes have declared their maximum resource requirements using a matrix named Max as given below. For example,  $\text{Max}[P_2, F]$  is the maximum number of instances of  $F$  that  $P_2$  would require. The number of instances of the resources allocated to the various processes at any given state is given by a matrix named Allocation.

Consider a state of the system with the Allocation matrix as shown below, and in which 3 instances of  $E$  and 3 instances of  $F$  are only resources available.

| Allocation |   |   |   | Max   |   |   |   |
|------------|---|---|---|-------|---|---|---|
|            | E | F | G |       | E | F | G |
| $P_0$      | 1 | 0 | 1 | $P_0$ | 4 | 3 | 1 |
| $P_1$      | 1 | 1 | 2 | $P_1$ | 2 | 1 | 4 |
| $P_2$      | 1 | 0 | 3 | $P_2$ | 1 | 3 | 3 |
| $P_3$      | 2 | 0 | 0 | $P_3$ | 5 | 4 | 1 |

From the perspective of deadlock avoidance, which one of the following is true?

- A. The system is in *safe* state
- B. The system is not in *safe* state, but would be *safe* if one more instance of  $E$  were available
- C. The system is not in *safe* state, but would be *safe* if one more instance of  $F$  were available
- D. The system is not in *safe* state, but would be *safe* if one more instance of  $G$  were available

gatecse-2018   operating-system   deadlock-prevention-avoidance-detection   bankers-algorithm   normal   two-marks

Answer key 

### 5.1.2 Bankers Algorithm: GATE CSE 2026 | Set 1 | Question: 19



With respect to deadlocks in an operating system, which of the following statements is/are FALSE?

- A. Banker's algorithm is used to prevent deadlocks
- B. Deadlock formation can be prevented by ensuring that the hold and wait condition is not allowed
- C. An assignment edge in a resource allocation graph is marked from a process to a resource
- D. A safe state guarantees that all processes can finish without formation of a deadlock

gatecse-2026-set1   operating-system   deadlock-prevention-avoidance-detection   bankers-algorithm   multiple-selects   one-mark

Answer key 

## 5.2.1 Best Fit: GATE CSE 2026 | Set 2 | Question: 45



Consider contiguous allocation of physical memory to processes using variable partitioning scheme. Suppose there are 8 holes in the memory of sizes 20KB, 4 KB, 25 KB, 18 KB, 7 KB, 9 KB, 15 KB, and 12 KB. Assume that no two holes are adjacent. Two processes P1 of size 16 KB and P2 of size 9 KB arrive in that order, and they are allocated memory using the best-fit technique. After allocating space to P1 and P2, the number of holes of size less than 8 KB is \_\_\_\_\_. (answer in integer)

Note:  $1K = 2^{10}$

gatecse-2026-set2 operating-system memory-management best-fit numerical-answers two-marks

Answer key

Practice Test: Test 1 (6Q)

## 5.3.1 Context Switch: GATE CSE 1999 | Question: 2.12



Which of the following actions is/are typically not performed by the operating system when switching context from process  $A$  to process  $B$ ?

- A. Saving current register values and restoring saved register values for process  $B$ .
- B. Changing address translation tables.
- C. Swapping out the memory image of process  $A$  to the disk.
- D. Invalidating the translation look-aside buffer.

gate1999 operating-system context-switch normal

Answer key

## 5.3.2 Context Switch: GATE CSE 2000 | Question: 1.20, ISRO2008-47



Which of the following need not necessarily be saved on a context switch between processes?

- A. General purpose registers
- B. Translation look-aside buffer
- C. Program counter
- D. All of the above

gatecse-2000 operating-system easy isro2008 context-switch

Answer key

## 5.3.3 Context Switch: GATE CSE 2011 | Question: 6, UGCNET-June2013-III: 62



Let the time taken to switch from user mode to kernel mode of execution be  $T_1$  while time taken to switch between two user processes be  $T_2$ . Which of the following is correct?

- A.  $T_1 > T_2$
- B.  $T_1 = T_2$
- C.  $T_1 < T_2$
- D. Nothing can be said about the relation between  $T_1$  and  $T_2$

gatecse-2011 operating-system context-switch easy ugcnetcse-june2013-paper3

Answer key

## 5.3.4 Context Switch: GATE CSE 2024 | Set 2 | Question: 15



Consider a process  $P$  running on a CPU. Which one or more of the following events will *a/ways* trigger a context switch by the OS that results in process  $P$  moving to a non-running state (e.g., ready, blocked)?

- A.  $P$  makes a blocking system call to read a block of data from the disk
- B.  $P$  tries to access a page that is in the swap space, triggering a page fault
- C. An interrupt is raised by the disk to deliver data requested by some other process
- D. A timer interrupt is raised by the hardware

## Answer key

## 5.4

## DMA (1)

## 5.4.1 DMA: GATE CSE 2001 | Question: 8



Consider a disk with the following specifications: 20 surfaces, 1000 tracks/surface, 16 sectors/track, data density 1 KB/sector, rotation speed 3000 rpm. The operating system initiates the transfer between the disk and the memory sector-wise. Once the head has been placed on the right track, the disk reads a sector in a single scan. It reads bits from the sector while the head is passing over the sector. The read bits are formed into bytes in a serial-in-parallel-out buffer and each byte is then transferred to memory. The disk writing is exactly a complementary process.

For parts (C) and (D) below, assume memory read-write time = 0.1 microseconds/byte, interrupt driven transfer has an interrupt overhead = 0.4 microseconds, the DMA initialization, and termination overhead is negligible compared to the total sector transfer time. DMA requests are always granted.

- What is the total capacity of the disk?
- What is the data transfer rate?
- What is the percentage of time the CPU is required for this disk I/O for byte-wise interrupts driven transfer?
- What is the maximum percentage of time the CPU is held up for this disk I/O for cycle-stealing DMA transfer?

gatecse-2001 operating-system disk normal descriptive dma

## Answer key

## 5.5

## Deadlock Prevention Avoidance Detection (4)

**Practice Test:** Test 1 (9Q)

## 5.5.1 Deadlock Prevention Avoidance Detection: GATE CSE 2018 | Question: 24



Consider a system with 3 processes that share 4 instances of the same resource type. Each process can request a maximum of  $K$  instances. Resources can be requested and releases only one at a time. The largest value of  $K$  that will always avoid deadlock is \_\_\_\_

gatecse-2018 operating-system deadlock-prevention-avoidance-detection easy numerical-answers one-mark

## Answer key

## 5.5.2 Deadlock Prevention Avoidance Detection: GATE CSE 2021 | Set 2 | Question: 43



Consider a computer system with multiple shared resource types, with one instance per resource type. Each instance can be owned by only one process at a time. Owning and freeing of resources are done by holding a global lock ( $L$ ). The following scheme is used to own a resource instance:

```
function OWNRESOURCE(Resource R)
  Acquire lock L // a global lock
  if R is available then
    Acquire R
    Release lock L
  else
    if R is owned by another process P then
      Terminate P, after releasing all resources owned by P
      Acquire R
      Restart P
      Release lock L
    end if
  end if
end function
```

Which of the following choice(s) about the above scheme is/are correct?

- The scheme ensures that deadlocks will not occur
- The scheme may lead to live-lock
- The scheme may lead to starvation
- The scheme violates the mutual exclusion property



Answer key

## 5.5.3 Deadlock Prevention Avoidance Detection: GATE CSE 2026 | Set 1 | Question: 25



Consider a system consisting of  $k$  instances of a resource  $R$ , being shared by 5 processes. Assume that each process requires a maximum of two instances of resource  $R$  and a process can request or release only one instance at a time. Further, a process can request the second instance of the resource only after acquiring the first instance.

The minimum value of  $k$  for the system to be deadlock-free is \_\_\_\_\_. (answer in integer)

Answer key

## 5.5.4 Deadlock Prevention Avoidance Detection: GATE IT 2004 | Question: 63



In a certain operating system, deadlock prevention is attempted using the following scheme. Each process is assigned a unique timestamp, and is restarted with the same timestamp if killed. Let  $P_h$  be the process holding a resource  $R$ ,  $P_r$  be a process requesting for the same resource  $R$ , and  $T(P_h)$  and  $T(P_r)$  be their timestamps respectively. The decision to wait or preempt one of the processes is based on the following algorithm.

```
if  $T(P_r) < T(P_h)$  then
    kill  $P_r$ 
else wait
```

Which one of the following is TRUE?

- A. The scheme is deadlock-free, but not starvation-free
- B. The scheme is not deadlock-free, but starvation-free
- C. The scheme is neither deadlock-free nor starvation-free
- D. The scheme is both deadlock-free and starvation-free

Answer key

## 5.6 Demand Paging (3)

## 5.6.1 Demand Paging: GATE CSE 2022 | Question: 54



Consider a demand paging system with four page frames (initially empty) and LRU page replacement policy. For the following page reference string

7, 2, 7, 3, 2, 5, 3, 4, 6, 7, 7, 1, 5, 6, 1

the page fault rate, defined as the ratio of number of page faults to the number of memory accesses (*rounded off to one decimal place*) is \_\_\_\_\_.

Answer key

## 5.6.2 Demand Paging: GATE CSE 2025 | Set 1 | Question: 4



Consider a demand paging memory management system with 32-bit logical address, 20-bit physical address, and page size of 2048 bytes. Assuming that the memory is byte addressable, what is the maximum number of entries in the page table?

- A.  $2^{21}$
- B.  $2^{20}$
- C.  $2^{22}$
- D.  $2^{24}$

Answer key

### 5.6.3 Demand Paging: GATE CSE 2025 | Set 2 | Question: 37



Consider a demand paging system with three frames, and the following page reference string: 1 2 3 4 5 4 1 6 4 5 1 3 2 . The contents of the frames are as follows initially and after each reference (from left to right):

| initially | after |    |    |    |    |   |   |    |   |   |    |    |    |
|-----------|-------|----|----|----|----|---|---|----|---|---|----|----|----|
| —         | 1*    | 2* | 3* | 4* | 5* | 4 | 1 | 6* | 4 | 5 | 1* | 3* | 2* |
| —         | 1     | 1  | 1  | 1  | 1  | 1 | 1 | 6  | 6 | 6 | 6  | 6  | 2  |
| —         | —     | 2  | 2  | 4  | 4  | 4 | 4 | 4  | 4 | 4 | 1  | 1  | 1  |
| —         | —     | —  | 3  | 3  | 5  | 5 | 5 | 5  | 5 | 5 | 5  | 3  | 3  |

The \*-marked references cause page replacements.

Which one or more of the following could be the page replacement policy/policies in use?

- A. Least Recently Used page replacement policy
- B. Least Frequently Used page replacement policy
- C. Most Frequently Used page replacement policy
- D. Optimal page replacement policy

gatecse2025-set2 operating-system page-replacement demand-paging multiple-selects two-marks

Answer key

### 5.7 Disk (30)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (2Q)

#### 5.7.1 Disk: GATE CSE 1990 | Question: 7-c



A certain moving arm disk-storage device has the following specifications:

- Number of tracks per surface = 404
- Track storage capacity = 130030 bytes.
- Disk speed = 3600 rpm
- Average seek time = 30 m secs.

Estimate the average latency, the disk storage capacity, and the data transfer rate.

gate1990 operating-system disk descriptive

Answer key

#### 5.7.2 Disk: GATE CSE 1993 | Question: 6.7



A certain moving arm disk storage, with one head, has the following specifications:

- Number of tracks/recording surface = 200
- Disk rotation speed = 2400 rpm
- Track storage capacity = 62,500 bits

The average latency of this device is P ms and the data transfer rate is Q bits/sec. Write the values of P and Q.

gate1993 operating-system disk normal descriptive

Answer key

#### 5.7.3 Disk: GATE CSE 1993 | Question: 7.8



The root directory of a disk should be placed

- A. at a fixed address in main memory
- B. at a fixed location on the disk
- C. anywhere on the disk
- D. at a fixed location on the system disk

E. anywhere on the system disk

gate1993 operating-system disk normal

Answer key

#### 5.7.4 Disk: GATE CSE 1995 | Question: 14



If the overhead for formatting a disk is 96 bytes for a 4000 byte sector,

- A. Compute the unformatted capacity of the disk for the following parameters:
- Number of surfaces: 8
  - Outer diameter of the disk: 12 cm
  - Inner diameter of the disk: 4 cm
  - Inner track space: 0.1 mm
  - Number of sectors per track: 20
- B. If the disk in (A) is rotating at 360 rpm, determine the effective data transfer rate which is defined as the number of bytes transferred per second between disk and memory.

gate1995 operating-system disk normal descriptive

Answer key

#### 5.7.5 Disk: GATE CSE 1996 | Question: 23



A file system with a one-level directory structure is implemented on a disk with disk block size of  $4K$  bytes. The disk is used as follows:

|              |                                                                                                                                             |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Disk-block 0 | File Allocation Table, consisting of one 8-bit entry per data block, representing the data block address of the next data block in the file |
| Disk-block 1 | Directory, with one 32 bit entry per file:                                                                                                  |
| Disk-block 2 | Data-block 1;                                                                                                                               |
| Disk-block 3 | Data-block 2; etc.                                                                                                                          |

- a. What is the maximum possible number of files?  
b. What is the maximum possible file size in blocks

gate1996 operating-system disk normal file-system descriptive

Answer key

#### 5.7.6 Disk: GATE CSE 1997 | Question: 74



A program  $P$  reads and processes 1000 consecutive records from a sequential file  $F$  stored on device  $D$  without using any file system facilities. Given the following

- Size of each record = 3200 bytes
- Access time of  $D$  = 10 msec
- Data transfer rate of  $D$  =  $800 \times 10^3$  bytes/second
- CPU time to process each record = 3 msec

What is the elapsed time of  $P$  if

- A.  $F$  contains unblocked records and  $P$  does not use buffering?  
B.  $F$  contains unblocked records and  $P$  uses one buffer (i.e., it always reads ahead into the buffer)?  
C. records of  $F$  are organized using a blocking factor of 2 (i.e., each block on  $D$  contains two records of  $F$ ) and  $P$  uses one buffer?

gate1997 operating-system disk

Answer key

### 5.7.7 Disk: GATE CSE 1998 | Question: 2-9



Formatting for a floppy disk refers to

- A. arranging the data on the disk in contiguous fashion
- B. writing the directory
- C. erasing the system data
- D. writing identification information on all tracks and sectors

gate1998 operating-system disk normal

Answer key

### 5.7.8 Disk: GATE CSE 1998 | Question: 25-a



Free disk space can be used to keep track of using a free list or a bit map. Disk addresses require  $d$  bits. For a disk with  $B$  blocks,  $F$  of which are free, state the condition under which the free list uses less space than the bit map.

gate1998 operating-system disk descriptive

Answer key

### 5.7.9 Disk: GATE CSE 1998 | Question: 25b



Consider a disk with  $c$  cylinders,  $t$  tracks per cylinder,  $s$  sectors per track and a sector length  $s_l$ . A logical file  $d_l$  with fixed record length  $r_l$  is stored continuously on this disk starting at location  $(c_L, t_L, s_L)$ , where  $c_L, t_L$  and  $s_L$  are the cylinder, track and sector numbers, respectively. Derive the formula to calculate the disk address (i.e. cylinder, track and sector) of a logical record  $n$  assuming that  $r_l = s_l$ .

gate1998 operating-system disk descriptive

Answer key

### 5.7.10 Disk: GATE CSE 1999 | Question: 2-18, ISRO2008-46



Raid configurations of the disks are used to provide

- A. Fault-tolerance
- B. High speed
- C. High data density
- D. (A) & (B)

gate1999 operating-system disk easy isro2008

Answer key

### 5.7.11 Disk: GATE CSE 2001 | Question: 1.22



Which of the following requires a device driver?

- A. Register
- B. Cache
- C. Main memory
- D. Disk

gatecse-2001 operating-system disk easy

Answer key

### 5.7.12 Disk: GATE CSE 2003 | Question: 25, ISRO2009-12



Using a larger block size in a fixed block size file system leads to

- A. better disk throughput but poorer disk space utilization
- B. better disk throughput and better disk space utilization
- C. poorer disk throughput but better disk space utilization
- D. poorer disk throughput and poorer disk space utilization

gatecse-2003 operating-system disk normal isro2009

Answer key

### 5.7.13 Disk: GATE CSE 2004 | Question: 49



A unix-style I-nodes has 10 direct pointers and one single, one double and one triple indirect pointers. Disk block size is 1 Kbyte, disk block address is 32 bits, and 48-bit integers are used. What is the maximum possible file size?

- A.  $2^{24}$  bytes                      B.  $2^{32}$  bytes                      C.  $2^{34}$  bytes                      D.  $2^{48}$  bytes

gatecse-2004    operating-system    disk    normal

Answer key

### 5.7.14 Disk: GATE CSE 2005 | Question: 21



What is the swap space in the disk used for?

- A. Saving temporary html pages                      B. Saving process data  
C. Storing the super-block                      D. Storing device drivers

gatecse-2005    operating-system    disk    easy

Answer key

### 5.7.15 Disk: GATE CSE 2007 | Question: 11, ISRO2009-36, ISRO2016-21



Consider a disk pack with 16 surfaces, 128 tracks per surface and 256 sectors per track. 512 bytes of data are stored in a bit serial manner in a sector. The capacity of the disk pack and the number of bits required to specify a particular sector in the disk are respectively:

- A. 256 Mbyte, 19 bits                      B. 256 Mbyte, 28 bits  
C. 512 Mbyte, 20 bits                      D. 64 Gbyte, 28 bits

gatecse-2007    operating-system    disk    normal    isro2016

Answer key

### 5.7.16 Disk: GATE CSE 2008 | Question: 32



For a magnetic disk with concentric circular tracks, the seek latency is not linearly proportional to the seek distance due to

- A. non-uniform distribution of requests  
B. arm starting and stopping inertia  
C. higher capacity of tracks on the periphery of the platter  
D. use of unfair arm scheduling policies

gatecse-2008    operating-system    disk    normal

Answer key

### 5.7.17 Disk: GATE CSE 2009 | Question: 51



A hard disk has 63 sectors per track, 10 platters each with 2 recording surfaces and 1000 cylinders. The address of a sector is given as a triple  $\langle c, h, s \rangle$ , where  $c$  is the cylinder number,  $h$  is the surface number and  $s$  is the sector number. Thus, the  $0^{th}$  sector is addresses as  $\langle 0, 0, 0 \rangle$ , the  $1^{st}$  sector as  $\langle 0, 0, 1 \rangle$ , and so on. The address  $\langle 400, 16, 29 \rangle$  corresponds to sector number:

- A. 505035                      B. 505036                      C. 505037                      D. 505038

gatecse-2009    operating-system    disk    normal

Answer key

### 5.7.18 Disk: GATE CSE 2009 | Question: 52



A hard disk has 63 sectors per track, 10 platters each with 2 recording surfaces and 1000 cylinders. The address of a sector is given as a triple  $\langle c, h, s \rangle$ , where  $c$  is the cylinder number,  $h$  is the surface number and  $s$  is the sector number. Thus, the  $0^{th}$  sector is addresses as  $\langle 0, 0, 0 \rangle$ , the  $1^{st}$  sector as  $\langle 0, 0, 1 \rangle$ , and so on

The address of the 1039<sup>th</sup> sector is

- A.  $\langle 0, 15, 31 \rangle$  B.  $\langle 0, 16, 30 \rangle$   
C.  $\langle 0, 16, 31 \rangle$  D.  $\langle 0, 17, 31 \rangle$

gatecse-2009 operating-system disk normal

Answer key

#### 5.7.19 Disk: GATE CSE 2011 | Question: 44



An application loads 100 libraries at startup. Loading each library requires exactly one disk access. The seek time of the disk to a random location is given as 10 ms. Rotational speed of disk is 6000 rpm. If all 100 libraries are loaded from random locations on the disk, how long does it take to load all libraries? (The time to transfer data from the disk block once the head has been positioned at the start of the block may be neglected.)

- A. 0.50 s B. 1.50 s C. 1.25 s D. 1.00 s

gatecse-2011 operating-system disk normal

Answer key

#### 5.7.20 Disk: GATE CSE 2012 | Question: 41



A file system with 300 GByte disk uses a file descriptor with 8 direct block addresses, 1 indirect block address and 1 doubly indirect block address. The size of each disk block is 128 Bytes and the size of each disk block address is 8 Bytes. The maximum possible file size in this file system is

- A. 3 KBytes B. 35 KBytes  
C. 280 KBytes D. dependent on the size of the disk

gatecse-2012 operating-system disk normal

Answer key

#### 5.7.21 Disk: GATE CSE 2013 | Question: 29



Consider a hard disk with 16 recording surfaces (0 – 15) having 16384 cylinders (0 – 16383) and each cylinder contains 64 sectors (0 – 63). Data storage capacity in each sector is 512 bytes. Data are organized cylinder-wise and the addressing format is  $\langle \text{cylinder no.}, \text{surface no.}, \text{sector no.} \rangle$ . A file of size 42797 KB is stored in the disk and the starting disk location of the file is  $\langle 1200, 9, 40 \rangle$ . What is the cylinder number of the last sector of the file, if it is stored in a contiguous manner?

- A. 1281 B. 1282 C. 1283 D. 1284

gatecse-2013 operating-system disk normal

Answer key

#### 5.7.22 Disk: GATE CSE 2014 | Set 2 | Question: 20



A FAT (file allocation table) based file system is being used and the total overhead of each entry in the FAT is 4 bytes in size. Given a  $100 \times 10^6$  bytes disk on which the file system is stored and data block size is  $10^3$  bytes, the maximum size of a file that can be stored on this disk in units of  $10^6$  bytes is \_\_\_\_\_.

gatecse-2014-set2 operating-system disk numerical-answers normal file-system

Answer key

#### 5.7.23 Disk: GATE CSE 2015 | Set 1 | Question: 48



Consider a disk pack with a seek time of 4 milliseconds and rotational speed of 10000 rotations per minute (RPM). It has 600 sectors per track and each sector can store 512 bytes of data. Consider a file stored in the disk. The file contains 2000 sectors. Assume that every sector access necessitates a seek, and the average rotational latency for accessing each sector is half of the time for one complete rotation. The total time (in milliseconds) needed to read the entire file is \_\_\_\_\_.

gatecse-2015-set1 operating-system disk normal numerical-answers

Answer key

#### 5.7.24 Disk: GATE CSE 2015 | Set 2 | Question: 49



Consider a typical disk that rotates at 15000 rotations per minute (RPM) and has a transfer rate of  $50 \times 10^6$  bytes/sec. If the average seek time of the disk is twice the average rotational delay and the controller's transfer time is 10 times the disk transfer time, the average time (in milliseconds) to read or write a 512-byte sector of the disk is \_\_\_\_\_.

gatecse-2015-set2 operating-system disk normal numerical-answers

Answer key

#### 5.7.25 Disk: GATE CSE 2024 | Set 1 | Question: 44



Consider a 512 GB hard disk with 32 storage surfaces. There are 4096 sectors per track and each sector holds 1024 bytes of data. The number of cylinders in the hard disk is \_\_\_\_\_.

gatecse-2024-set1 numerical-answers operating-system disk two-marks

Answer key

#### 5.7.26 Disk: GATE CSE 2024 | Set 2 | Question: 43



Consider a disk with the following specifications: rotation speed of 6000 RPM, average seek time of 5 milliseconds, 500 sectors/track, 512-byte sectors. A file has content stored in 3000 sectors located randomly on the disk. Assuming average rotational latency, the total time (in seconds, rounded off to 2 decimal places) to read the entire file from the disk is \_\_\_\_\_.

gatecse-2024-set2 numerical-answers operating-system disk two-marks

Answer key

#### 5.7.27 Disk: GATE IT 2005 | Question: 63



In a computer system, four files of size 11050 bytes, 4990 bytes, 5170 bytes and 12640 bytes need to be stored. For storing these files on disk, we can use either 100 byte disk blocks or 200 byte disk blocks (but can't mix block sizes). For each block used to store a file, 4 bytes of bookkeeping information also needs to be stored on the disk. Thus, the total space used to store a file is the sum of the space taken to store the file and the space taken to store the book keeping information for the blocks allocated for storing the file. A disk block can store either bookkeeping information for a file or data from a file, but not both.

What is the total space required for storing the files using 100 byte disk blocks and 200 byte disk blocks respectively?

- A. 35400 and 35800 bytes
- B. 35800 and 35400 bytes
- C. 35600 and 35400 bytes
- D. 35400 and 35600 bytes

gateit-2005 operating-system disk normal

Answer key

#### 5.7.28 Disk: GATE IT 2005 | Question: 81-a



A disk has 8 equidistant tracks. The diameters of the innermost and outermost tracks are 1 cm and 8 cm respectively. The innermost track has a storage capacity of 10 MB.

What is the total amount of data that can be stored on the disk if it is used with a drive that rotates it with

- I. Constant Linear Velocity
- II. Constant Angular Velocity?

- A. I. 80 MB; II. 2040 MB
- B. I. 2040 MB; II. 80 MB
- C. I. 80 MB; II. 360 MB
- D. I. 360 MB; II. 80 MB

gateit-2005 operating-system disk normal

Answer key

#### 5.7.29 Disk: GATE IT 2005 | Question: 81-b



A disk has 8 equidistant tracks. The diameters of the innermost and outermost tracks are 1 cm and 8 cm respectively. The innermost track has a storage capacity of 10 MB.

If the disk has 20 sectors per track and is currently at the end of the 5<sup>th</sup> sector of the inner-most track and the head can move at a speed of 10 meters/sec and it is rotating at constant angular velocity of 6000 RPM, how much time will it take to read 1 MB contiguous data starting from the sector 4 of the outer-most track?

- A. 13.5 ms                      B. 10 ms                      C. 9.5 ms                      D. 20 ms

gateit-2005   operating-system   disk   normal

Answer key

#### 5.7.30 Disk: GATE IT 2007 | Question: 44, ISRO2015-34



A hard disk system has the following parameters :

- Number of tracks = 500
- Number of sectors/track = 100
- Number of bytes /sector = 500
- Time taken by the head to move from one track to adjacent track = 1 ms
- Rotation speed = 600 rpm.

What is the average time taken for transferring 250 bytes from the disk ?

- A. 300.5 ms                      B. 255.5 ms                      C. 255 ms                      D. 300 ms

gateit-2007   operating-system   disk   normal   isro2015

Answer key

### 5.8 Disk Scheduling (16)

Practice Test: Test 1 (14Q)

#### 5.8.1 Disk Scheduling: GATE CSE 1989 | Question: 4-xii



Disk requests come to disk driver for cylinders 10, 22, 20, 2, 40, 6 and 38, in that order at a time when the disk drive is reading from cylinder 20. The seek time is 6 msec per cylinder. Compute the total seek time if the disk arm scheduling algorithm is.

- A. First come first served.  
B. Closest cylinder next.

gate1989   descriptive   operating-system   disk-scheduling

Answer key

#### 5.8.2 Disk Scheduling: GATE CSE 1990 | Question: 9b



Assuming the current disk cylinder to be 50 and the sequence for the cylinders to be 1, 36, 49, 65, 53, 12, 3, 20, 55, 16, 65 and 78 find the sequence of servicing using

1. Shortest seek time first (SSTF) and
2. Elevator disk scheduling policies.

gate1990   descriptive   operating-system   disk-scheduling

Answer key

#### 5.8.3 Disk Scheduling: GATE CSE 1995 | Question: 20



The head of a moving head disk with 100 tracks numbered 0 to 99 is currently serving a request at track 55. If the queue of requests kept in FIFO order is

10, 70, 75, 23, 65

which of the two disk scheduling algorithms FCFS (First Come First Served) and SSTF (Shortest Seek Time First) will require less head movement? Find the head movement for each of the algorithms.



Answer key

## 5.8.4 Disk Scheduling: GATE CSE 1997 | Question: 3.6



The correct matching for the following pairs is:

|                          |                 |
|--------------------------|-----------------|
| (A) Disk Scheduling      | (1) Round robin |
| (B) Batch Processing     | (2) SCAN        |
| (C) Time-sharing         | (3) LIFO        |
| (D) Interrupt processing | (4) FIFO        |

- A. A-3 B-4 C-2 D-1      B. A-4 B-3 C-2 D-1      C. A-2 B-4 C-1 D-3      D. A-3 B-4 C-3 D-2

gate1997 operating-system normal disk-scheduling match-the-following

Answer key

## 5.8.5 Disk Scheduling: GATE CSE 1999 | Question: 1.10



Which of the following disk scheduling strategies is likely to give the best throughput?

- A. Farthest cylinder next      B. Nearest cylinder next  
C. First come first served      D. Elevator algorithm

gate1999 operating-system disk-scheduling normal

Answer key

## 5.8.6 Disk Scheduling: GATE CSE 2001 | Question: 20



Consider a disk with the 100 tracks numbered from 0 to 99 rotating at 3000 rpm. The number of sectors per track is 100 and the time to move the head between two successive tracks is 0.2 millisecond.

- A. Consider a set of disk requests to read data from tracks 32, 7, 45, 5 and 10. Assuming that the elevator algorithm is used to schedule disk requests, and the head is initially at track 25 moving up (towards larger track numbers), what is the total seek time for servicing the requests?  
B. Consider an initial set of 100 arbitrary disk requests and assume that no new disk requests arrive while servicing these requests. If the head is initially at track 0 and the elevator algorithm is used to schedule disk requests, what is the worst case time to complete all the requests?

gatecse-2001 operating-system disk disk-scheduling normal descriptive

Answer key

## 5.8.7 Disk Scheduling: GATE CSE 2004 | Question: 12



Consider an operating system capable of loading and executing a single sequential user process at a time. The disk head scheduling algorithm used is First Come First Served (FCFS). If FCFS is replaced by Shortest Seek Time First (SSTF), claimed by the vendor to give 50% better benchmark results, what is the expected improvement in the I/O performance of user programs?

- A. 50%      B. 40%      C. 25%      D. 0%

gatecse-2004 operating-system disk-scheduling normal

Answer key

## 5.8.8 Disk Scheduling: GATE CSE 2009 | Question: 31



Consider a disk system with 100 cylinders. The requests to access the cylinders occur in following sequence:

4, 34, 10, 7, 19, 73, 2, 15, 6, 20

Assuming that the head is currently at cylinder 50, what is the time taken to satisfy all requests if it takes 1 ms to move from one cylinder to adjacent one and shortest seek time first policy is used?

A. 95 ms

B. 119 ms

C. 233 ms

D. 276 ms

gatecse-2009 operating-system disk-scheduling normal

Answer key

#### 5.8.9 Disk Scheduling: GATE CSE 2014 | Set 1 | Question: 19



Suppose a disk has 201 cylinders, numbered from 0 to 200. At some time the disk arm is at cylinder 100, and there is a queue of disk access requests for cylinders 30, 85, 90, 100, 105, 110, 135 and 145. If Shortest-Seek Time First (SSTF) is being used for scheduling the disk access, the request for cylinder 90 is serviced after servicing \_\_\_\_\_ number of requests.

gatecse-2014-set1 operating-system disk-scheduling numerical-answers normal

Answer key

#### 5.8.10 Disk Scheduling: GATE CSE 2015 | Set 1 | Question: 30



Suppose the following disk request sequence (track numbers) for a disk with 100 tracks is given:

45, 20, 90, 10, 50, 60, 80, 25, 70.

Assume that the initial position of the R/W head is on track 50. The additional distance that will be traversed by the R/W head when the Shortest Seek Time First (SSTF) algorithm is used compared to the SCAN (Elevator) algorithm (assuming that SCAN algorithm moves towards 100 when it starts execution) is \_\_\_\_\_ tracks.

gatecse-2015-set1 operating-system disk-scheduling normal numerical-answers

Answer key

#### 5.8.11 Disk Scheduling: GATE CSE 2016 | Set 1 | Question: 48



Cylinder a disk queue with requests for  $I/O$  to blocks on cylinders 47, 38, 121, 191, 87, 11, 92, 10. The C-LOOK scheduling algorithm is used. The head is initially at cylinder number 63, moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

gatecse-2016-set1 operating-system disk-scheduling normal numerical-answers

Answer key

#### 5.8.12 Disk Scheduling: GATE CSE 2018 | Question: 53



Consider a storage disk with 4 platters (numbered as 0, 1, 2 and 3), 200 cylinders (numbered as 0, 1, ..., 199), and 256 sectors per track (numbered as 0, 1, ..., 255). The following 6 disk requests of the form [sector number, cylinder number, platter number] are received by the disk controller at the same time:

[120, 72, 2], [180, 134, 1], [60, 20, 0], [212, 86, 3], [56, 116, 2], [118, 16, 1]

Currently head is positioned at sector number 100 of cylinder 80, and is moving towards higher cylinder numbers. The average power dissipation in moving the head over 100 cylinders is 20 milliwatts and for reversing the direction of the head movement once is 15 milliwatts. Power dissipation associated with rotational latency and switching of head between different platters is negligible.

The total power consumption in milliwatts to satisfy all of the above disk requests using the Shortest Seek Time First disk scheduling algorithm is \_\_\_\_\_

gatecse-2018 operating-system disk-scheduling numerical-answers two-marks

Answer key

#### 5.8.13 Disk Scheduling: GATE CSE 2020 | Question: 35



Consider the following five disk access requests of the form (request id, cylinder number) that are present in the disk scheduler queue at a given time.

(P, 155), (Q, 85), (R, 110), (S, 30), (T, 115)

Assume the head is positioned at cylinder 100. The scheduler follows Shortest Seek Time First scheduling to service the requests.

Which one of the following statements is FALSE?

- A.  $T$  is serviced before  $P$ .
- B.  $Q$  is serviced after  $S$ , but before  $T$ .
- C. The head reverses its direction of movement between servicing of  $Q$  and  $P$ .
- D.  $R$  is serviced before  $P$ .

gatecse-2020 operating-system disk-scheduling two-marks

Answer key

#### 5.8.14 Disk Scheduling: GATE IT 2004 | Question: 62



A disk has 200 tracks (numbered 0 through 199). At a given time, it was servicing the request of reading data from track 120, and at the previous request, service was for track 90. The pending requests (in order of their arrival) are for track numbers.

30 70 115 130 110 80 20 25.

How many times will the head change its direction for the disk scheduling policies SSTF (Shortest Seek Time First) and FCFS (First Come First Serve)?

- A. 2 and 3
- B. 3 and 3
- C. 3 and 4
- D. 4 and 4

gateit-2004 operating-system disk-scheduling normal

Answer key

#### 5.8.15 Disk Scheduling: GATE IT 2007 | Question: 82



The head of a hard disk serves requests following the shortest seek time first (SSTF) policy. The head is initially positioned at track number 180.

Which of the request sets will cause the head to change its direction after servicing every request assuming that the head does not change direction if there is a tie in SSTF and all the requests arrive before the servicing starts?

- A. 11, 139, 170, 178, 181, 184, 201, 265
- B. 10, 138, 170, 178, 181, 185, 201, 265
- C. 10, 139, 169, 178, 181, 184, 201, 265
- D. 10, 138, 170, 178, 181, 185, 200, 265

gateit-2007 operating-system disk-scheduling normal

Answer key

#### 5.8.16 Disk Scheduling: GATE IT 2007 | Question: 83



The head of a hard disk serves requests following the shortest seek time first (SSTF) policy.

What is the maximum cardinality of the request set, so that the head changes its direction after servicing every request if the total number of tracks are 2048 and the head can start from any track?

- A. 9
- B. 10
- C. 11
- D. 12

gateit-2007 operating-system disk-scheduling normal

Answer key

### 5.9 File System (7)

Practice Test: Test 1 (9Q)

#### 5.9.1 File System: GATE CSE 2002 | Question: 2.22



In the index allocation scheme of blocks to a file, the maximum possible size of the file depends on

- A. the size of the blocks, and the size of the address of the blocks.
- B. the number of blocks used for the index, and the size of the blocks.
- C. the size of the blocks, the number of blocks used for the index, and the size of the address of the blocks.
- D. None of the above

Answer key

## 5.9.2 File System: GATE CSE 2008 | Question: 20



The data blocks of a very large file in the Unix file system are allocated using

- A. continuous allocation
- B. linked allocation
- C. indexed allocation
- D. an extension of indexed allocation

gatecse-2008 file-system operating-system normal

Answer key

## 5.9.3 File System: GATE CSE 2017 | Set 2 | Question: 08



In a file allocation system, which of the following allocation scheme(s) can be used if no external fragmentation is allowed?

1. Contiguous
2. Linked
3. Indexed

- A. 1 and 3 only
- B. 2 only
- C. 3 only
- D. 2 and 3 only

gatecse-2017-set2 operating-system file-system normal

Answer key

## 5.9.4 File System: GATE CSE 2019 | Question: 42



The index node (inode) of a Unix-like file system has 12 direct, one single-indirect and one double-indirect pointers. The disk block size is 4 kB, and the disk block address is 32-bits long. The maximum possible file size is (rounded off to 1 decimal place) \_\_\_\_ GB

gatecse-2019 numerical-answers operating-system file-system two-marks

Answer key

## 5.9.5 File System: GATE CSE 2021 | Set 1 | Question: 15



Consider a linear list based directory implementation in a file system. Each directory is a list of nodes, where each node contains the file name along with the file metadata, such as the list of pointers to the data blocks. Consider a given directory **foo**.

Which of the following operations will necessarily require a full scan of **foo** for successful completion?

- A. Creation of a new file in **foo**
- B. Deletion of an existing file from **foo**
- C. Renaming of an existing file in **foo**
- D. Opening of an existing file in **foo**

gatecse-2021-set1 multiple-selects operating-system file-system one-mark

Answer key

## 5.9.6 File System: GATE CSE 2022 | Question: 53



Consider two files systems **A** and **B**, that use contiguous allocation and linked allocation, respectively. A file of size 100 blocks is already stored in **A** and also in **B**. Now, consider inserting a new block in the middle of the file (between  $50^{th}$  and  $51^{st}$  block), whose data is already available in the memory. Assume that there are enough free blocks at the end of the file and that the file control blocks are already in memory. Let the number of disk accesses required to insert a block in the middle of the file in **A** and **B** are  $n_A$  and  $n_B$ , respectively, then the value of  $n_A + n_B$  is \_\_\_\_\_.

gatecse-2022 numerical-answers operating-system file-system two-marks

Answer key

## 5.9.7 File System: GATE IT 2004 | Question: 67



In a particular Unix OS, each data block is of size 1024 bytes, each node has 10 direct data block

addresses and three additional addresses: one for single indirect block, one for double indirect block and one for triple indirect block. Also, each block can contain addresses for 128 blocks. Which one of the following is approximately the maximum size of a file in the file system?

- A. 512 MB                      B. 2 GB                      C. 8 GB                      D. 16 GB

gateit-2004    operating-system    file-system    normal

Answer key

## 5.10 Fork System Call (8)

 **Practice Test:** Test 1 (14Q)

### 5.10.1 Fork System Call: GATE CSE 2005 | Question: 72



Consider the following code fragment:

```
if (fork() == 0)
{
    a = a + 5;
    printf("%d, %p n", a, &a);
}
else
{
    a = a - 5;
    printf ("%d, %p n", a, &a);
}
```

Let  $u, v$  be the values printed by the parent process and  $x, y$  be the values printed by the child process. Which one of the following is **TRUE**?

- A.  $u = x + 10$  and  $v = y$                       B.  $u = x + 10$  and  $v! = y$   
C.  $u + 10 = x$  and  $v = y$                       D.  $u + 10 = x$  and  $v! = y$

gatecse-2005    operating-system    fork-system-call    normal

Answer key

### 5.10.2 Fork System Call: GATE CSE 2008 | Question: 66



A process executes the following code

```
for(i=0; i<n; i++) fork();
```

The total number of child processes created is

- A.  $n$                       B.  $2^n - 1$                       C.  $2^n$                       D.  $2^{n+1} - 1$

gatecse-2008    operating-system    fork-system-call    normal

Answer key

### 5.10.3 Fork System Call: GATE CSE 2012 | Question: 8



A process executes the code

```
fork();
fork();
fork();
```

The total number of child processes created is

- A. 3                      B. 4                      C. 7                      D. 8

gatecse-2012    operating-system    easy    fork-system-call

Answer key

### 5.10.4 Fork System Call: GATE CSE 2019 | Question: 17



The following C program is executed on a Unix/Linux system :

```
#include<unistd.h>
int main()
```

```

{
    int i;
    for(i=0; i<10; i++)
        if(i%2 == 0)
            fork();
    return 0;
}

```

The total number of child processes created is \_\_\_\_\_ .

gatecse-2019   numerical-answers   operating-system   fork-system-call   one-mark

Answer key

#### 5.10.5 Fork System Call: GATE CSE 2023 | Question: 13



Which one or more of the following options guarantee that a computer system will transition from user mode to kernel mode?

- A. Function Call                      B. malloc Call                      C. Page Fault                      D. System Call

gatecse-2023   operating-system   fork-system-call   multiple-selects   one-mark

Answer key

#### 5.10.6 Fork System Call: GATE CSE 2024 | Set 1 | Question: 47



Consider the following code snippet using the fork () and wait () system calls. Assume that the code compiles and runs correctly, and that the system calls run successfully without any errors.

```

int x=3;
while (x>0){
    fork ();
    printf("hello");
    wait (NULL) ;
    X-- ;
}

```

The total number of times the printf statement is executed is \_\_\_\_\_.

gatecse-2024-set1   numerical-answers   operating-system   fork-system-call   two-marks

Answer key

#### 5.10.7 Fork System Call: GATE CSE 2026 | Set 1 | Question: 53



Consider the following program snippet. Assume that the program compiles and runs successfully. Further, assume that the fork() system call is always successful in creating a process.

```

int main () {
    int i;
    for (i = 0; i < 3; i++) {
        if (fork() == 0) {
            continue;
        }
        break;
    }
    printf("Hello!");
    return 0;
}

```

The total number of times that the printf statement gets executed is \_\_\_\_\_. (answer in integer)

gatecse-2026-set1   numerical-answers   operating-system   fork-system-call   two-marks

Answer key

### 5.10.8 Fork System Call: GATE IT 2004 | Question: 64



A process executes the following segment of code :

```
for(i = 1; i <= n; i++)  
    fork ();
```

The number of new processes created is

- A.  $n$                       B.  $((n(n+1))/2)$                       C.  $2^n - 1$                       D.  $3^n - 1$

gateit-2004   operating-system   fork-system-call   easy

Answer key

### 5.11 IO Handling (7)

Practice Test: Test 1 (8Q)

#### 5.11.1 IO Handling: GATE CSE 1996 | Question: 1.20, ISRO2008-56



Which of the following is an example of spooled device?

- A. A line printer used to print the output of a number of jobs  
B. A terminal used to enter input data to a running program  
C. A secondary storage device in a virtual memory system  
D. A graphic display device

gate1996   operating-system   io-handling   normal   isro2008

Answer key

#### 5.11.2 IO Handling: GATE CSE 1998 | Question: 1.29



Which of the following is an example of a spooled device?

- A. The terminal used to enter the input data for the C program being executed  
B. An output device used to print the output of a number of jobs  
C. The secondary memory device in a virtual storage system  
D. The swapping area on a disk used by the swapper

gate1998   operating-system   io-handling   easy

Answer key

#### 5.11.3 IO Handling: GATE CSE 2005 | Question: 19



Which one of the following is true for a CPU having a single interrupt request line and a single interrupt grant line?

- A. Neither vectored interrupt nor multiple interrupting devices are possible  
B. Vectored interrupts are not possible but multiple interrupting devices are possible  
C. Vectored interrupts and multiple interrupting devices are both possible  
D. Vectored interrupts are possible but multiple interrupting devices are not possible

gatecse-2005   operating-system   io-handling   normal

Answer key

#### 5.11.4 IO Handling: GATE CSE 2005 | Question: 20



Normally user programs are prevented from handling I/O directly by I/O instructions in them. For CPUs having explicit I/O instructions, such I/O protection is ensured by having the I/O instruction privileged. In a

CPU with memory mapped I/O, there is no explicit I/O instruction. Which one of the following is true for a CPU with memory mapped I/O?

- A. I/O protection is ensured by operating system routine(s)
- B. I/O protection is ensured by a hardware trap
- C. I/O protection is ensured during system configuration
- D. I/O protection is not possible

gatecse-2005 operating-system io-handling normal

Answer key

#### 5.11.5 IO Handling: GATE CSE 2018 | Question: 9



The following are some events that occur after a device controller issues an interrupt while process  $L$  is under execution.

- P. The processor pushes the process status of  $L$  onto the control stack
- Q. The processor finishes the execution of the current instruction
- R. The processor executes the interrupt service routine
- S. The processor pops the process status of  $L$  from the control stack
- T. The processor loads the new PC value based on the interrupt

Which of the following is the correct order in which the events above occur?

- A. QPTRS
- B. PTRSQ
- C. TRPQS
- D. QTPRS

gatecse-2018 operating-system interrupts normal one-mark co-and-architecture io-handling

Answer key

#### 5.11.6 IO Handling: GATE IT 2004 | Question: 11, ISRO2011-33



What is the bit rate of a video terminal unit with 80 characters/line, 8 bits/character and horizontal sweep time of  $100\ \mu\text{s}$  (including  $20\ \mu\text{s}$  of retrace time)?

- A. 8 Mbps
- B. 6.4 Mbps
- C. 0.8 Mbps
- D. 0.64 Mbps

gateit-2004 operating-system io-handling easy isro2011

Answer key

#### 5.11.7 IO Handling: GATE IT 2006 | Question: 8



Which of the following DMA transfer modes and interrupt handling mechanisms will enable the highest I/O band-width?

- A. Transparent DMA and Polling interrupts
- B. Cycle-stealing and Vectored interrupts
- C. Block transfer and Vectored interrupts
- D. Block transfer and Polling interrupts

gateit-2006 operating-system io-handling dma normal

Answer key

### 5.12

### Input Output (1)

#### 5.12.1 Input Output: GATE CSE 2025 | Set 1 | Question: 19



Suppose in a multiprogramming environment, the following C program segment is executed. A process goes into I/O queue whenever an I/O related operation is performed. Assume that there will always be a context switch whenever a process requests for an I/O, and also whenever the process returns from an I/O. The number of times the process will enter the ready queue during its lifetime (not counting the time the process enters the ready queue when it is run initially) is \_\_\_\_\_. Answer in integer)

```
int main()
{
```



```
int x=0,i=0;
scanf("%d",&x);
for(i=0; i<20; i++)
{
    x = x+20;
    printf("%d\n",x);
}
return 0;
}
```

gatecse2025-set1   operating-system   process-scheduling   input-output   numerical-answers   one-mark

Answer key

5.13

## Inter Process Communication (1)

### 5.13.1 Inter Process Communication: GATE CSE 1997 | Question: 3.7



I/O redirection

- A. implies changing the name of a file
- B. can be employed to use an existing file as input file for a program
- C. implies connecting 2 programs through a pipe
- D. None of the above

gate1997   operating-system   normal   inter-process-communication

Answer key

5.14

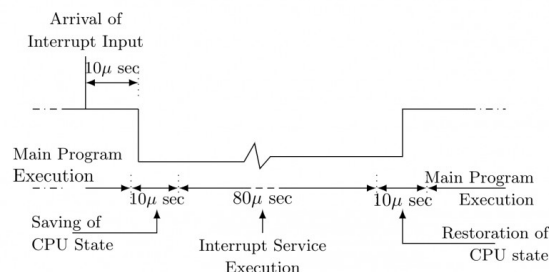
## Interrupts (6)

**Practice Test:** Test 1 (6Q)

### 5.14.1 Interrupts: GATE CSE 1993 | Question: 6.8



The details of an interrupt cycle are shown in figure.



Given that an interrupt input arrives every 1 msec, what is the percentage of the total time that the CPU devotes for the main program execution.

gate1993   operating-system   interrupts   normal   descriptive

Answer key

### 5.14.2 Interrupts: GATE CSE 1997 | Question: 3.8



When an interrupt occurs, an operating system

- A. ignores the interrupt
- B. always changes state of interrupted process after processing the interrupt
- C. always resumes execution of interrupted process after processing the interrupt
- D. may change state of interrupted process to 'blocked' and schedule another process.

gate1997   operating-system   interrupts   normal

Answer key

### 5.14.3 Interrupts: GATE CSE 1998 | Question: 1.18



Which of the following devices should get higher priority in assigning interrupts?

- A. Hard disk                      B. Printer                      C. Keyboard                      D. Floppy disk

gate1998   operating-system   interrupts   normal

Answer key

### 5.14.4 Interrupts: GATE CSE 1999 | Question: 1.9



Listed below are some operating system abstractions (in the left column) and the hardware components (in the right column)

|                           |              |
|---------------------------|--------------|
| (A) Thread                | 1. Interrupt |
| (B) Virtual address space | 2. Memory    |
| (C) File system           | 3. CPU       |
| (D) Signal                | 4. Disk      |

- A. (A) – 2 (B) – 4 (C) – 3 (D) – 1  
B. (A) – 1 (B) – 2 (C) – 3 (D) – 4  
C. (A) – 3 (B) – 2 (C) – 4 (D) – 1  
D. (A) – 4 (B) – 1 (C) – 2 (D) – 3

gate1999   operating-system   easy   interrupts   virtual-memory   disk

Answer key

### 5.14.5 Interrupts: GATE CSE 2001 | Question: 1.12



A processor needs software interrupt to

- A. test the interrupt system of the processor  
B. implement co-routines  
C. obtain system services which need execution of privileged instructions  
D. return from subroutine

gatecse-2001   operating-system   interrupts   easy

Answer key

### 5.14.6 Interrupts: GATE CSE 2011 | Question: 11



A computer handles several interrupt sources of which of the following are relevant for this question.

- Interrupt from CPU temperature sensor (raises interrupt if CPU temperature is too high)
- Interrupt from Mouse (raises Interrupt if the mouse is moved or a button is pressed)
- Interrupt from Keyboard (raises Interrupt if a key is pressed or released)
- Interrupt from Hard Disk (raises Interrupt when a disk read is completed)

Which one of these will be handled at the **HIGHEST** priority?

1. Interrupt from Hard Disk  
2. Interrupt from Mouse  
3. Interrupt from Keyboard  
4. Interrupt from CPU temperature sensor

gatecse-2011   operating-system   interrupts   normal

Answer key

### 5.15.1 Least Recently Used: GATE CSE 2023 | Question: 47



Consider the following two-dimensional array  $D$  in the C programming language, which is stored in row-major order:

```
int D[128][128];
```

Demand paging is used for allocating memory and each physical page frame holds 512 elements of the array  $D$ . The Least Recently Used (LRU) page-replacement policy is used by the operating system. A total of 30 physical page frames are allocated to a process which executes the following code snippet:

```
for (int i = 0; i < 128; i++)  
    for (int j = 0; j < 128; j++)  
        D[j][i] *= 10;
```

The number of page faults generated during the execution of this code snippet is \_\_\_\_\_.

gatecse-2023 operating-system page-replacement least-recently-used page-faults numerical-answers two-marks

Answer key

## 5.16

### Linked Allocation (1)

#### 5.16.1 Linked Allocation: GATE CSE 2025 | Set 1 | Question: 41



A disk of size 512 M bytes is divided into blocks of 64 K bytes. A file is stored in the disk using linked allocation. In linked allocation, each data block reserves 4 bytes to store the pointer to the next data block. The link part of the last data block contains a NULL pointer (also of 4 bytes). Suppose a file of 1 M bytes needs to be stored in the disk. Assume,  $1\text{ K} = 2^{10}$  and  $1\text{ M} = 2^{20}$ . The amount of space in bytes that will be wasted due to internal fragmentation is \_\_\_\_\_. (Answer in integer)

gatecse2025-set1 operating-system linked-allocation internal-fragmentation numerical-answers two-marks

Answer key

## 5.17

### Memory Management (9)

**Practice Tests:** [Test 1 \(15Q\)](#) [Test 2 \(2Q\)](#)

#### 5.17.1 Memory Management: GATE CSE 1992 | Question: 12-b



Let the page reference and the working set window be  $ccdbcecead$  and 4, respectively. The initial working set at time  $t = 0$  contains the pages  $\{a, d, e\}$ , where  $a$  was referenced at time  $t = 0$ ,  $d$  was referenced at time  $t = -1$ , and  $e$  was referenced at time  $t = -2$ . Determine the total number of page faults and the average number of page frames used by computing the working set at each reference.

gate1992 operating-system memory-management normal descriptive

Answer key

#### 5.17.2 Memory Management: GATE CSE 1995 | Question: 5



A computer installation has 1000k of main memory. The jobs arrive and finish in the following sequences.

```
Job 1 requiring 200k arrives  
Job 2 requiring 350k arrives  
Job 3 requiring 300k arrives  
Job 1 finishes  
Job 4 requiring 120k arrives  
Job 5 requiring 150k arrives  
Job 6 requiring 80k arrives
```

- Draw the memory allocation table using Best Fit and First Fit algorithms.
- Which algorithm performs better for this sequence?

gate1995 operating-system memory-management normal descriptive

Answer key

### 5.17.3 Memory Management: GATE CSE 1996 | Question: 2.18



A 1000 Kbyte memory is managed using variable partitions but no compaction. It currently has two partitions of sizes 200 Kbyte and 260 Kbyte respectively. The smallest allocation request in Kbyte that could be denied is for

- A. 151                      B. 181                      C. 231                      D. 541

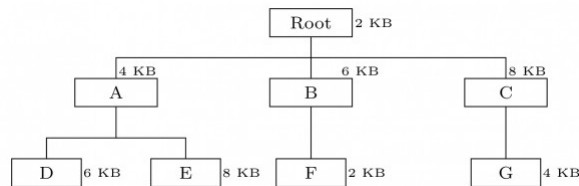
gate1996   operating-system   memory-management   normal

Answer key

### 5.17.4 Memory Management: GATE CSE 1998 | Question: 2.16



The overlay tree for a program is as shown below:



What will be the size of the partition (in physical memory) required to load (and run) this program?

- A. 12 KB                      B. 14 KB                      C. 10 KB                      D. 8 KB

gate1998   operating-system   normal   memory-management

Answer key

### 5.17.5 Memory Management: GATE CSE 2014 | Set 2 | Question: 55



Consider the main memory system that consists of 8 memory modules attached to the system bus, which is one word wide. When a write request is made, the bus is occupied for 100 nanoseconds (ns) by the data, address, and control signals. During the same 100 ns, and for 500 ns thereafter, the addressed memory module executes one cycle accepting and storing the data. The (internal) operation of different memory modules may overlap in time, but only one request can be on the bus at any time. The maximum number of stores (of one word each) that can be initiated in 1 millisecond is \_\_\_\_\_

gatecse-2014-set2   operating-system   memory-management   numerical-answers   normal

Answer key

### 5.17.6 Memory Management: GATE CSE 2015 | Set 2 | Question: 30



Consider 6 memory partitions of sizes 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB, where KB refers to kilobyte. These partitions need to be allotted to four processes of sizes 357 KB, 210 KB, 468 KB, 491 KB in that order. If the best-fit algorithm is used, which partitions are NOT allotted to any process?

- A. 200 KB and 300 KB                      B. 200 KB and 250 KB  
C. 250 KB and 300 KB                      D. 300 KB and 400 KB

gatecse-2015-set2   operating-system   memory-management   easy

Answer key

### 5.17.7 Memory Management: GATE CSE 2020 | Question: 11



Consider allocation of memory to a new process. Assume that none of the existing holes in the memory will exactly fit the process's memory requirement. Hence, a new hole of smaller size will be created if allocation is made in any of the existing holes. Which one of the following statement is TRUE?

- A. The hole created by first fit is always larger than the hole created by next fit.  
B. The hole created by worst fit is always larger than the hole created by first fit.  
C. The hole created by best fit is never larger than the hole created by first fit.  
D. The hole created by next fit is never larger than the hole created by best fit.

gatecse-2020   operating-system   memory-management   one-mark

Answer key

### 5.17.8 Memory Management: GATE IT 2006 | Question: 56



For each of the four processes  $P_1, P_2, P_3$ , and  $P_4$ . The total size in kilobytes (KB) and the number of segments are given below.

| Process | Total size (in KB) | Number of segments |
|---------|--------------------|--------------------|
| $P_1$   | 195                | 4                  |
| $P_2$   | 254                | 5                  |
| $P_3$   | 45                 | 3                  |
| $P_4$   | 364                | 8                  |

The page size is 1 KB. The size of an entry in the page table is 4 bytes. The size of an entry in the segment table is 8 bytes. The maximum size of a segment is 256 KB. The paging method for memory management uses two-level paging, and its storage overhead is  $P$ . The storage overhead for the segmentation method is  $S$ . The storage overhead for the segmentation and paging method is  $T$ . What is the relation among the overheads for the different methods of memory management in the concurrent execution of the above four processes?

- A.  $P < S < T$   
B.  $S < P < T$   
C.  $S < T < P$   
D.  $T < S < P$

gateit-2006 operating-system memory-management difficult

Answer key

### 5.17.9 Memory Management: GATE IT 2007 | Question: 11



Let a memory have four free blocks of sizes  $4k, 8k, 20k, 2k$ . These blocks are allocated following the best-fit strategy. The allocation requests are stored in a queue as shown below.

| Request No    | J1 | J2  | J3 | J4 | J5 | J6  | J7 | J8  |
|---------------|----|-----|----|----|----|-----|----|-----|
| Request Sizes | 2k | 14k | 3k | 6k | 6k | 10k | 7k | 20k |
| Usage Time    | 4  | 10  | 2  | 8  | 4  | 1   | 8  | 6   |

The time at which the request for  $J7$  will be completed will be

- A. 16  
B. 19  
C. 20  
D. 37

gateit-2007 operating-system memory-management normal

Answer key

## 5.18 Multilevel Paging (1)

Practice Test: Test 1 (10Q)

### 5.18.1 Multilevel Paging: GATE CSE 2025 | Set 2 | Question: 48



A computer system supports a logical address space of  $2^{32}$  bytes. It uses two-level hierarchical paging with a page size of 4096 bytes. A logical address is divided into a  $b$ -bit index to the outer page table, an offset within the page of the inner page table, and an offset within the desired page. Each entry of the inner page table uses eight bytes. All the pages in the system have the same size.

The value of  $b$  is \_\_\_\_\_. (Answer in integer)

gatecse2025-set2 operating-system paging multilevel-paging numerical-answers two-marks

Answer key

## 5.19 OS Protection (3)

### 5.19.1 OS Protection: GATE CSE 1999 | Question: 1.11, UGCNET-Dec2015-II: 44



System calls are usually invoked by using

- A. a software interrupt
- C. an indirect jump

- B. polling
- D. a privileged instruction

gate1999 operating-system normal ugcnetcse-dec2015-paper2 os-protection

Answer key

### 5.19.2 OS Protection: GATE CSE 2001 | Question: 1.13



A CPU has two modes -- privileged and non-privileged. In order to change the mode from privileged to non-privileged

- A. a hardware interrupt is needed
- B. a software interrupt is needed
- C. a privileged instruction (which does not generate an interrupt) is needed
- D. a non-privileged instruction (which does not generate an interrupt) is needed

gatecse-2001 operating-system normal os-protection

Answer key

### 5.19.3 OS Protection: GATE IT 2005 | Question: 19, UGCNET-June2012-III: 57



A user level process in Unix traps the signal sent on a Ctrl-C input, and has a signal handling routine that saves appropriate files before terminating the process. When a Ctrl-C input is given to this process, what is the mode in which the signal handling routine executes?

- A. User mode
- B. Kernel mode
- C. Superuser mode
- D. Privileged mode

gateit-2005 operating-system os-protection normal ugcnetcse-june2012-paper3

Answer key

## 5.20

## Page Replacement (31)

**Practice Tests:** [Test 1 \(15Q\)](#) [Test 2 \(15Q\)](#) [Test 3 \(11Q\)](#)

### 5.20.1 Page Replacement: GATE CSE 1993 | Question: 21



The following page addresses, in the given sequence, were generated by a program:

1 2 3 4 1 3 5 2 1 5 4 3 2 3

This program is run on a demand paged virtual memory system, with main memory size equal to 4 pages. Indicate the page references for which page faults occur for the following page replacement algorithms.

- A. LRU
- B. FIFO

Assume that the main memory is initially empty.

gate1993 operating-system page-replacement normal descriptive

Answer key

### 5.20.2 Page Replacement: GATE CSE 1994 | Question: 1.13



A memory page containing a heavily used variable that was initialized very early and is in constant use is removed then

- A. LRU page replacement algorithm is used
- B. FIFO page replacement algorithm is used
- C. LFU page replacement algorithm is used
- D. None of the above

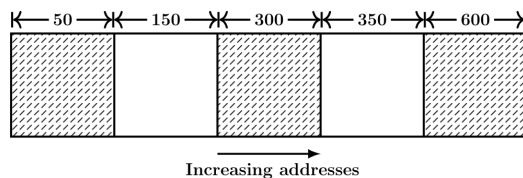
gate1994 operating-system page-replacement easy

Answer key

### 5.20.3 Page Replacement: GATE CSE 1994 | Question: 1.24



Consider the following heap (figure) in which blank regions are not in use and hatched region are in use.



The sequence of requests for blocks of sizes 300, 25, 125, 50 can be satisfied if we use

- A. either first fit or best fit policy (any one)
- B. first fit but not best fit policy
- C. best fit but not first fit policy
- D. None of the above

gate1994 operating-system page-replacement normal

Answer key

#### 5.20.4 Page Replacement: GATE CSE 1995 | Question: 1.8



Which of the following page replacement algorithms suffers from Belady's anomaly?

- A. Optimal replacement
- B. LRU
- C. FIFO
- D. Both (A) and (C)

gate1995 operating-system page-replacement normal

Answer key

#### 5.20.5 Page Replacement: GATE CSE 1995 | Question: 2.7



The address sequence generated by tracing a particular program executing in a pure demand based paging system with 100 records per page with 1 free main memory frame is recorded as follows. What is the number of page faults?

0100, 0200, 0430, 0499, 0510, 0530, 0560, 0120, 0220, 0240, 0260, 0320, 0370

- A. 13
- B. 8
- C. 7
- D. 10

gate1995 operating-system page-replacement normal

Answer key

#### 5.20.6 Page Replacement: GATE CSE 1997 | Question: 3.10, ISRO2008-57, ISRO2015-64



Dirty bit for a page in a page table

- A. helps avoid unnecessary writes on a paging device
- B. helps maintain LRU information
- C. allows only read on a page
- D. None of the above

gate1997 operating-system page-replacement easy isro2008 isro2015

Answer key

#### 5.20.7 Page Replacement: GATE CSE 1997 | Question: 3.5



Locality of reference implies that the page reference being made by a process

- A. will always be to the page used in the previous page reference
- B. is likely to be to one of the pages used in the last few page references
- C. will always be to one of the pages existing in memory
- D. will always lead to a page fault

gate1997 operating-system page-replacement easy

Answer key

#### 5.20.8 Page Replacement: GATE CSE 1997 | Question: 3.9



Thrashing

- A. reduces page I/O
- B. decreases the degree of

C. implies excessive page I/O

multiprogramming  
D. improve the system performance

gate1997 operating-system page-replacement easy

Answer key

#### 5.20.9 Page Replacement: GATE CSE 2001 | Question: 1.21



Consider a virtual memory system with FIFO page replacement policy. For an arbitrary page access pattern, increasing the number of page frames in main memory will

- A. always decrease the number of page faults
- B. always increase the number of page faults
- C. sometimes increase the number of page faults
- D. never affect the number of page faults

gatecse-2001 operating-system page-replacement normal

Answer key

#### 5.20.10 Page Replacement: GATE CSE 2002 | Question: 1.23



The optimal page replacement algorithm will select the page that

- A. Has not been used for the longest time in the past
- B. Will not be used for the longest time in the future
- C. Has been used least number of times
- D. Has been used most number of times

gatecse-2002 operating-system page-replacement easy

Answer key

#### 5.20.11 Page Replacement: GATE CSE 2004 | Question: 21, ISRO2007-44



The minimum number of page frames that must be allocated to a running process in a virtual memory environment is determined by

- A. the instruction set architecture
- B. page size
- C. number of processes in memory
- D. physical memory size

gatecse-2004 operating-system virtual-memory page-replacement normal isro2007

Answer key

#### 5.20.12 Page Replacement: GATE CSE 2005 | Question: 22, ISRO2015-36



Increasing the RAM of a computer typically improves performance because:

- A. Virtual Memory increases
- B. Larger RAMs are faster
- C. Fewer page faults occur
- D. Fewer segmentation faults occur

gatecse-2005 operating-system page-replacement easy isro2015

Answer key

#### 5.20.13 Page Replacement: GATE CSE 2007 | Question: 56



A virtual memory system uses First In First Out (FIFO) page replacement policy and allocates a fixed number of frames to a process. Consider the following statements:

**P:** Increasing the number of page frames allocated to a process sometimes increases the page fault rate.

**Q:** Some programs do not exhibit locality of reference.

Which one of the following is TRUE?

- A. Both P and Q are true, and Q is the reason for P
- B. Both P and Q are true, but Q is not the reason for P.
- C. P is false but Q is true
- D. Both P and Q are false.

gatecse-2007 operating-system page-replacement normal

Answer key



#### 5.20.14 Page Replacement: GATE CSE 2007 | Question: 82



A process has been allocated 3 page frames. Assume that none of the pages of the process are available in the memory initially. The process makes the following sequence of page references (reference string):  
**1, 2, 1, 3, 7, 4, 5, 6, 3, 1**

If optimal page replacement policy is used, how many page faults occur for the above reference string?

- A. 7                                      B. 8                                      C. 9                                      D. 10

gatecse-2007    operating-system    page-replacement    easy

Answer key

#### 5.20.15 Page Replacement: GATE CSE 2007 | Question: 83



A process, has been allocated 3 page frames. Assume that none of the pages of the process are available in the memory initially. The process makes the following sequence of page references (reference string):  
**1, 2, 1, 3, 7, 4, 5, 6, 3, 1**

Least Recently Used (LRU) page replacement policy is a practical approximation to optimal page replacement. For the above reference string, how many more page faults occur with LRU than with the optimal page replacement policy?

- A. 0                                      B. 1                                      C. 2                                      D. 3

gatecse-2007    normal    operating-system    page-replacement

Answer key

#### 5.20.16 Page Replacement: GATE CSE 2009 | Question: 9, ISRO2016-52



In which one of the following page replacement policies, Belady's anomaly may occur?

- A. FIFO                                      B. Optimal                                      C. LRU                                      D. MRU

gatecse-2009    operating-system    page-replacement    easy    isro2016

Answer key

#### 5.20.17 Page Replacement: GATE CSE 2010 | Question: 24



A system uses FIFO policy for system replacement. It has 4 page frames with no pages loaded to begin with. The system first accesses 100 distinct pages in some order and then accesses the same 100 pages but now in the reverse order. How many page faults will occur?

- A. 196                                      B. 192                                      C. 197                                      D. 195

gatecse-2010    operating-system    page-replacement    normal

Answer key

#### 5.20.18 Page Replacement: GATE CSE 2012 | Question: 42



Consider the virtual page reference string

**1, 2, 3, 2, 4, 1, 3, 2, 4, 1**

on a demand paged virtual memory system running on a computer system that has main memory size of 3 page frames which are initially empty. Let LRU, FIFO and OPTIMAL denote the number of page faults under the corresponding page replacement policy. Then

- A. OPTIMAL < LRU < FIFO                                      B. OPTIMAL < FIFO < LRU  
C. OPTIMAL = LRU                                      D. OPTIMAL = FIFO

gatecse-2012    operating-system    page-replacement    normal

Answer key

#### 5.20.19 Page Replacement: GATE CSE 2014 | Set 1 | Question: 33



Assume that there are 3 page frames which are initially empty. If the page reference string is

1, 2, 3, 4, 2, 1, 5, 3, 2, 4, 6 the number of page faults using the optimal replacement policy is \_\_\_\_\_.

gatecse-2014-set1 operating-system page-replacement numerical-answers

Answer key

#### 5.20.20 Page Replacement: GATE CSE 2014 | Set 2 | Question: 33



A computer has twenty physical page frames which contain pages numbered 101 through 120. Now a program accesses the pages numbered 1, 2, ..., 100 in that order, and repeats the access sequence **THRICE**. Which one of the following page replacement policies experiences the same number of page faults as the optimal page replacement policy for this program?

- A. Least-recently-used
- B. First-in-first-out
- C. Last-in-first-out
- D. Most-recently-used

gatecse-2014-set2 operating-system page-replacement ambiguous

Answer key

#### 5.20.21 Page Replacement: GATE CSE 2014 | Set 3 | Question: 20



A system uses 3 page frames for storing process pages in main memory. It uses the Least Recently Used (**LRU**) page replacement policy. Assume that all the page frames are initially empty. What is the total number of page faults that will occur while processing the page reference string given below?

4, 7, 6, 1, 7, 6, 1, 2, 7, 2

gatecse-2014-set3 operating-system page-replacement numerical-answers normal

Answer key

#### 5.20.22 Page Replacement: GATE CSE 2015 | Set 1 | Question: 47



Consider a main memory with five-page frames and the following sequence of page references: 3, 8, 2, 3, 9, 1, 6, 3, 8, 9, 3, 6, 2, 1, 3. Which one of the following is true with respect to page replacement policies First In First Out (FIFO) and Least Recently Used (LRU)?

- A. Both incur the same number of page faults
- B. FIFO incurs 2 more page faults than LRU
- C. LRU incurs 2 more page faults than FIFO
- D. FIFO incurs 1 more page faults than LRU

gatecse-2015-set1 operating-system page-replacement normal

Answer key

#### 5.20.23 Page Replacement: GATE CSE 2016 | Set 1 | Question: 49



Consider a computer system with ten physical page frames. The system is provided with an access sequence  $(a_1, a_2, \dots, a_{20}, a_1, a_2, \dots, a_{20})$ , where each  $a_i$  is a distinct virtual page number. The difference in the number of page faults between the last-in-first-out page replacement policy and the optimal page replacement policy is \_\_\_\_\_.

gatecse-2016-set1 operating-system page-replacement normal numerical-answers

Answer key

#### 5.20.24 Page Replacement: GATE CSE 2016 | Set 2 | Question: 20



In which one of the following page replacement algorithms it is possible for the page fault rate to increase even when the number of allocated frames increases?

- A. LRU (Least Recently Used)
- B. OPT (Optimal Page Replacement)
- C. MRU (Most Recently Used)
- D. FIFO (First In First Out)

gatecse-2016-set2 operating-system page-replacement easy

Answer key

### 5.20.25 Page Replacement: GATE CSE 2017 | Set 1 | Question: 40



Recall that Belady's anomaly is that the page-fault rate may *increase* as the number of allocated frames increases. Now, consider the following statements:

- $S_1$ : Random page replacement algorithm (where a page chosen at random is replaced) suffers from Belady's anomaly.
- $S_2$ : LRU page replacement algorithm suffers from Belady's anomaly.

Which of the following is CORRECT?

- A.  $S_1$  is true,  $S_2$  is true
- B.  $S_1$  is true,  $S_2$  is false
- C.  $S_1$  is false,  $S_2$  is true
- D.  $S_1$  is false,  $S_2$  is false

gatecse-2017-set1 page-replacement operating-system normal

Answer key

### 5.20.26 Page Replacement: GATE CSE 2021 | Set 1 | Question: 11



In the context of operating systems, which of the following statements is/are correct with respect to paging?

- A. Paging helps solve the issue of external fragmentation
- B. Page size has no impact on internal fragmentation
- C. Paging incurs memory overheads
- D. Multi-level paging is necessary to support pages of different sizes

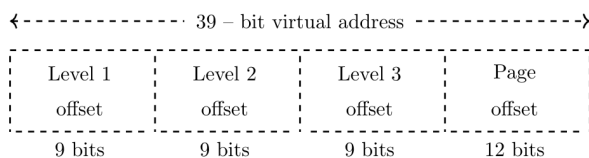
gatecse-2021-set1 multiple-selects operating-system page-replacement one-mark

Answer key

### 5.20.27 Page Replacement: GATE CSE 2021 | Set 2 | Question: 48



Consider a three-level page table to translate a 39-bit virtual address as shown below:



The page size is 4 KB ( $1\text{KB} = 2^{10}$  bytes) and page table entry size at every level is 8 bytes. A process  $P$  is currently using 2GB ( $1\text{GB} = 2^{30}$  bytes) virtual memory which is mapped to 2GB of physical memory. The minimum amount of memory required for the page table of  $P$  across all levels is \_\_\_\_\_ KB.

gatecse-2021-set2 numerical-answers operating-system memory-management page-replacement two-marks

Answer key

### 5.20.28 Page Replacement: GATE CSE 2025 | Set 1 | Question: 44



In optimal page replacement algorithm, information about all future page references is available to the operating system (OS). A modification of the optimal page replacement algorithm is as follows:

The OS correctly predicts only up to next 4 page references (including the current page) at the time of allocating a frame to a page.

A process accesses the pages in the following order of page numbers:

1, 3, 2, 4, 2, 3, 1, 2, 4, 3, 1, 4

If the system has three memory frames that are initially empty, the number of page faults that will occur during execution of the process is \_\_\_\_\_. (Answer in integer)

gatecse2025-set1 operating-system page-replacement numerical-answers two-marks

Answer key

#### 5.20.29 Page Replacement: GATE IT 2007 | Question: 12



The address sequence generated by tracing a particular program executing in a pure demand paging system with 100 bytes per page is

0100, 0200, 0430, 0499, 0510, 0530, 0560, 0120, 0220, 0240, 0260, 0320, 0410.

Suppose that the memory can store only one page and if  $x$  is the address which causes a page fault then the bytes from addresses  $x$  to  $x + 99$  are loaded on to the memory.

How many page faults will occur?

- A. 0                                      B. 4                                      C. 7                                      D. 8

gateit-2007   operating-system   virtual-memory   page-replacement   normal

Answer key

#### 5.20.30 Page Replacement: GATE IT 2007 | Question: 58



A demand paging system takes 100 time units to service a page fault and 300 time units to replace a dirty page. Memory access time is 1 time unit. The probability of a page fault is  $p$ . In case of a page fault, the probability of page being dirty is also  $p$ . It is observed that the average access time is 3 time units. Then the value of  $p$  is

- A. 0.194                                      B. 0.233                                      C. 0.514                                      D. 0.981

gateit-2007   operating-system   page-replacement   probability   normal

Answer key

#### 5.20.31 Page Replacement: GATE IT 2008 | Question: 41



Assume that a main memory with only 4 pages, each of 16 bytes, is initially empty. The CPU generates the following sequence of virtual addresses and uses the Least Recently Used (LRU) page replacement policy.

0, 4, 8, 20, 24, 36, 44, 12, 68, 72, 80, 84, 28, 32, 88, 92

How many page faults does this sequence cause? What are the page numbers of the pages present in the main memory at the end of the sequence?

- A. 6 and 1,2,3,4                                      B. 7 and 1,2,4,5                                      C. 8 and 1,2,4,5                                      D. 9 and 1,2,3,5

gateit-2008   operating-system   page-replacement   normal

Answer key

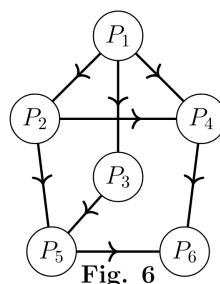
### 5.21

### Precedence Graph (3)

#### 5.21.1 Precedence Graph: GATE CSE 1989 | Question: 11b



Consider the following precedence graph (Fig.6) of processes where a node denotes a process and a directed edge from node  $P_i$  to node  $P_j$  implies; that  $P_i$  must complete before  $P_j$  commences. Implement the graph using FORK and JOIN constructs. The actual computation done by a process may be indicated by a comment line.



gate1989   descriptive   operating-system   precedence-graph   process-synchronization

Answer key

### 5.21.2 Precedence Graph: GATE CSE 1991 | Question: 01-xii



A given set of processes can be implemented by using only **parbegin/parend** statement, if the precedence graph of these processes is \_\_\_\_\_

gate1991 operating-system normal precedence-graph fill-in-the-blanks

Answer key

### 5.21.3 Precedence Graph: GATE CSE 1992 | Question: 12-a



Draw the precedence graph for the concurrent program given below

```
S1
parbegin
  begin
    S2:S4
  end;
  begin
    S3;
    parbegin
      S5;
      begin
        S6:S8
      end
    parend
  end;
end;
S7
parend;
S9
```

gate1992 operating-system normal concurrency precedence-graph descriptive

Answer key

## 5.22

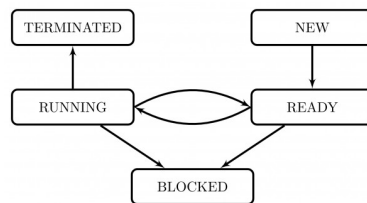
## Process (5)

**Practice Test:** Test 1 (7Q)

### 5.22.1 Process: GATE CSE 1996 | Question: 1.18



The process state transition diagram in the below figure is representative of



- A. a batch operating system
- B. an operating system with a preemptive scheduler
- C. an operating system with a non-preemptive scheduler
- D. a uni-programmed operating system

gate1996 operating-system normal process

Answer key

### 5.22.2 Process: GATE CSE 2001 | Question: 2.20



Which of the following does not interrupt a running process?

- A. A device
- B. Timer
- C. Scheduler process
- D. Power failure

gatecse-2001 operating-system easy process

### Answer key

#### 5.22.3 Process: GATE CSE 2002 | Question: 2.21



Which combination of the following features will suffice to characterize an OS as a multi-programmed OS?

- a. More than one program may be loaded into main memory at the same time for execution
- b. If a program waits for certain events such as I/O, another program is immediately scheduled for execution
- c. If the execution of a program terminates, another program is immediately scheduled for execution.

- A. (a)
- B. (a) and (b)
- C. (a) and (c)
- D. (a), (b) and (c)

gatecse-2002 operating-system normal process

### Answer key

#### 5.22.4 Process: GATE CSE 2023 | Question: 12



Which one or more of the following need to be saved on a context switch from one thread (T1) of a process to another thread (T2) of the same process?

- A. Page table base register
- B. Stack pointer
- C. Program counter
- D. General purpose registers

gatecse-2023 operating-system process multiple-selects one-mark

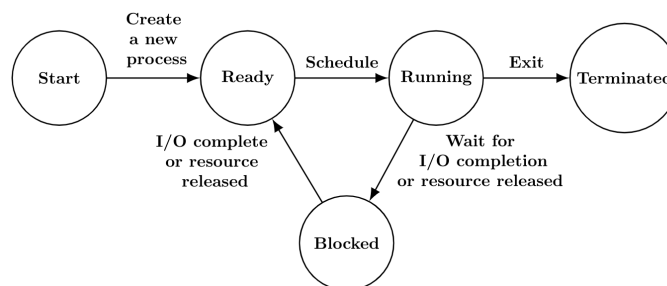
### Answer key

#### 5.22.5 Process: GATE IT 2006 | Question: 13



The process state transition diagram of an operating system is as given below.

Which of the following must be FALSE about the above operating system?



- A. It is a multiprogrammed operating system
- B. It uses preemptive scheduling
- C. It uses non-preemptive scheduling
- D. It is a multi-user operating system

gateit-2006 operating-system normal process

### Answer key

#### 5.23

#### Process Scheduling (49)

**Practice Tests:** [Test 1 \(15Q\)](#) [Test 2 \(15Q\)](#) [Test 3 \(15Q\)](#) [Test 4 \(15Q\)](#) [Test 5 \(2Q\)](#)

#### 5.23.1 Process Scheduling: GATE CSE 1988 | Question: 2xa



State any undesirable characteristic of the following criteria for measuring performance of an operating system:

**Turn around time**

gate1988 normal descriptive operating-system process-scheduling

### Answer key

### 5.23.2 Process Scheduling: GATE CSE 1988 | Question: 2xb



State any undesirable characteristic of the following criteria for measuring performance of an operating system:

**Waiting time**

gate1988 normal descriptive operating-system process-scheduling

Answer key

### 5.23.3 Process Scheduling: GATE CSE 1990 | Question: 1-vi



The highest-response ratio next scheduling policy favours \_\_\_\_\_ jobs, but it also limits the waiting time of \_\_\_\_\_ jobs.

gate1990 operating-system process-scheduling fill-in-the-blanks

Answer key

### 5.23.4 Process Scheduling: GATE CSE 1993 | Question: 7.10



Assume that the following jobs are to be executed on a single processor system

| Job Id | CPU Burst Time |
|--------|----------------|
| p      | 4              |
| q      | 1              |
| r      | 8              |
| s      | 1              |
| t      | 2              |

The jobs are assumed to have arrived at time  $0^+$  and in the order  $p, q, r, s, t$ . Calculate the departure time (completion time) for job  $p$  if scheduling is round robin with time slice 1

- A. 4                      B. 10                      C. 11                      D. 12  
E. None of the above

gate1993 operating-system process-scheduling normal

Answer key

### 5.23.5 Process Scheduling: GATE CSE 1995 | Question: 1.15



Which scheduling policy is most suitable for a time shared operating system?

- A. Shortest Job First                      B. Round Robin  
C. First Come First Serve                      D. Elevator

gate1995 operating-system process-scheduling easy

Answer key

### 5.23.6 Process Scheduling: GATE CSE 1995 | Question: 2.6



The sequence \_\_\_\_\_ is an optimal non-preemptive scheduling sequence for the following jobs which leaves the CPU idle for \_\_\_\_\_ unit(s) of time.

| Job | Arrival Time | Burst Time |
|-----|--------------|------------|
| 1   | 0.0          | 9          |
| 2   | 0.6          | 5          |
| 3   | 1.0          | 1          |

- A. {3,2,1},1                      B. {2,1,3},0                      C. {3,2,1},0                      D. {1,2,3},5

gate1995 operating-system process-scheduling normal

Answer key

5.23.7 Process Scheduling: GATE CSE 1996 | Question: 2.20, ISRO2008-15



Four jobs to be executed on a single processor system arrive at time 0 in the order  $A, B, C, D$ . Their burst CPU time requirements are 4, 1, 8, 1 time units respectively. The completion time of  $A$  under round robin scheduling with time slice of one time unit is

- A. 10                      B. 4                      C. 8                      D. 9

gate1996   operating-system   process-scheduling   normal   isro2008

Answer key

5.23.8 Process Scheduling: GATE CSE 1998 | Question: 2.17, UGCNET-Dec2012-III: 43



Consider  $n$  processes sharing the CPU in a round-robin fashion. Assuming that each process switch takes  $s$  seconds, what must be the quantum size  $q$  such that the overhead resulting from process switching is minimized but at the same time each process is guaranteed to get its turn at the CPU at least every  $t$  seconds?

- A.  $q \leq \frac{t-ns}{n-1}$                       B.  $q \geq \frac{t-ns}{n-1}$   
C.  $q \leq \frac{t-ns}{n+1}$                       D.  $q \geq \frac{t-ns}{n+1}$

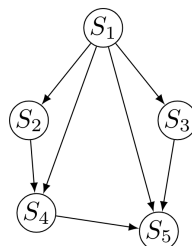
gate1998   operating-system   process-scheduling   normal   ugcnetcse-dec2012-paper3

Answer key

5.23.9 Process Scheduling: GATE CSE 1998 | Question: 24



- a. Four jobs are waiting to be run. Their expected run times are 6, 3, 5 and  $x$ . In what order should they be run to minimize the average response time?  
b. Write a concurrent program using `par` `begin-par` `end` to represent the precedence graph shown below.



gate1998   operating-system   process-scheduling   descriptive

Answer key

5.23.10 Process Scheduling: GATE CSE 1998 | Question: 7-b



In a computer system where the 'best-fit' algorithm is used for allocating 'jobs' to 'memory partitions', the following situation was encountered:

|                              |                            |
|------------------------------|----------------------------|
| <b>Partitions size in KB</b> | 4K 8K 20K 2K               |
| <b>Job sizes in KB</b>       | 2K 14K 3K 6K 6K 10K 20K 2K |
| <b>Time for execution</b>    | 4 10 2 1 4 1 8 6           |

When will the 20K job complete?

gate1998   operating-system   process-scheduling   normal

Answer key

5.23.11 Process Scheduling: GATE CSE 2002 | Question: 1.22



Which of the following scheduling algorithms is non-preemptive?



- A. Round Robin
- C. Multilevel Queue Scheduling

- B. First-In First-Out
- D. Multilevel Queue Scheduling with Feedback

gatecse-2002 operating-system process-scheduling easy

Answer key

#### 5.23.12 Process Scheduling: GATE CSE 2003 | Question: 77



A uni-processor computer system only has two processes, both of which alternate 10 ms CPU bursts with 90 ms I/O bursts. Both the processes were created at nearly the same time. The I/O of both processes can proceed in parallel. Which of the following scheduling strategies will result in the *least* CPU utilization (over a long period of time) for this system?

- A. First come first served scheduling
- B. Shortest remaining time first scheduling
- C. Static priority scheduling with different priorities for the two processes
- D. Round robin scheduling with a time quantum of 5 ms

gatecse-2003 operating-system process-scheduling normal

Answer key

#### 5.23.13 Process Scheduling: GATE CSE 2004 | Question: 46



Consider the following set of processes, with the arrival times and the CPU-burst times gives in milliseconds.

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 5          |
| P2      | 1            | 3          |
| P3      | 2            | 3          |
| P4      | 4            | 1          |

What is the average turnaround time for these processes with the preemptive shortest remaining processing time first (SRPT) algorithm?

- A. 5.50
- B. 5.75
- C. 6.00
- D. 6.25

gatecse-2004 operating-system process-scheduling normal

Answer key

#### 5.23.14 Process Scheduling: GATE CSE 2006 | Question: 06, ISRO2009-14



Consider three CPU-intensive processes, which require 10, 20 and 30 time units and arrive at times 0, 2 and 6, respectively. How many context switches are needed if the operating system implements a shortest remaining time first scheduling algorithm? Do not count the context switches at time zero and at the end.

- A. 1
- B. 2
- C. 3
- D. 4

gatecse-2006 operating-system process-scheduling normal isro2009

Answer key

#### 5.23.15 Process Scheduling: GATE CSE 2006 | Question: 64



Consider three processes (process id 0, 1, 2 respectively) with compute time bursts 2, 4 and 8 time units. All processes arrive at time zero. Consider the longest remaining time first (LRTF) scheduling algorithm. In LRTF ties are broken by giving priority to the process with the lowest process id. The average turn around time is:

- A. 13 units
- B. 14 units
- C. 15 units
- D. 16 units

gatecse-2006 operating-system process-scheduling normal

Answer key

### 5.23.16 Process Scheduling: GATE CSE 2006 | Question: 65



Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle?

- A. 0%                      B. 10.6%                      C. 30.0%                      D. 89.4%

gategse-2006    operating-system    process-scheduling    normal

Answer key

### 5.23.17 Process Scheduling: GATE CSE 2007 | Question: 16



Group 1 contains some CPU scheduling algorithms and Group 2 contains some applications. Match entries in Group 1 to entries in Group 2.

| Group I                       | Group II                  |
|-------------------------------|---------------------------|
| (P) Gang Scheduling           | (1) Guaranteed Scheduling |
| (Q) Rate Monotonic Scheduling | (2) Real-time Scheduling  |
| (R) Fair Share Scheduling     | (3) Thread Scheduling     |

- A.  $P - 3; Q - 2; R - 1$     B.  $P - 1; Q - 2; R - 3$     C.  $P - 2; Q - 3; R - 1$     D.  $P - 1; Q - 3; R - 2$

gategse-2007    operating-system    process-scheduling    normal

Answer key

### 5.23.18 Process Scheduling: GATE CSE 2007 | Question: 55



An operating system used Shortest Remaining System Time first (SRT) process scheduling algorithm. Consider the arrival times and execution times for the following processes:

| Process | Execution Time | Arrival Time |
|---------|----------------|--------------|
| P1      | 20             | 0            |
| P2      | 25             | 15           |
| P3      | 10             | 30           |
| P4      | 15             | 45           |

What is the total waiting time for process  $P2$  ?

- A. 5                      B. 15                      C. 40                      D. 55

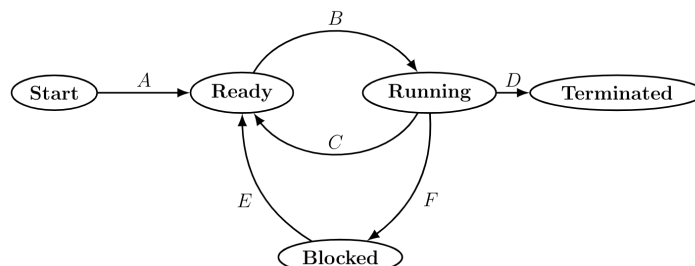
gategse-2007    operating-system    process-scheduling    normal

Answer key

### 5.23.19 Process Scheduling: GATE CSE 2009 | Question: 32



In the following process state transition diagram for a uniprocessor system, assume that there are always some processes in the ready state:



Now consider the following statements:

- I. If a process makes a transition  $D$ , it would result in another process making transition  $A$  immediately.
- II. A process  $P_2$  in blocked state can make transition  $E$  while another process  $P_1$  is in running state.
- III. The OS uses preemptive scheduling.
- IV. The OS uses non-preemptive scheduling.

Which of the above statements are TRUE?

- A. I and II                      B. I and III                      C. II and III                      D. II and IV

gatecse-2009   operating-system   process-scheduling   normal

Answer key

#### 5.23.20 Process Scheduling: GATE CSE 2010 | Question: 25



Which of the following statements are true?

- I. Shortest remaining time first scheduling may cause starvation
- II. Preemptive scheduling may cause starvation
- III. Round robin is better than FCFS in terms of response time

- A. I only                      B. I and III only                      C. II and III only                      D. I, II and III

gatecse-2010   operating-system   process-scheduling   easy

Answer key

#### 5.23.21 Process Scheduling: GATE CSE 2011 | Question: 35



Consider the following table of arrival time and burst time for three processes  $P_0$ ,  $P_1$  and  $P_2$ .

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P0      | 0 ms         | 9          |
| P1      | 1 ms         | 4          |
| P2      | 2 ms         | 9          |

The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

- A. 5.0 ms                      B. 4.33 ms                      C. 6.33 ms                      D. 7.33 ms

gatecse-2011   operating-system   process-scheduling   normal

Answer key

#### 5.23.22 Process Scheduling: GATE CSE 2012 | Question: 31



Consider the 3 processes,  $P_1$ ,  $P_2$  and  $P_3$  shown in the table.

| Process | Arrival Time | Time Units Required |
|---------|--------------|---------------------|
| P1      | 0            | 5                   |
| P2      | 1            | 7                   |
| P3      | 3            | 4                   |

The completion order of the 3 processes under the policies FCFS and RR2 (round robin scheduling with CPU quantum of 2 time units) are

- A. **FCFS:**  $P_1, P_2, P_3$  **RR2:**  $P_1, P_2, P_3$   
B. **FCFS:**  $P_1, P_3, P_2$  **RR2:**  $P_1, P_3, P_2$   
C. **FCFS:**  $P_1, P_2, P_3$  **RR2:**  $P_1, P_3, P_2$   
D. **FCFS:**  $P_1, P_3, P_2$  **RR2:**  $P_1, P_2, P_3$

gatecse-2012   operating-system   process-scheduling   normal

Answer key

5.23.23 Process Scheduling: GATE CSE 2013 | Question: 10



A scheduling algorithm assigns priority proportional to the waiting time of a process. Every process starts with zero (the lowest priority). The scheduler re-evaluates the process priorities every  $T$  time units and decides the next process to schedule. Which one of the following is **TRUE** if the processes have no I/O operations and all arrive at time zero?

- A. This algorithm is equivalent to the first-come-first-serve algorithm.
- B. This algorithm is equivalent to the round-robin algorithm.
- C. This algorithm is equivalent to the shortest-job-first algorithm.
- D. This algorithm is equivalent to the shortest-remaining-time-first algorithm.

gatecse-2013 operating-system process-scheduling normal

Answer key

5.23.24 Process Scheduling: GATE CSE 2014 | Set 1 | Question: 32



Consider the following set of processes that need to be scheduled on a single CPU. All the times are given in milliseconds.

| Process Name | Arrival Time | Execution Time |
|--------------|--------------|----------------|
| A            | 0            | 6              |
| B            | 3            | 2              |
| C            | 5            | 4              |
| D            | 7            | 6              |
| E            | 10           | 3              |

Using the *shortest remaining time first* scheduling algorithm, the average process turnaround time (in msec) is \_\_\_\_\_.

gatecse-2014-set1 operating-system process-scheduling numerical-answers normal

Answer key

5.23.25 Process Scheduling: GATE CSE 2014 | Set 2 | Question: 32



Three processes  $A$ ,  $B$  and  $C$  each execute a loop of 100 iterations. In each iteration of the loop, a process performs a single computation that requires  $t_c$  CPU milliseconds and then initiates a single I/O operation that lasts for  $t_{io}$  milliseconds. It is assumed that the computer where the processes execute has sufficient number of I/O devices and the OS of the computer assigns different I/O devices to each process. Also, the scheduling overhead of the OS is negligible. The processes have the following characteristics:

| Process id | $t_c$  | $t_{io}$ |
|------------|--------|----------|
| A          | 100 ms | 500 ms   |
| B          | 350 ms | 500 ms   |
| C          | 200 ms | 500 ms   |

The processes  $A$ ,  $B$ , and  $C$  are started at times 0, 5 and 10 milliseconds respectively, in a pure time sharing system (round robin scheduling) that uses a time slice of 50 milliseconds. The time in milliseconds at which process C would **complete** its first I/O operation is \_\_\_\_\_.

gatecse-2014-set2 operating-system process-scheduling numerical-answers normal

Answer key

5.23.26 Process Scheduling: GATE CSE 2014 | Set 3 | Question: 32



An operating system uses *shortest remaining time first* scheduling algorithm for pre-emptive scheduling of processes. Consider the following set of processes with their arrival times and CPU burst times (in

milliseconds):

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 12         |
| P2      | 2            | 4          |
| P3      | 3            | 6          |
| P4      | 8            | 5          |

The average waiting time (in milliseconds) of the processes is \_\_\_\_\_.

gatecse-2014-set3 operating-system process-scheduling numerical-answers normal

Answer key

#### 5.23.27 Process Scheduling: GATE CSE 2015 | Set 1 | Question: 46



Consider a uniprocessor system executing three tasks  $T_1, T_2$  and  $T_3$  each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of 3, 7 and 20 milliseconds, respectively. The priority of each task is the inverse of its period, and the available tasks are scheduled in order of priority, which is the highest priority task scheduled first. Each instance of  $T_1, T_2$  and  $T_3$  requires an execution time of 1, 2 and 4 milliseconds, respectively. Given that all tasks initially arrive at the beginning of the 1<sup>st</sup> millisecond and task preemptions are allowed, the first instance of  $T_3$  completes its execution at the end of \_\_\_\_\_ milliseconds.

gatecse-2015-set1 operating-system process-scheduling normal numerical-answers

Answer key

#### 5.23.28 Process Scheduling: GATE CSE 2015 | Set 3 | Question: 1



The maximum number of processes that can be in *Ready* state for a computer system with  $n$  CPUs is :

- A.  $n$                       B.  $n^2$                       C.  $2^n$                       D. Independent of  $n$

gatecse-2015-set3 operating-system process-scheduling easy

Answer key

#### 5.23.29 Process Scheduling: GATE CSE 2015 | Set 3 | Question: 34



For the processes listed in the following table, which of the following scheduling schemes will give the lowest average turnaround time?

| Process | Arrival Time | Process Time |
|---------|--------------|--------------|
| A       | 0            | 3            |
| B       | 1            | 6            |
| C       | 4            | 4            |
| D       | 6            | 2            |

- A. First Come First Serve                      B. Non-preemptive Shortest job first  
C. Shortest Remaining Time                      D. Round Robin with Quantum value two

gatecse-2015-set3 operating-system process-scheduling normal

Answer key

#### 5.23.30 Process Scheduling: GATE CSE 2016 | Set 1 | Question: 20



Consider an arbitrary set of CPU-bound processes with unequal CPU burst lengths submitted at the same time to a computer system. Which one of the following process scheduling algorithms would minimize the average waiting time in the ready queue?

- A. Shortest remaining time first  
B. Round-robin with the time quantum less than the shortest CPU burst  
C. Uniform random

D. Highest priority first with priority proportional to CPU burst length

gatecse-2016-set1 operating-system process-scheduling normal

Answer key

#### 5.23.31 Process Scheduling: GATE CSE 2016 | Set 2 | Question: 47



Consider the following processes, with the arrival time and the length of the CPU burst given in milliseconds. The scheduling algorithm used is preemptive shortest remaining-time first.

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0            | 10         |
| $P_2$   | 3            | 6          |
| $P_3$   | 7            | 1          |
| $P_4$   | 8            | 3          |

The average turn around time of these processes is \_\_\_\_\_ milliseconds.

gatecse-2016-set2 operating-system process-scheduling normal numerical-answers

Answer key

#### 5.23.32 Process Scheduling: GATE CSE 2017 | Set 1 | Question: 24



Consider the following CPU processes with arrival times (in milliseconds) and length of CPU bursts (in milliseconds) as given below:

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0            | 7          |
| $P_2$   | 3            | 3          |
| $P_3$   | 5            | 5          |
| $P_4$   | 6            | 2          |

If the pre-emptive shortest remaining time first scheduling algorithm is used to schedule the processes, then the average waiting time across all processes is \_\_\_\_\_ milliseconds.

gatecse-2017-set1 operating-system process-scheduling numerical-answers

Answer key

#### 5.23.33 Process Scheduling: GATE CSE 2017 | Set 2 | Question: 51



Consider the set of process with arrival time (in milliseconds), CPU burst time (in milliseconds) and priority (0 is the highest priority) shown below. None of the process have I/O burst time

| Process | Arrival Time | Burst Time | Priority |
|---------|--------------|------------|----------|
| $P_1$   | 0            | 11         | 2        |
| $P_2$   | 5            | 28         | 0        |
| $P_3$   | 12           | 2          | 3        |
| $P_4$   | 2            | 10         | 1        |
| $P_5$   | 9            | 16         | 4        |

The average waiting time (in milli seconds) of all the process using premtive priority scheduling algorithm is \_\_\_\_\_

gatecse-2017-set2 operating-system process-scheduling numerical-answers

Answer key

#### 5.23.34 Process Scheduling: GATE CSE 2019 | Question: 41



Consider the following four processes with arrival times (in milliseconds) and their length of CPU bursts (in milliseconds) as shown below:

| Process        | $P_1$ | $P_2$ | $P_3$ | $P_4$ |
|----------------|-------|-------|-------|-------|
| Arrival Time   | 0     | 1     | 3     | 4     |
| CPU burst time | 3     | 1     | 3     | $Z$   |

These processes are run on a single processor using preemptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of  $Z$  is \_\_\_\_\_

gatecse-2019 numerical-answers operating-system process-scheduling two-marks

Answer key

#### 5.23.35 Process Scheduling: GATE CSE 2020 | Question: 12



Consider the following statements about process state transitions for a system using preemptive scheduling.

- I. A running process can move to ready state.
- II. A ready process can move to running state.
- III. A blocked process can move to running state.
- IV. A blocked process can move to ready state.

Which of the above statements are TRUE?

- A. I, II, and III only
- B. II and III only
- C. I, II, and IV only
- D. I, II, III and IV only

gatecse-2020 operating-system process-scheduling one-mark easy

Answer key

#### 5.23.36 Process Scheduling: GATE CSE 2020 | Question: 50



Consider the following set of processes, assumed to have arrived at time 0. Consider the CPU scheduling algorithms Shortest Job First (SJF) and Round Robin (RR). For RR, assume that the processes are scheduled in the order  $P_1, P_2, P_3, P_4$ .

| Processes          | $P_1$ | $P_2$ | $P_3$ | $P_4$ |
|--------------------|-------|-------|-------|-------|
| Burst time (in ms) | 8     | 7     | 2     | 4     |

If the time quantum for RR is 4 ms, then the absolute value of the difference between the average turnaround times (in ms) of SJF and RR (round off to 2 decimal places) is \_\_\_\_\_

gatecse-2020 numerical-answers operating-system process-scheduling two-marks

Answer key

#### 5.23.37 Process Scheduling: GATE CSE 2021 | Set 1 | Question: 25



Three processes arrive at time zero with CPU bursts of 16, 20 and 10 milliseconds. If the scheduler has prior knowledge about the length of the CPU bursts, the minimum achievable average waiting time for these three processes in a non-preemptive scheduler (rounded to nearest integer) is \_\_\_\_\_ milliseconds.

gatecse-2021-set1 operating-system process-scheduling numerical-answers one-mark

Answer key

#### 5.23.38 Process Scheduling: GATE CSE 2021 | Set 2 | Question: 14



Which of the following statement(s) is/are correct in the context of CPU scheduling?

- A. Turnaround time includes waiting time
- B. The goal is to only maximize CPU utilization and minimize throughput
- C. Round-robin policy can be used even when the CPU time required by each of the processes is not known apriori
- D. Implementing preemptive scheduling needs hardware support

Answer key

## 5.23.39 Process Scheduling: GATE CSE 2023 | Question: 17



Which one or more of the following CPU scheduling algorithms can potentially cause starvation?

- A. First-in First-Out  
 C. Priority Scheduling  
 B. Round Robin  
 D. Shortest Job First

gatecse-2023 operating-system process-scheduling multiple-selects one-mark

Answer key

## 5.23.40 Process Scheduling: GATE CSE 2024 | Set 1 | Question: 15



Which of the following process state transitions is/are NOT possible?

- A. Running to Ready  
 C. Ready to Waiting  
 B. Waiting to Running  
 D. Running to Terminated

gatecse-2024-set1 operating-system process-scheduling multiple-selects one-mark

Answer key

## 5.23.41 Process Scheduling: GATE CSE 2024 | Set 2 | Question: 27



Consider a single processor system with four processes A, B, C, and D, represented as given below, where for each process the first value is its arrival time, and the second value is its CPU burst time.

A(0, 10), B(2, 6), C(4, 3), and D(6, 7).

Which one of the following options gives the average waiting times when preemptive Shortest Remaining Time First (SRTF) and Non-Preemptive Shortest Job First (NP-SJF) CPU scheduling algorithms are applied to the processes?

- A. SRTF = 6, NP – SJF = 7  
 B. SRTF = 6, NP – SJF = 7.5  
 C. SRTF = 7, NP – SJF = 7.5  
 D. SRTF = 7, NP – SJF = 8.5

gatecse-2024-set2 operating-system process-scheduling two-marks

Answer key

## 5.23.42 Process Scheduling: GATE CSE 2025 | Set 1 | Question: 28



A computer has two processors,  $M_1$  and  $M_2$ . Four processes  $P_1, P_2, P_3, P_4$  with CPU bursts of 20, 16, 25, and 10 milliseconds, respectively, arrive at the same time and these are the only processes in the system. The scheduler uses non-preemptive priority scheduling, with priorities decided as follows:

- $M_1$  uses priority of execution for the processes as,  $P_1 > P_3 > P_2 > P_4$ , i.e.,  $P_1$  and  $P_4$  have highest and lowest priorities, respectively.
- $M_2$  uses priority of execution for the processes as,  $P_2 > P_3 > P_4 > P_1$ , i.e.,  $P_2$  and  $P_1$  have highest and lowest priorities, respectively.

A process  $P_i$  is scheduled to a processor  $M_k$ , if the processor is free and no other process  $P_j$  is waiting with higher priority. At any given point of time, a process can be allocated to any one of the free processors without violating the execution priority rules. Ignore the context switch time. What will be the average waiting time of the processes in milliseconds?

- A. 9.00  
 B. 8.75  
 C. 6.50  
 D. 7.50

gatecse2025-set1 operating-system process-scheduling two-marks

Answer key



**5.23.43 Process Scheduling: GATE CSE 2026 | Set 1 | Question: 54**

Consider a CPU that has to execute two types of processes. The first type, Actuators (A), requires a CPU burst of 6 seconds. The second type, Controllers (C), requires a CPU burst of 8 seconds. A new process of type A arrives at time  $t = 10, 20, 30, 40$ , and  $50$  (in seconds). Similarly, a new process of type C arrives at time  $t = 11, 22, 33, 44$ , and  $55$  (in seconds). The CPU scheduling policy is First Come First Serve (FCFS). The first process of type A starts running at  $t = 10$  seconds. The average waiting time (in seconds) for the 10 processes is \_\_\_\_\_. (rounded off to one decimal place)

gatecse-2026-set1 numerical-answers operating-system process-scheduling two-marks

**Answer key****5.23.44 Process Scheduling: GATE CSE 2026 | Set 2 | Question: 13**

Which one of the following CPU scheduling algorithms **cannot** be preemptive?

- A. Shortest Remaining Time First (SRTF) Scheduling  
 B. First Come First Serve (FCFS) Scheduling  
 C. Round Robin Scheduling  
 D. Priority Scheduling

gatecse-2026-set2 operating-system process-scheduling one-mark

**Answer key****5.23.45 Process Scheduling: GATE IT 2005 | Question: 60**

We wish to schedule three processes  $P_1$ ,  $P_2$  and  $P_3$  on a uniprocessor system. The priorities, CPU time requirements and arrival times of the processes are as shown below.

| Process | Priority     | CPU time required | Arrival time (hh:mm:ss) |
|---------|--------------|-------------------|-------------------------|
| P1      | 10 (highest) | 20 sec            | 00 : 00 : 05            |
| P2      | 9            | 10 sec            | 00 : 00 : 03            |
| P3      | 8 (lowest)   | 15 sec            | 00 : 00 : 00            |

We have a choice of preemptive or non-preemptive scheduling. In preemptive scheduling, a late-arriving higher priority process can preempt a currently running process with lower priority. In non-preemptive scheduling, a late-arriving higher priority process must wait for the currently executing process to complete before it can be scheduled on the processor.

What are the turnaround times (time from arrival till completion) of  $P_2$  using preemptive and non-preemptive scheduling respectively?

- A. 30 sec, 30 sec  
 B. 30 sec, 10 sec  
 C. 42 sec, 42 sec  
 D. 30 sec, 42 sec

gateit-2005 operating-system process-scheduling normal

**Answer key****5.23.46 Process Scheduling: GATE IT 2006 | Question: 12**

In the working-set strategy, which of the following is done by the operating system to prevent thrashing?

- I. It initiates another process if there are enough extra frames.  
 II. It selects a process to suspend if the sum of the sizes of the working-sets exceeds the total number of available frames.

- A. I only  
 B. II only  
 C. Neither I nor II  
 D. Both I and II

gateit-2006 operating-system process-scheduling normal

**Answer key****5.23.47 Process Scheduling: GATE IT 2006 | Question: 54**

The arrival time, priority, and duration of the CPU and I/O bursts for each of three processes  $P_1$ ,  $P_2$  and  $P_3$  are given in the table below. Each process has a CPU burst followed by an I/O burst followed by another

CPU burst. Assume that each process has its own I/O resource.

| Process | Arrival Time | Priority    | Burst duration (CPU) | Burst duration (I/O) | Burst duration (CPU) |
|---------|--------------|-------------|----------------------|----------------------|----------------------|
| $P_1$   | 0            | 2           | 1                    | 5                    | 3                    |
| $P_2$   | 2            | 3 (lowest)  | 3                    | 3                    | 1                    |
| $P_3$   | 3            | 1 (highest) | 2                    | 3                    | 1                    |

The multi-programmed operating system uses preemptive priority scheduling. What are the finish times of the processes  $P_1$ ,  $P_2$  and  $P_3$ ?

- A. 11, 15, 9      B. 10, 15, 9      C. 11, 16, 10      D. 12, 17, 11

gateit-2006   operating-system   process-scheduling   normal

Answer key

#### 5.23.48 Process Scheduling: GATE IT 2007 | Question: 26



Consider  $n$  jobs  $J_1, J_2, \dots, J_n$  such that job  $J_i$  has execution time  $t_i$  and a non-negative integer weight  $w_i$ .

The weighted mean completion time of the jobs is defined to be  $\frac{\sum_{i=1}^n w_i T_i}{\sum_{i=1}^n w_i}$ , where  $T_i$  is the completion time of job  $J_i$ . Assuming that there is only one processor available, in what order must the jobs be executed in order to minimize the weighted mean completion time of the jobs?

- A. Non-decreasing order of  $t_i$       B. Non-increasing order of  $w_i$   
C. Non-increasing order of  $w_i t_i$       D. Non-increasing order of  $w_i/t_i$

gateit-2007   operating-system   process-scheduling   normal

Answer key

#### 5.23.49 Process Scheduling: GATE IT 2008 | Question: 55



If the time-slice used in the round-robin scheduling policy is more than the maximum time required to execute any process, then the policy will

- A. degenerate to shortest job first      B. degenerate to priority scheduling  
C. degenerate to first come first serve      D. none of the above

gateit-2008   operating-system   process-scheduling   easy

Answer key

### 5.24 Process Synchronization (52)

**Practice Tests:** [Test 1 \(15Q\)](#) [Test 2 \(15Q\)](#) [Test 3 \(11Q\)](#) [Weekly Quiz 1 \(16Q\)](#)

#### 5.24.1 Process Synchronization: GATE CSE 1987 | Question: 1-xvi



A critical region is

- A. One which is enclosed by a pair of  $P$  and  $V$  operations on semaphores.  
B. A program segment that has not been proved bug-free.  
C. A program segment that often causes unexpected system crashes.  
D. A program segment where shared resources are accessed.

gate1987   operating-system   process-synchronization

Answer key

#### 5.24.2 Process Synchronization: GATE CSE 1987 | Question: 8a



Consider the following proposal to the "readers and writers problem."

Shared variables and semaphores:

aw, ar, rw, rr : interger;  
mutex, reading, writing: semaphore;  
initial values of variables and states of semaphores:



#### 5.24.4 Process Synchronization: GATE CSE 1990 | Question: 2-iii



Match the pairs:

|                     |                           |
|---------------------|---------------------------|
| (a) Critical region | (p) Hoare's monitor       |
| (b) Wait/Signal     | (q) Mutual exclusion      |
| (c) Working Set     | (r) Principle of locality |
| (d) Deadlock        | (s) Circular Wait         |

match-the-following gate1990 operating-system process-synchronization

Answer key

#### 5.24.5 Process Synchronization: GATE CSE 1991 | Question: 11,a



Consider the following scheme for implementing a critical section in a situation with three processes  $P_i, P_j$  and  $P_k$ .

```
Pi;
repeat
  flag[i] := true;
  while flag[j] or flag[k] do
    case turn of
      j: if flag[j] then
        begin
          flag[i] := false;
          while turn != i do skip;
          flag[i] := true;
        end;
      k: if flag[k] then
        begin
          flag[i] := false;
          while turn != i do skip;
          flag[i] := true;
        end;
    end
  end
  critical section
  if turn = i then turn := j;
  flag[i] := false
non-critical section
until false;
```

a. Does the scheme ensure mutual exclusion in the critical section? Briefly explain.

gate1991 process-synchronization normal operating-system descriptive

Answer key

#### 5.24.6 Process Synchronization: GATE CSE 1991 | Question: 11,b



Consider the following scheme for implementing a critical section in a situation with three processes  $P_i, P_j$  and  $P_k$ .

```
Pi;
repeat
  flag[i] := true;
  while flag[j] or flag[k] do
    case turn of
      j: if flag[j] then
        begin
          flag[i] := false;
          while turn != i do skip;
          flag[i] := true;
        end;
      k: if flag[k] then
        begin
          flag[i] := false;
          while turn != i do skip;
          flag[i] := true;
        end;
    end
  end
  critical section
  if turn = i then turn := j;
  flag[i] := false
non-critical section
until false;
```

```

critical section
if turn = i then turn := j;
  flag [i] := false
non-critical section
until false;

```

Is there a situation in which a waiting process can never enter the critical section? If so, explain and suggest modifications to the code to solve this problem

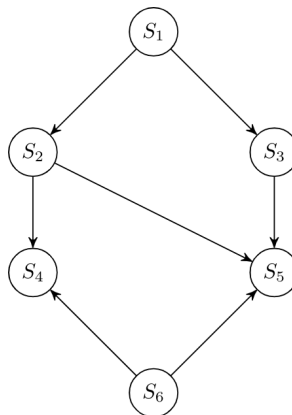
gate1991 process-synchronization normal operating-system descriptive

Answer key

#### 5.24.7 Process Synchronization: GATE CSE 1993 | Question: 22



Write a concurrent program using `parbegin-parend` and semaphores to represent the precedence constraints of the statements  $S_1$  to  $S_6$ , as shown in figure below.



gate1993 operating-system process-synchronization normal descriptive

Answer key

#### 5.24.8 Process Synchronization: GATE CSE 1994 | Question: 27



A. Draw a precedence graph for the following sequential code. The statements are numbered from  $S_1$  to  $S_6$

```

S1    read n
S2    i := 1
S3    if i > n next
S4    a(i) := i+1
S5    i := i+1
S6    next : write a(i)

```

B. Can this graph be converted to a concurrent program using `parbegin-parend` construct only?

gate1994 operating-system process-synchronization normal descriptive

Answer key

#### 5.24.9 Process Synchronization: GATE CSE 1995 | Question: 19



Consider the following program segment for concurrent processing using semaphore operators  $P$  and  $V$  for synchronization. Draw the precedence graph for the statements  $S_1$  to  $S_9$ .

```

var
a,b,c,d,e,f,g,h,i,j,k : semaphore;
begin
cobegin
  begin S1; V(a); V(b) end;
  begin P(a); S2; V(c); V(d) end;
  begin P(c); S4; V(e) end;
  begin P(d); S5; V(f) end;
  begin P(e); P(f); S7; V(k) end
endcobegin

```

```

begin P(b); S3; V(g); V(h) end;
begin P(g); S6; V(i) end;
begin P(h); P(i); S8; V(j) end;
begin P(j); P(k); S9 end;
coend
end;

```

gate1995 operating-system process-synchronization normal descriptive

Answer key

#### 5.24.10 Process Synchronization: GATE CSE 1996 | Question: 1.19, ISRO2008-61



A critical section is a program segment

- A. which should run in a certain amount of time
- B. which avoids deadlocks
- C. where shared resources are accessed
- D. which must be enclosed by a pair of semaphore operations,  $P$  and  $V$

gate1996 operating-system process-synchronization easy isro2008

Answer key

#### 5.24.11 Process Synchronization: GATE CSE 1996 | Question: 2.19



A solution to the Dining Philosophers Problem which avoids deadlock is to

- A. ensure that all philosophers pick up the left fork before the right fork
- B. ensure that all philosophers pick up the right fork before the left fork
- C. ensure that one particular philosopher picks up the left fork before the right fork, and that all other philosophers pick up the right fork before the left fork
- D. None of the above

gate1996 operating-system process-synchronization normal

Answer key

#### 5.24.12 Process Synchronization: GATE CSE 1996 | Question: 21



The concurrent programming constructs fork and join are as below:

Fork <label> which creates a new process executing from the specified label

Join <variable> which decrements the specified synchronization variable (by 1) and terminates the process if the new value is not 0.

Show the precedence graph for  $S1$ ,  $S2$ ,  $S3$ ,  $S4$ , and  $S5$  of the concurrent program below.

```

N=2
M=2
Fork L3
Fork L4
S1
L1 : join N
S3
L2 : join M
S5
L3:S2
Goto L1
L4:S4
Goto L2
Next:

```

gate1996 operating-system process-synchronization normal descriptive

Answer key

## 5.24.13 Process Synchronization: GATE CSE 1997 | Question: 6.8



Each Process  $P_i, i = 1 \dots 9$  is coded as follows

```
repeat
  P(mutex)
  {Critical section}
  V(mutex)
forever
```

The code for  $P_{10}$  is identical except it uses V(mutex) in place of P(mutex). What is the largest number of processes that can be inside the critical section at any moment?

- A. 1                      B. 2                      C. 3                      D. None

gate1997   operating-system   process-synchronization   normal

Answer key

## 5.24.14 Process Synchronization: GATE CSE 1997 | Question: 73



A concurrent system consists of 3 processes using a shared resource  $R$  in a non-preemptible and mutually exclusive manner. The processes have unique priorities in the range  $1 \dots 3$ , 3 being the highest priority. It is required to synchronize the processes such that the resource is always allocated to the highest priority requester. The pseudo code for the system is as follows.

### Shared data

```
mutex:semaphore = 1; /* initialized to 1 */
process[3]:semaphore = 0; /* all initialized to 0 */
R_requested [3]:boolean = false; /* all initialized to false */
busy: boolean = false; /* initialized to false */
```

### Code for processes

```
begin process
my-priority:integer;
my-priority:=___; /* in the range 1..3 */
repeat
  request_R(my-priority);
  P (proceed [my-priority]);
  {use shared resource R}
  release_R (my-priority);
forever
end process;
```

### Procedures

```
procedure request_R(priority);
P(mutex);
if busy = true then
  R_requested [priority]:=true;
else
begin
  V(proceed [priority]);
  busy:=true;
end
V(mutex)
```

Give the pseudo code for the procedure release\_R.

gate1997   operating-system   process-synchronization   descriptive

Answer key

## 5.24.15 Process Synchronization: GATE CSE 1998 | Question: 1.30



When the result of a computation depends on the speed of the processes involved, there is said to be

- A. cycle stealing                      B. race condition                      C. a time lock                      D. a deadlock

## Answer key

## 5.24.16 Process Synchronization: GATE CSE 1999 | Question: 20-a



A certain processor provides a 'test and set' instruction that is used as follows:

TSET register, flag

This instruction atomically copies flag to register and sets flag to 1. Give pseudo-code for implementing the entry and exit code to a critical region using this instruction.

## Answer key

## 5.24.17 Process Synchronization: GATE CSE 1999 | Question: 20-b



Consider the following solution to the producer-consumer problem using a buffer of size 1. Assume that the initial value of count is 0. Also assume that the testing of count and assignment to count are atomic operations.

Producer:

Repeat

Produce an item;  
if count = 1 then sleep;  
place item in buffer.  
count = 1;  
Wakeup(Consumer);

Forever

Consumer:

Repeat

if count = 0 then sleep;  
Remove item from buffer;  
count = 0;  
Wakeup(Producer);  
Consume item;

Forever;

Show that in this solution it is possible that both the processes are sleeping at the same time.

## Answer key

## 5.24.18 Process Synchronization: GATE CSE 2000 | Question: 1.21



Let  $m[0] \dots m[4]$  be mutexes (binary semaphores) and  $P[0] \dots P[4]$  be processes. Suppose each process  $P[i]$  executes the following:

```
wait (m[i]); wait (m(i+1) mod 4);
.....
release (m[i]); release (m(i+1) mod 4);
```

This could cause

- |                                 |                      |
|---------------------------------|----------------------|
| A. Thrashing                    | B. Deadlock          |
| C. Starvation, but not deadlock | D. None of the above |

## Answer key

## 5.24.19 Process Synchronization: GATE CSE 2000 | Question: 20



a. Fill in the boxes below to get a solution for the reader-writer problem, using a single binary semaphore, mutex (initialized to 1) and busy waiting. Write the box numbers (1, 2 and 3), and their contents in your answer book.



```

int R = 0, W = 0;

Reader () {
L1: wait (mutex);
  if (W == 0) {
    R = R + 1;
    _____ (1)
  }
  else {
    _____ (2)
    goto L1;
  }
  ..../* do the read*/
  wait (mutex);
  R = R - 1;
  signal (mutex);
}

```

```

Writer () {
L2: wait (mutex);
  if (R == 0) { _____ (3)
    signal (mutex);
    goto L2;
  }
  W=1;
  signal (mutex);
  ..../*do the write*/
  wait( mutex);
  W=0;
  signal (mutex);
}

```

b. Can the above solution lead to starvation of writers?

gatecse-2000 operating-system process-synchronization normal descriptive

Answer key

#### 5.24.20 Process Synchronization: GATE CSE 2001 | Question: 2.22



Consider Peterson's algorithm for mutual exclusion between two concurrent processes i and j. The program executed by process i is shown below.

```

repeat
  flag[i] = true;
  turn = j;
  while (P) do no-op;
  Enter critical section, perform actions, then
  exit critical section
  flag[i] = false;
  Perform other non-critical section actions.
Until false;

```

For the program to guarantee mutual exclusion, the predicate P in the while loop should be

- A.  $\text{flag}[j] = \text{true}$  and  $\text{turn} = i$
- B.  $\text{flag}[j] = \text{true}$  and  $\text{turn} = j$
- C.  $\text{flag}[i] = \text{true}$  and  $\text{turn} = j$
- D.  $\text{flag}[i] = \text{true}$  and  $\text{turn} = i$

gatecse-2001 operating-system process-synchronization normal

Answer key

#### 5.24.21 Process Synchronization: GATE CSE 2002 | Question: 18-a



Draw the process state transition diagram of an OS in which (i) each process is in one of the five states: created, ready, running, blocked (i.e., sleep or wait), or terminated, and (ii) only non-preemptive scheduling is used by the OS. Label the transitions appropriately.

gatecse-2002 operating-system process-synchronization normal descriptive

Answer key

### 5.24.22 Process Synchronization: GATE CSE 2002 | Question: 18-b



The functionality of atomic TEST-AND-SET assembly language instruction is given by the following C function

```
int TEST-AND-SET (int *x)
{
    int y;
    A1: y=*x;
    A2: *x=1;
    A3: return y;
}
```

- i. Complete the following C functions for implementing code for entering and leaving critical sections on the above TEST-AND-SET instruction.

```
int mutex=0;
void enter-cs()
{
    while(.....);
}
void leave-cs()
{
    .....;
}
```

- ii. Is the above solution to the critical section problem deadlock free and starvation-free?  
 iii. For the above solution, show by an example that mutual exclusion is not ensured if TEST-AND-SET instruction is not atomic?

gatecse-2002 operating-system process-synchronization normal descriptive

Answer key

### 5.24.23 Process Synchronization: GATE CSE 2002 | Question: 20



The following solution to the single producer single consumer problem uses semaphores for synchronization.

```
#define BUFFSIZE 100
buffer buf[BUFFSIZE];
int first = last = 0;
semaphore b_full = 0;
semaphore b_empty = BUFFSIZE

void producer()
{
    while(1) {
        produce an item;
        p1:.....;
        put the item into buf[first];
        first = (first+1)%BUFFSIZE;
        p2: .....;
    }
}

void consumer()
{
    while(1) {
        c1:.....
        take the item from buf[last];
        last = (last+1)%BUFFSIZE;
        c2:.....;
        consume the item;
    }
}
```

- A. Complete the dotted part of the above solution.  
 B. Using another semaphore variable, insert one line statement each immediately after *p1*, immediately before *p2*, immediately after *c1* and immediately before *c2* so that the program works correctly for multiple producers and consumers.

Answer key

## 5.24.24 Process Synchronization: GATE CSE 2003 | Question: 80



Suppose we want to synchronize two concurrent processes  $P$  and  $Q$  using binary semaphores  $S$  and  $T$ . The code for the processes  $P$  and  $Q$  is shown below.

| Process P:                                             | Process Q:                                             |
|--------------------------------------------------------|--------------------------------------------------------|
| <pre>while(1){ W:   print '0';   print '0'; X: }</pre> | <pre>while(1){ Y:   print '1';   print '1'; Z: }</pre> |

Synchronization statements can be inserted only at points  $W, X, Y$ , and  $Z$

Which of the following will always lead to an output starting with '001100110011'?

- A.  $P(S)$  at  $W, V(S)$  at  $X, P(T)$  at  $Y, V(T)$  at  $Z, S$  and  $T$  initially 1
- B.  $P(S)$  at  $W, V(T)$  at  $X, P(T)$  at  $Y, V(S)$  at  $Z, S$  initially 1, and  $T$  initially 0
- C.  $P(S)$  at  $W, V(T)$  at  $X, P(T)$  at  $Y, V(S)$  at  $Z, S$  and  $T$  initially 1
- D.  $P(S)$  at  $W, V(S)$  at  $X, P(T)$  at  $Y, V(T)$  at  $Z, S$  initially 1, and  $T$  initially 0

Answer key

## 5.24.25 Process Synchronization: GATE CSE 2003 | Question: 81



Suppose we want to synchronize two concurrent processes  $P$  and  $Q$  using binary semaphores  $S$  and  $T$ . The code for the processes  $P$  and  $Q$  is shown below.

| Process P:                                              | Process Q:                                              |
|---------------------------------------------------------|---------------------------------------------------------|
| <pre>while(1) { W:   print '0';   print '0'; X: }</pre> | <pre>while(1) { Y:   print '1';   print '1'; Z: }</pre> |

Synchronization statements can be inserted only at points  $W, X, Y$ , and  $Z$

Which of the following will ensure that the output string never contains a substring of the form  $01^n0$  and  $10^n1$  where  $n$  is odd?

- A.  $P(S)$  at  $W, V(S)$  at  $X, P(T)$  at  $Y, V(T)$  at  $Z, S$  and  $T$  initially 1
- B.  $P(S)$  at  $W, V(T)$  at  $X, P(T)$  at  $Y, V(S)$  at  $Z, S$  and  $T$  initially 1
- C.  $P(S)$  at  $W, V(S)$  at  $X, P(S)$  at  $Y, V(S)$  at  $Z, S$  initially 1
- D.  $V(S)$  at  $W, V(T)$  at  $X, P(S)$  at  $Y, P(T)$  at  $Z, S$  and  $T$  initially 1

Answer key

## 5.24.26 Process Synchronization: GATE CSE 2004 | Question: 48



Consider two processes  $P_1$  and  $P_2$  accessing the shared variables  $X$  and  $Y$  protected by two binary

semaphores  $S_X$  and  $S_Y$  respectively, both initialized to 1.  $P$  and  $V$  denote the usual semaphore operators, where  $P$  decrements the semaphore value, and  $V$  increments the semaphore value. The pseudo-code of  $P_1$  and  $P_2$  is as follows:

| $P_1$ :            | $P_2$ :            |
|--------------------|--------------------|
| While true do {    | While true do {    |
| $L_1 : \dots\dots$ | $L_3 : \dots\dots$ |
| $L_2 : \dots\dots$ | $L_4 : \dots\dots$ |
| $X = X + 1;$       | $Y = Y + 1;$       |
| $Y = Y - 1;$       | $X = Y - 1;$       |
| $V(S_X);$          | $V(S_Y);$          |
| $V(S_Y);$          | $V(S_X);$          |
| }                  | }                  |

In order to avoid deadlock, the correct operators at  $L_1$ ,  $L_2$ ,  $L_3$  and  $L_4$  are respectively.

- A.  $P(S_Y), P(S_X); P(S_X), P(S_Y)$
- B.  $P(S_X), P(S_Y); P(S_Y), P(S_X)$
- C.  $P(S_X), P(S_X); P(S_Y), P(S_Y)$
- D.  $P(S_X), P(S_Y); P(S_X), P(S_Y)$

gatecse-2004    operating-system    process-synchronization    normal

Answer key 

#### 5.24.27 Process Synchronization: GATE CSE 2006 | Question: 61



The atomic *fetch-and-set*  $x, y$  instruction unconditionally sets the memory location  $x$  to 1 and fetches the old value of  $x$  in  $y$  without allowing any intervening access to the memory location  $x$ . Consider the following implementation of  $P$  and  $V$  functions on a binary semaphore  $S$ .

```
void P (binary_semaphore *s) {
    unsigned y;
    unsigned *x = &(s->value);
    do {
        fetch-and-set x, y;
    } while (y);
}

void V (binary_semaphore *s) {
    S->value = 0;
}
```

Which one of the following is true?

- A. The implementation may not work if context switching is disabled in  $P$
- B. Instead of using *fetch-and-set*, a pair of normal load/store can be used
- C. The implementation of  $V$  is wrong
- D. The code does not implement a binary semaphore

gatecse-2006    operating-system    process-synchronization    normal

Answer key 

#### 5.24.28 Process Synchronization: GATE CSE 2006 | Question: 78



Barrier is a synchronization construct where a set of processes synchronizes globally i.e., each process in the set arrives at the barrier and waits for all others to arrive and then all processes leave the barrier. Let the number of processes in the set be three and  $S$  be a binary semaphore with the usual  $P$  and  $V$  functions. Consider the following  $C$  implementation of a barrier with line numbers shown on left.

```
void barrier (void) {
```

```
1: P(S);
2: process_arrived++;
3: V(S);
```

```

4: while (process_arrived !=3);
5: P(S);
6: process_left++;
7: if (process_left==3) {
8:   process_arrived = 0;
9:   process_left = 0;
10: }
11: V(S);

```

The variables *process\_arrived* and *process\_left* are shared among all processes and are initialized to zero. In a concurrent program all the three processes call the barrier function when they need to synchronize globally.

The above implementation of barrier is incorrect. Which one of the following is true?

- A. The barrier implementation is wrong due to the use of binary semaphore  $S$
- B. The barrier implementation may lead to a deadlock if two barrier in invocations are used in immediate succession.
- C. Lines 6 to 10 need not be inside a critical section
- D. The barrier implementation is correct if there are only two processes instead of three.

gatecse-2006 operating-system process-synchronization normal

Answer key

#### 5.24.29 Process Synchronization: GATE CSE 2006 | Question: 79



Barrier is a synchronization construct where a set of processes synchronizes globally i.e., each process in the set arrives at the barrier and waits for all others to arrive and then all processes leave the barrier. Let the number of processes in the set be three and  $S$  be a binary semaphore with the usual  $P$  and  $V$  functions. Consider the following  $C$  implementation of a barrier with line numbers shown on left.

void barrier (void) {

```

1 P(S);
2 process_arrived++;
3 V(S);
4 while (process_arrived !=3);
5 P(S);
6 process_left++;
7 if (process_left==3) {
8   process_arrived = 0;
9   process_left = 0;
10 }
11 V(S);

```

The variables *process\_arrived* and *process\_left* are shared among all processes and are initialized to zero. In a concurrent program all the three processes call the barrier function when they need to synchronize globally.

Which one of the following rectifies the problem in the implementation?

- A. Lines 6 to 10 are simply replaced by `process_arrived--`
- B. At the beginning of the barrier the first process to enter the barrier waits until *process\_arrived* becomes zero before proceeding to execute  $P(S)$ .
- C. Context switch is disabled at the beginning of the barrier and re-enabled at the end.
- D. The variable *process\_left* is made private instead of shared

gatecse-2006 operating-system process-synchronization normal

Answer key

#### 5.24.30 Process Synchronization: GATE CSE 2007 | Question: 58



Two processes,  $P_1$  and  $P_2$ , need to access a critical section of code. Consider the following synchronization construct used by the processes:

|                                                                                                                                                              |                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> /* P1 */ while (true) {     wants1 = true;     while (wants2 == true);     /* Critical Section */     wants1 = false; } /* Remainder section */ </pre> | <pre> /* P2 */ while (true) {     wants2 = true;     while (wants1 == true);     /* Critical Section */     wants2=false; } /* Remainder section */ </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|

Here, `wants1` and `wants2` are shared variables, which are initialized to false. Which one of the following statements is TRUE about the construct?

- A. It does not ensure mutual exclusion.
- B. It does not ensure bounded waiting.
- C. It requires that processes enter the critical section in strict alternation.
- D. It does not prevent deadlocks, but ensures mutual exclusion.

gatecse-2007   operating-system   process-synchronization   normal

Answer key 

#### 5.24.31 Process Synchronization: GATE CSE 2009 | Question: 33



The **enter\_CS()** and **leave\_CS()** functions to implement critical section of a process are realized using test-and-set instruction as follows:

```

void enter_CS(X)
{
    while(test-and-set(X));
}

```

```

void leave_CS(X)
{
    X = 0;
}

```

In the above solution,  $X$  is a memory location associated with the  $CS$  and is initialized to 0. Now consider the following statements:

- I. The above solution to  $CS$  problem is deadlock-free
- II. The solution is starvation free
- III. The processes enter  $CS$  in FIFO order
- IV. More than one process can enter  $CS$  at the same time

Which of the above statements are TRUE?

- A. (I) only
- B. (I) and (II)
- C. (II) and (III)
- D. (IV) only

gatecse-2009   operating-system   process-synchronization   normal

Answer key 

#### 5.24.32 Process Synchronization: GATE CSE 2010 | Question: 23



Consider the methods used by processes  $P1$  and  $P2$  for accessing their critical sections whenever needed, as given below. The initial values of shared boolean variables  $S1$  and  $S2$  are randomly assigned.

| Method used by P1                                 | Method used by P2                                      |
|---------------------------------------------------|--------------------------------------------------------|
| while (S1 == S2);<br>Critical Section<br>S1 = S2; | while (S1 != S2);<br>Critical Section<br>S2 = not(S1); |

Which one of the following statements describes the properties achieved?

- A. Mutual exclusion but not progress
- B. Progress but not mutual exclusion
- C. Neither mutual exclusion nor progress
- D. Both mutual exclusion and progress

gatecse-2010 operating-system process-synchronization normal

Answer key

#### 5.24.33 Process Synchronization: GATE CSE 2010 | Question: 45



The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as  $S_0 = 1$ ,  $S_1 = 0$  and  $S_2 = 0$ .

| Process P0                                                                        | Process P1                  | Process P2                  |
|-----------------------------------------------------------------------------------|-----------------------------|-----------------------------|
| while (true) {<br>wait (S0);<br>print '0';<br>release (S1);<br>release (S2);<br>} | wait (S1);<br>release (S0); | wait (S2);<br>release (S0); |

How many times will process  $P_0$  print '0'?

- A. At least twice
- B. Exactly twice
- C. Exactly thrice
- D. Exactly once

gatecse-2010 operating-system process-synchronization normal

Answer key

#### 5.24.34 Process Synchronization: GATE CSE 2012 | Question: 32



$\text{Fetch\_And\_Add}(X, i)$  is an atomic Read-Modify-Write instruction that reads the value of memory location  $X$ , increments it by the value  $i$ , and returns the old value of  $X$ . It is used in the pseudocode shown below to implement a busy-wait lock.  $L$  is an unsigned integer shared variable initialized to 0. The value of 0 corresponds to lock being available, while any non-zero value corresponds to the lock being not available.

```

AcquireLock(L){
  while (Fetch_And_Add(L,1))
    L = 1;
}

ReleaseLock(L){
  L = 0;
}

```

This implementation

- A. fails as  $L$  can overflow
- B. fails as  $L$  can take on a non-zero value when the lock is actually available
- C. works correctly but may starve some processes
- D. works correctly without starvation

gatecse-2012 operating-system process-synchronization normal

Answer key

5.24.35 Process Synchronization: GATE CSE 2013 | Question: 34



A shared variable  $x$ , initialized to zero, is operated on by four concurrent processes  $W, X, Y, Z$  as follows. Each of the processes  $W$  and  $X$  reads  $x$  from memory, increments by one, stores it to memory, and then terminates. Each of the processes  $Y$  and  $Z$  reads  $x$  from memory, decrements by two, stores it to memory, and then terminates. Each process before reading  $x$  invokes the  $P$  operation (i.e., wait) on a counting semaphore  $S$  and invokes the  $V$  operation (i.e., signal) on the semaphore  $S$  after storing  $x$  to memory. Semaphore  $S$  is initialized to two. What is the maximum possible value of  $x$  after all processes complete execution?

- A. -2                      B. -1                      C. 1                      D. 2

gatecse-2013    operating-system    process-synchronization    normal

Answer key

5.24.36 Process Synchronization: GATE CSE 2013 | Question: 39



A certain computation generates two arrays  $a$  and  $b$  such that  $a[i] = f(i)$  for  $0 \leq i < n$  and  $b[i] = g(a[i])$  for  $0 \leq i < n$ . Suppose this computation is decomposed into two concurrent processes  $X$  and  $Y$  such that  $X$  computes the array  $a$  and  $Y$  computes the array  $b$ . The processes employ two binary semaphores  $R$  and  $S$ , both initialized to zero. The array  $a$  is shared by the two processes. The structures of the processes are shown below.

Process X:

```
private i;
for (i=0; i<n; i++) {
    a[i] = f(i);
    ExitX(R, S);
}
```

Process Y:

```
private i;
for (i=0; i<n; i++) {
    EntryY(R, S);
    b[i] = g(a[i]);
}
```

Which one of the following represents the **CORRECT** implementations of ExitX and EntryY?

- |                                                                                                                                                                                                                       |                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>A. <code>ExitX(R, S) {</code><br/> <code>  P(R);</code><br/> <code>  V(S);</code><br/> <code>}</code><br/> <code>EntryY(R, S) {</code><br/> <code>  P(S);</code><br/> <code>  V(R);</code><br/> <code>}</code></p> | <p>B. <code>ExitX(R, S) {</code><br/> <code>  V(R);</code><br/> <code>  V(S);</code><br/> <code>}</code><br/> <code>EntryY(R, S) {</code><br/> <code>  P(R);</code><br/> <code>  P(S);</code><br/> <code>}</code></p> |
| <p>C. <code>ExitX(R, S) {</code><br/> <code>  P(S);</code><br/> <code>  V(R);</code><br/> <code>}</code><br/> <code>EntryY(R, S) {</code><br/> <code>  V(S);</code><br/> <code>  P(R);</code><br/> <code>}</code></p> | <p>D. <code>ExitX(R, S) {</code><br/> <code>  V(R);</code><br/> <code>  P(S);</code><br/> <code>}</code><br/> <code>EntryY(R, S) {</code><br/> <code>  V(S);</code><br/> <code>  P(R);</code><br/> <code>}</code></p> |

gatecse-2013    operating-system    process-synchronization    normal

Answer key

5.24.37 Process Synchronization: GATE CSE 2014 | Set 2 | Question: 31



Consider the procedure below for the *Producer-Consumer* problem which uses semaphores:

```
semaphore n = 0;
semaphore s = 1;
```



```
void producer()
{
    while(true)
    {
        produce();
        semWait(s);
        addToBuffer();
        semSignal(s);
        semSignal(n);
    }
}
```

```
void consumer()
{
    while(true)
    {
        semWait(s);
        semWait(n);
        removeFromBuffer();
        semSignal(s);
        consume();
    }
}
```

Which one of the following is **TRUE**?

- A. The producer will be able to add an item to the buffer, but the consumer can never consume it.
- B. The consumer will remove no more than one item from the buffer.
- C. Deadlock occurs if the consumer succeeds in acquiring semaphore  $s$  when the buffer is empty.
- D. The starting value for the semaphore  $n$  must be 1 and not 0 for deadlock-free operation.

gatecse-2014-set2   operating-system   process-synchronization   normal

Answer key 

#### 5.24.38 Process Synchronization: GATE CSE 2015 | Set 1 | Question: 9



The following two functions  $P1$  and  $P2$  that share a variable  $B$  with an initial value of 2 execute concurrently.

|                                                   |                                                   |
|---------------------------------------------------|---------------------------------------------------|
| $P1() \{$<br>$C = B - 1;$<br>$B = 2 * C;$<br>$\}$ | $P2() \{$<br>$D = 2 * B;$<br>$B = D - 1;$<br>$\}$ |
|---------------------------------------------------|---------------------------------------------------|

The number of distinct values that  $B$  can possibly take after the execution is \_\_\_\_\_.

gatecse-2015-set1   operating-system   process-synchronization   normal   numerical-answers

Answer key 

#### 5.24.39 Process Synchronization: GATE CSE 2015 | Set 3 | Question: 10



Two processes  $X$  and  $Y$  need to access a critical section. Consider the following synchronization construct used by both the processes

| Process X                                                                                                                                                                                              | Process Y                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> /* other code for process X */ while (true) {     varP = true;     while (varQ == true)     {         /* Critical Section */         varP = false;     } } /* other code for process X */ </pre> | <pre> /* other code for process Y */ while (true) {     varQ = true;     while (varP == true)     {         /* Critical Section */         varQ = false;     } } /* other code for process Y */ </pre> |

Here  $varP$  and  $varQ$  are shared variables and both are initialized to false. Which one of the following statements is true?

- A. The proposed solution prevents deadlock but fails to guarantee mutual exclusion
- B. The proposed solution guarantees mutual exclusion but fails to prevent deadlock
- C. The proposed solution guarantees mutual exclusion and prevents deadlock
- D. The proposed solution fails to prevent deadlock and fails to guarantee mutual exclusion

gatecse-2015-set3 operating-system process-synchronization normal

Answer key

#### 5.24.40 Process Synchronization: GATE CSE 2016 | Set 1 | Question: 50



Consider the following proposed solution for the critical section problem. There are  $n$  processes :  $P_0 \dots P_{n-1}$ . In the code, function  $\text{pmax}$  returns an integer not smaller than any of its arguments. For all  $i$ ,  $t[i]$  is initialized to zero.

Code for  $P_i$ :

```

do {
    c[i]=1; t[i]= pmax (t[0],...,t[n-1])+1; c[i]=0;
    for every j != i in {0,...,n-1} {
        while (c[j]);
        while (t[j] != 0 && t[j] <= t[i]);
    }
    Critical Section;
    t[i]=0;

    Remainder Section;
} while (true);

```

Which of the following is TRUE about the above solution?

- A. At most one process can be in the critical section at any time
- B. The bounded wait condition is satisfied
- C. The progress condition is satisfied
- D. It cannot cause a deadlock

gatecse-2016-set1 operating-system process-synchronization difficult ambiguous

Answer key

#### 5.24.41 Process Synchronization: GATE CSE 2016 | Set 2 | Question: 48



Consider the following two-process synchronization solution.

|                                                      |                                                      |
|------------------------------------------------------|------------------------------------------------------|
| PROCESS 0                                            | Process 1                                            |
| Entry: loop while (turn == 1);<br>(critical section) | Entry: loop while (turn == 0);<br>(critical section) |
| Exit: turn = 1;                                      | Exit turn = 0;                                       |

The shared variable turn is initialized to zero. Which one of the following is TRUE?

- A. This is a correct two- process synchronization solution.
- B. This solution violates mutual exclusion requirement.
- C. This solution violates progress requirement.
- D. This solution violates bounded wait requirement.

gatecse-2016-set2 operating-system process-synchronization normal

Answer key

#### 5.24.42 Process Synchronization: GATE CSE 2017 | Set 1 | Question: 27



A multithreaded program  $P$  executes with  $x$  number of threads and uses  $y$  number of locks for ensuring mutual exclusion while operating on shared memory locations. All locks in the program are *non-reentrant*, i.e., if a thread holds a lock  $l$ , then it cannot re-acquire lock  $l$  without releasing it. If a thread is unable to acquire a lock, it blocks until the lock becomes available. The *minimum* value of  $x$  and the *minimum* value of  $y$  together for which execution of  $P$  can result in a deadlock are:

- A.  $x = 1, y = 2$
- B.  $x = 2, y = 1$
- C.  $x = 2, y = 2$
- D.  $x = 1, y = 1$

gatecse-2017-set1 operating-system process-synchronization normal

Answer key

#### 5.24.43 Process Synchronization: GATE CSE 2018 | Question: 40



Consider the following solution to the producer-consumer synchronization problem. The shared buffer size is  $N$ . Three semaphores *empty*, *full* and *mutex* are defined with respective initial values of 0,  $N$  and 1. Semaphore *empty* denotes the number of available slots in the buffer, for the consumer to read from. Semaphore *full* denotes the number of available slots in the buffer, for the producer to write to. The placeholder variables, denoted by  $P$ ,  $Q$ ,  $R$  and  $S$ , in the code below can be assigned either *empty* or *full*. The valid semaphore operations are: *wait()* and *signal()*.

|                                                                                                                                 |                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Producer:</b><br>do {<br>wait (P);<br>wait (mutex);<br>//Add item to buffer<br>signal (mutex);<br>signal (Q);<br>}while (1); | <b>Consumer:</b><br>do {<br>wait (R);<br>wait (mutex);<br>//consume item from buffer<br>signal (mutex);<br>signal (S);<br>}while (1); |
|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|

Which one of the following assignments to  $P$ ,  $Q$ ,  $R$  and  $S$  will yield the correct solution?

- A.  $P : full, Q : full, R : empty, S : empty$
- B.  $P : empty, Q : empty, R : full, S : full$
- C.  $P : full, Q : empty, R : empty, S : full$
- D.  $P : empty, Q : full, R : full, S : empty$

## Answer key

## 5.24.44 Process Synchronization: GATE CSE 2019 | Question: 23



Consider three concurrent processes  $P_1$ ,  $P_2$  and  $P_3$  as shown below, which access a shared variable  $D$  that has been initialized to 100.

| $P_1$        | $P_2$        | $P_3$        |
|--------------|--------------|--------------|
| :            | :            | :            |
| :            | :            | :            |
| $D = D + 20$ | $D = D - 50$ | $D = D + 10$ |
| :            | :            | :            |
| :            | :            | :            |

The processes are executed on a uniprocessor system running a time-shared operating system. If the minimum and maximum possible values of  $D$  after the three processes have completed execution are  $X$  and  $Y$  respectively, then the value of  $Y - X$  is \_\_\_\_\_

## Answer key

## 5.24.45 Process Synchronization: GATE CSE 2024 | Set 2 | Question: 36



Consider a multi-threaded program with two threads T1 and T2. The threads share two semaphores:  $s_1$  (initialized to 1) and  $s_2$  (initialized to 0). The threads also share a global variable  $x$  (initialized to 0). The threads execute the code shown below.

```
//code of T 1
wait (s1);
x = x+1;
print (x);
wait (s2);
signal(s1);
```

```
// code of T2
wait (s1);
x= x+1;
print (x) ;
signal (s2);
signal (s1);
```

Which of the following outcomes is/are possible when threads T1 and T2 execute concurrently?

- A. T1 runs first and prints 1, T2 runs next and prints 2
- B. T2 runs first and prints 1, T1 runs next and prints 2
- C. T1 runs first and prints 1, T2 does not print anything (deadlock)
- D. T2 runs first and prints 1, T1 does not print anything (deadlock)

## Answer key

## 5.24.46 Process Synchronization: GATE IT 2004 | Question: 65



The semaphore variables full, empty and mutex are initialized to 0,  $n$  and 1, respectively. Process  $P_1$  repeatedly adds one item at a time to a buffer of size  $n$ , and process  $P_2$  repeatedly removes one item at a time from the same buffer using the programs given below. In the programs,  $K$ ,  $L$ ,  $M$  and  $N$  are unspecified statements.

$P_1$

```
while (1) {
    K;
    P(mutex);
    Add an item to the buffer;
    V(mutex);
    L;
}
```

$P_2$

```
while (1) {
    M;
    P(mutex);
    Remove an item from the buffer;
    V(mutex);
    N;
}
```

The statements  $K$ ,  $L$ ,  $M$  and  $N$  are respectively

- A. P(full), V(empty), P(full), V(empty)  
 B. P(full), V(empty), P(empty), V(full)  
 C. P(empty), V(full), P(empty), V(full)  
 D. P(empty), V(full), P(full), V(empty)

gateit-2004 operating-system process-synchronization normal

Answer key

#### 5.24.47 Process Synchronization: GATE IT 2005 | Question: 41



Given below is a program which when executed spawns two concurrent processes :  
 semaphore  $X := 0$ ;  
 /\* Process now forks into concurrent processes  $P_1$  &  $P_2$  \*/

| $P_1$                                              | $P_2$                                              |
|----------------------------------------------------|----------------------------------------------------|
| repeat forever<br>$V(X)$ ;<br>Compute;<br>$P(X)$ ; | repeat forever<br>$P(X)$ ;<br>Compute;<br>$V(X)$ ; |

Consider the following statements about processes  $P_1$  and  $P_2$  :

- I. It is possible for process  $P_1$  to starve.  
 II. It is possible for process  $P_2$  to starve.

Which of the following holds?

- A. Both (I) and (II) are true.  
 B. (I) is true but (II) is false.  
 C. (II) is true but (I) is false  
 D. Both (I) and (II) are false

gateit-2005 operating-system process-synchronization normal

Answer key

#### 5.24.48 Process Synchronization: GATE IT 2005 | Question: 42



Two concurrent processes  $P_1$  and  $P_2$  use four shared resources  $R_1$ ,  $R_2$ ,  $R_3$  and  $R_4$ , as shown below.

| $P_1$                                                                | $P_2$                                                                |
|----------------------------------------------------------------------|----------------------------------------------------------------------|
| Compute:<br>Use $R_1$ ;<br>Use $R_2$ ;<br>Use $R_3$ ;<br>Use $R_4$ ; | Compute;<br>Use $R_1$ ;<br>Use $R_2$ ;<br>Use $R_3$ ;<br>Use $R_4$ ; |

Both processes are started at the same time, and each resource can be accessed by only one process at a time  
 The following scheduling constraints exist between the access of resources by the processes:

- $P_2$  must complete use of  $R_1$  before  $P_1$  gets access to  $R_1$ .
- $P_1$  must complete use of  $R_2$  before  $P_2$  gets access to  $R_2$ .
- $P_2$  must complete use of  $R_3$  before  $P_1$  gets access to  $R_3$ .
- $P_1$  must complete use of  $R_4$  before  $P_2$  gets access to  $R_4$ .

There are no other scheduling constraints between the processes. If only binary semaphores are used to enforce the above scheduling constraints, what is the minimum number of binary semaphores needed?

- A. 1                                      B. 2                                      C. 3                                      D. 4

gateit-2005    operating-system    process-synchronization    normal

Answer key 

#### 5.24.49 Process Synchronization: GATE IT 2006 | Question: 55



Consider the solution to the bounded buffer producer/consumer problem by using general semaphores  $S$ ,  $F$ , and  $E$ . The semaphore  $S$  is the mutual exclusion semaphore initialized to 1. The semaphore  $F$  corresponds to the number of free slots in the buffer and is initialized to  $N$ . The semaphore  $E$  corresponds to the number of elements in the buffer and is initialized to 0.

| Producer Process               | Consumer Process                |
|--------------------------------|---------------------------------|
| Produce an item;               | Wait( $E$ );                    |
| Wait( $F$ );                   | Wait( $S$ );                    |
| Wait( $S$ );                   | Remove an item from the buffer; |
| Append the item to the buffer; | Signal( $S$ );                  |
| Signal( $S$ );                 | Signal( $F$ );                  |
| Signal( $E$ );                 | Consume the item;               |

Which of the following interchange operations may result in a deadlock?

- Interchanging Wait ( $F$ ) and Wait ( $S$ ) in the Producer process
- Interchanging Signal ( $S$ ) and Signal ( $F$ ) in the Consumer process

- A. (I) only                                      B. (II) only  
C. Neither (I) nor (II)                      D. Both (I) and (II)

gateit-2006    operating-system    process-synchronization    normal

Answer key 

#### 5.24.50 Process Synchronization: GATE IT 2007 | Question: 10



Processes  $P_1$  and  $P_2$  use `critical_flag` in the following routine to achieve mutual exclusion. Assume that `critical_flag` is initialized to FALSE in the main program.

```
get_exclusive_access ()
{
    if (critical_flag == FALSE) {
        critical_flag = TRUE ;
        critical_region () ;
        critical_flag = FALSE;
    }
}
```

Consider the following statements.

- It is possible for both  $P_1$  and  $P_2$  to access `critical_region` concurrently.
- This may lead to a deadlock.

Which of the following holds?

- A. (i) is false (ii) is true                      B. Both (i) and (ii) are false  
C. (i) is true (ii) is false                      D. Both (i) and (ii) are true

gateit-2007    operating-system    process-synchronization    normal

Answer key

5.24.51 Process Synchronization: GATE IT 2007 | Question: 56



Synchronization in the classical readers and writers problem can be achieved through use of semaphores. In the following incomplete code for readers-writers problem, two binary semaphores mutex and wrt are used to obtain synchronization

```
wait (wrt)
writing is performed
signal (wrt)
wait (mutex)
readcount = readcount + 1
if readcount = 1 then S1
S2
reading is performed
S3
readcount = readcount - 1
if readcount = 0 then S4
signal (mutex)
```

The values of  $S1, S2, S3, S4$ , (in that order) are

- A. signal (mutex), wait (wrt), signal (wrt), wait (mutex)
- B. signal (wrt), signal (mutex), wait (mutex), wait (wrt)
- C. wait (wrt), signal (mutex), wait (mutex), signal (wrt)
- D. signal (mutex), wait (mutex), signal (mutex), wait (mutex)

gateit-2007 operating-system process-synchronization normal

Answer key

5.24.52 Process Synchronization: GATE IT 2008 | Question: 53



The following is a code with two threads, producer and consumer, that can run in parallel. Further,  $S$  and  $Q$  are binary semaphores quipped with the standard  $P$  and  $V$  operations.

```
semaphore S = 1, Q = 0;
integer x;

producer:      consumer:
while (true) do while (true) do
  P(S);        P(Q);
  x = produce (); consume (x);
  V(Q);        V(S);
done          done
```

Which of the following is TRUE about the program above?

- A. The process can deadlock
- B. One of the threads can starve
- C. Some of the items produced by the producer may be lost
- D. Values generated and stored in 'x' by the producer will always be consumed before the producer can generate a new value

gateit-2008 operating-system process-synchronization normal

Answer key

5.25

Resource Allocation (27)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(10Q\)](#)

5.25.1 Resource Allocation: GATE CSE 1988 | Question: 11



A number of processes could be in a deadlock state if none of them can execute due to non-availability of sufficient resources. Let  $P_i, 0 \leq i \leq 4$  represent five processes and let there be four resources types  $r_j, 0 \leq j \leq 3$ . Suppose the following data structures have been used.

**Available:** A vector of length 4 such that if  $\text{Available}[i] = k$ , there are  $k$  instances of resource type  $r_j$  available in

the system.

**Allocation.** A  $5 \times 4$  matrix defining the number of each type currently allocated to each process. If  $Allocation[i, j] = k$  then process  $p_i$  is currently allocated  $k$  instances of resource type  $r_j$ .

**Max.** A  $5 \times 4$  matrix indicating the maximum resource need of each process. If  $Max[i, j] = k$  then process  $p_i$ , may need a maximum of  $k$  instances of resource type  $r_j$  in order to complete the task.

Assume that system allocated resources only when it does not lead into an unsafe state such that resource requirements in future never cause a deadlock state. Now consider the following snapshot of the system.

| Allocation |       |       |       |       | Max |       |       |       |       | Available |       |       |       |       |
|------------|-------|-------|-------|-------|-----|-------|-------|-------|-------|-----------|-------|-------|-------|-------|
|            | $r_0$ | $r_1$ | $r_2$ | $r_3$ |     | $r_0$ | $r_1$ | $r_2$ | $r_3$ |           | $r_0$ | $r_1$ | $r_2$ | $r_3$ |
| $p_0$      | 0     | 0     | 1     | 2     |     | 0     | 0     | 1     | 2     |           | 1     | 5     | 2     | 0     |
| $p_1$      | 1     | 0     | 0     | 0     |     | 1     | 7     | 5     | 0     |           |       |       |       |       |
| $p_2$      | 1     | 3     | 5     | 4     |     | 2     | 3     | 5     | 6     |           |       |       |       |       |
| $p_3$      | 0     | 6     | 3     | 2     |     | 0     | 6     | 5     | 2     |           |       |       |       |       |
| $p_4$      | 0     | 0     | 1     | 4     |     | 0     | 6     | 5     | 6     |           |       |       |       |       |

Is the system currently in a safe state? If yes, explain why.

gate1988 normal descriptive operating-system resource-allocation

Answer key

#### 5.25.2 Resource Allocation: GATE CSE 1989 | Question: 11a



- i. A system of four concurrent processes,  $P, Q, R$  and  $S$ , use shared resources  $A, B$  and  $C$ . The sequences in which processes,  $P, Q, R$  and  $S$  request and release resources are as follows:

- Process P:
1. P requests A
  2. P requests B
  3. P releases A
  4. P releases B
- Process Q:
1. Q requests C
  2. Q requests A
  3. Q releases C
  4. P releases A
- Process R:
1. R requests B
  2. R requests C
  3. R releases B
  4. R releases C
- Process S:
1. S requests A
  2. S requests C
  3. S releases A
  4. S releases C

If a resource is free, it is granted to a requesting process immediately. There is no preemption of granted resources. A resource is taken back from a process only when the process explicitly releases it.

Can the system of four processes get into a deadlock? If yes, give a sequence (ordering) of operations (for requesting and releasing resources) of these processes which leads to a deadlock.

- ii. Will the processes always get into a deadlock? If your answer is no, give a sequence of these operations which leads to completion of all processes.
- iii. What strategies can be used to prevent deadlocks in a system of concurrent processes using shared resources



if preemption of granted resources is not allowed?

descriptive gate1989 operating-system resource-allocation

Answer key

### 5.25.3 Resource Allocation: GATE CSE 1992 | Question: 02-xi

A computer system has 6 tape devices, with  $n$  processes competing for them. Each process may need 3 tape drives. The maximum value of  $n$  for which the system is guaranteed to be deadlock-free is:

- A. 2 B. 3 C. 4 D. 1

gate1992 operating-system resource-allocation normal multiple-selects

Answer key

### 5.25.4 Resource Allocation: GATE CSE 1993 | Question: 7.9, UGCNET-Dec2012-III: 41

Consider a system having  $m$  resources of the same type. These resources are shared by 3 processes  $A$ ,  $B$ , and  $C$  which have peak demands of 3, 4, and 6 respectively. For what value of  $m$  deadlock will not occur?

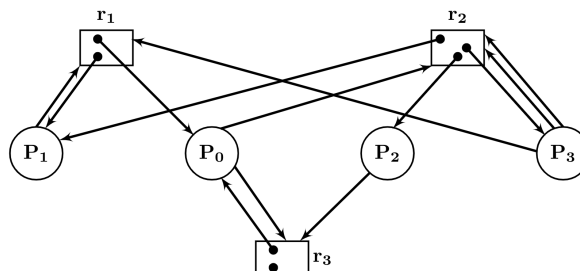
- A. 7 B. 9 C. 10 D. 13  
E. 15

gate1993 operating-system resource-allocation normal ugcnetcse-dec2012-paper3 multiple-selects

Answer key

### 5.25.5 Resource Allocation: GATE CSE 1994 | Question: 28

Consider the resource allocation graph in the figure.



- A. Find if the system is in a deadlock state  
B. Otherwise, find a safe sequence

gate1994 operating-system resource-allocation normal descriptive

Answer key

### 5.25.6 Resource Allocation: GATE CSE 1996 | Question: 22

A computer system uses the Banker's Algorithm to deal with deadlocks. Its current state is shown in the table below, where  $P0$ ,  $P1$ ,  $P2$  are processes, and  $R0$ ,  $R1$ ,  $R2$  are resource types.

| Maximum Need |      |      |      | Current Allocation |      |      |      | Available |      |      |
|--------------|------|------|------|--------------------|------|------|------|-----------|------|------|
|              | $R0$ | $R1$ | $R2$ |                    | $R0$ | $R1$ | $R2$ | $R0$      | $R1$ | $R2$ |
| $P0$         | 4    | 1    | 2    | $P0$               | 1    | 0    | 2    | 2         | 2    | 0    |
| $P1$         | 1    | 5    | 1    | $P1$               | 0    | 3    | 1    |           |      |      |
| $P2$         | 1    | 2    | 3    | $P2$               | 1    | 0    | 2    |           |      |      |

- A. Show that the system can be in this state  
B. What will the system do on a request by process  $P0$  for one unit of resource type  $R1$ ?

Answer key

## 5.25.7 Resource Allocation: GATE CSE 1997 | Question: 6.7



An operating system contains 3 user processes each requiring 2 units of resource  $R$ . The minimum number of units of  $R$  such that no deadlocks will ever arise is

- A. 3                                      B. 5                                      C. 4                                      D. 6

Answer key

## 5.25.8 Resource Allocation: GATE CSE 1997 | Question: 75



An operating system handles requests to resources as follows.

A process (which asks for some resources, uses them for some time and then exits the system) is assigned a unique timestamp at when it starts. The timestamps are monotonically increasing with time. Let us denote the timestamp of a process  $P$  by  $TS(P)$ .

When a process  $P$  requests for a resource the  $OS$  does the following:

- If no other process is currently holding the resource, the  $OS$  awards the resource to  $P$ .
- If some process  $Q$  with  $TS(Q) < TS(P)$  is holding the resource, the  $OS$  makes  $P$  wait for the resources.
- If some process  $Q$  with  $TS(Q) > TS(P)$  is holding the resource, the  $OS$  restarts  $Q$  and awards the resources to  $P$ . (Restarting means taking back the resources held by a process, killing it and starting it again with the same timestamp)

When a process releases a resource, the process with the smallest timestamp (if any) amongst those waiting for the resource is awarded the resource.

- A. Can a deadlock ever arise? If yes, show how. If not prove it.  
B. Can a process  $P$  ever starve? If yes, show how. If not prove it.

Answer key

## 5.25.9 Resource Allocation: GATE CSE 1998 | Question: 1.32



A computer has six tape drives, with  $n$  processes competing for them. Each process may need two drives. What is the maximum value of  $n$  for the system to be deadlock free?

- A. 6                                      B. 5                                      C. 4                                      D. 3

Answer key

## 5.25.10 Resource Allocation: GATE CSE 2000 | Question: 2.23



Which of the following is not a valid deadlock prevention scheme?

- A. Release all resources before requesting a new resource.  
B. Number the resources uniquely and never request a lower numbered resource than the last one requested.  
C. Never request a resource after releasing any resource.  
D. Request and all required resources be allocated before execution.

Answer key

### 5.25.11 Resource Allocation: GATE CSE 2001 | Question: 19



Two concurrent processes  $P1$  and  $P2$  want to use resources  $R1$  and  $R2$  in a mutually exclusive manner. Initially,  $R1$  and  $R2$  are free. The programs executed by the two processes are given below.

| Program for $P1$ :                  | Program for $P2$ :                  |
|-------------------------------------|-------------------------------------|
| S1: While ( $R1$ is busy) do no-op; | Q1: While ( $R1$ is busy) do no-op; |
| S2: Set $R1 \leftarrow$ busy;       | Q2: Set $R1 \leftarrow$ busy;       |
| S3: While ( $R2$ is busy) do no-op; | Q3: While ( $R2$ is busy) do no-op; |
| S4: Set $R2 \leftarrow$ busy;       | Q4: Set $R2 \leftarrow$ busy;       |
| S5: Use $R1$ and $R2$ ;             | Q5: Use $R1$ and $R2$ ;             |
| S6: Set $R1 \leftarrow$ free;       | Q6: Set $R2 \leftarrow$ free;       |
| S7: Set $R2 \leftarrow$ free;       | Q7: Set $R1 \leftarrow$ free;       |

- Is mutual exclusion guaranteed for  $R1$  and  $R2$ ? If not show a possible interleaving of the statements of  $P1$  and  $P2$  such mutual exclusion is violated (i.e., both  $P1$  and  $P2$  use  $R1$  and  $R2$  at the same time).
- Can deadlock occur in the above program? If yes, show a possible interleaving of the statements of  $P1$  and  $P2$  leading to deadlock.
- Exchange the statements  $Q1$  and  $Q3$  and statements  $Q2$  and  $Q4$ . Is mutual exclusion guaranteed now? Can deadlock occur?

gatecse-2001 operating-system resource-allocation normal descriptive

Answer key

### 5.25.12 Resource Allocation: GATE CSE 2005 | Question: 71



Suppose  $n$  processes,  $P_1, \dots, P_n$  share  $m$  identical resource units, which can be reserved and released one at a time. The maximum resource requirement of process  $P_i$  is  $s_i$ , where  $s_i > 0$ . Which one of the following is a sufficient condition for ensuring that deadlock does not occur?

- $\forall i, s_i < m$
- $\forall i, s_i < n$
- $\sum_{i=1}^n s_i < (m + n)$
- $\sum_{i=1}^n s_i < (m \times n)$

gatecse-2005 operating-system resource-allocation normal

Answer key

### 5.25.13 Resource Allocation: GATE CSE 2006 | Question: 66



Consider the following snapshot of a system running  $n$  processes. Process  $i$  is holding  $x_i$  instances of a resource  $R$ ,  $1 \leq i \leq n$ . Currently, all instances of  $R$  are occupied. Further, for all  $i$ , process  $i$  has placed a request for an additional  $y_i$  instances while holding the  $x_i$  instances it already has. There are exactly two processes  $p$  and  $q$  and such that  $y_p = y_q = 0$ . Which one of the following can serve as a necessary condition to guarantee that the system is not approaching a deadlock?

- $\min(x_p, x_q) < \max_{k \neq p, q} y_k$
- $x_p + x_q \geq \min_{k \neq p, q} y_k$
- $\max(x_p, x_q) > 1$
- $\min(x_p, x_q) > 1$

gatecse-2006 operating-system resource-allocation normal

Answer key

#### 5.25.14 Resource Allocation: GATE CSE 2007 | Question: 57



A single processor system has three resource types  $X$ ,  $Y$  and  $Z$ , which are shared by three processes. There are 5 units of each resource type. Consider the following scenario, where the column **alloc** denotes the number of units of each resource type allocated to each process, and the column **request** denotes the number of units of each resource type requested by a process in order to complete execution. Which of these processes will finish **LAST**?

|           | alloc |   |   | request |   |   |
|-----------|-------|---|---|---------|---|---|
|           | X     | Y | Z | X       | Y | Z |
| <b>P0</b> | 1     | 2 | 1 | 1       | 0 | 3 |
| <b>P1</b> | 2     | 0 | 1 | 0       | 1 | 2 |
| <b>P2</b> | 2     | 2 | 1 | 1       | 2 | 0 |

- A.  $P0$   
 C.  $P2$   
 B.  $P1$   
 D. None of the above, since the system is in a deadlock

gatecse-2007 operating-system resource-allocation normal

Answer key

#### 5.25.15 Resource Allocation: GATE CSE 2008 | Question: 65



Which of the following is NOT true of deadlock prevention and deadlock avoidance schemes?

- A. In deadlock prevention, the request for resources is always granted if the resulting state is safe  
 B. In deadlock avoidance, the request for resources is always granted if the resulting state is safe  
 C. Deadlock avoidance is less restrictive than deadlock prevention  
 D. Deadlock avoidance requires knowledge of resource requirements *a priori*..

gatecse-2008 operating-system easy resource-allocation

Answer key

#### 5.25.16 Resource Allocation: GATE CSE 2009 | Question: 30



Consider a system with 4 types of resources  $R1$  (3 units),  $R2$  (2 units),  $R3$  (3 units),  $R4$  (2 units). A non-preemptive resource allocation policy is used. At any given instance, a request is not entertained if it cannot be completely satisfied. Three processes  $P1$ ,  $P2$ ,  $P3$  request the resources as follows if executed independently.

| Process P1:                                                                                                                                                                                                                                                             | Process P2:                                                                                                                                                             | Process P3:                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $t = 0$ : requests 2 units of $R2$<br>$t = 1$ : requests 1 unit of $R3$<br>$t = 3$ : requests 2 units of $R1$<br>$t = 5$ : releases 1 unit of $R2$ and 1 unit of $R1$<br>$t = 7$ : releases 1 unit of $R3$<br>$t = 8$ : requests 2 units of $R4$<br>$t = 10$ : Finishes | $t = 0$ : requests 2 units of $R3$<br>$t = 2$ : requests 1 unit of $R4$<br>$t = 4$ : requests 1 unit of $R1$<br>$t = 6$ : releases 1 unit of $R3$<br>$t = 8$ : Finishes | $t = 0$ : requests 1 unit of $R4$<br>$t = 2$ : requests 2 units of $R1$<br>$t = 5$ : releases 2 units of $R1$<br>$t = 7$ : requests 1 unit of $R2$<br>$t = 8$ : requests 1 unit of $R3$<br>$t = 9$ : Finishes |

Which one of the following statements is TRUE if all three processes run concurrently starting at time  $t = 0$ ?

- A. All processes will finish without any deadlock  
 C. Only  $P1$  and  $P3$  will be in deadlock  
 B. Only  $P1$  and  $P2$  will be in deadlock  
 D. All three processes will be in deadlock

gatecse-2009 operating-system resource-allocation normal

Answer key

## 5.25.17 Resource Allocation: GATE CSE 2010 | Question: 46



A system has  $n$  resources  $R_0, \dots, R_{n-1}$ , and  $k$  processes  $P_0, \dots, P_{k-1}$ . The implementation of the resource request logic of each process  $P_i$  is as follows:

```
if( $i \% 2 == 0$ ) {
    if( $i < n$ ) request  $R_i$ ;
    if( $i + 2 < n$ ) request  $R_{i+2}$ ; }
else {
    if( $i < n$ ) request  $R_{n-i}$ ;
    if( $i + 2 < n$ ) request  $R_{n-i-2}$ ; }
```

In which of the following situations is a deadlock possible?

- A.  $n = 40, k = 26$       B.  $n = 21, k = 12$       C.  $n = 20, k = 10$       D.  $n = 41, k = 19$

gatecse-2010    operating-system    resource-allocation    normal

Answer key

## 5.25.18 Resource Allocation: GATE CSE 2013 | Question: 16



Three concurrent processes  $X$ ,  $Y$ , and  $Z$  execute three different code segments that access and update certain shared variables. Process  $X$  executes the  $P$  operation (i.e., *wait*) on semaphores  $a$ ,  $b$ , and  $c$ ; process  $Y$  executes the  $P$  operation on semaphores  $b$ ,  $c$ , and  $d$ ; process  $Z$  executes the  $P$  operation on semaphores  $c$ ,  $d$ , and  $a$  before entering the respective code segments. After completing the execution of its code segment, each process invokes the  $V$  operation (i.e., *signal*) on its three semaphores. All semaphores are binary semaphores initialized to one. Which one of the following represents a deadlock-free order of invoking the  $P$  operations by the processes?

- A.  $X : P(a)P(b)P(c) \ Y : P(b)P(c)P(d) \ Z : P(c)P(d)P(a)$   
 B.  $X : P(b)P(a)P(c) \ Y : P(b)P(c)P(d) \ Z : P(a)P(c)P(d)$   
 C.  $X : P(b)P(a)P(c) \ Y : P(c)P(b)P(d) \ Z : P(a)P(c)P(d)$   
 D.  $X : P(a)P(b)P(c) \ Y : P(c)P(b)P(d) \ Z : P(c)P(d)P(a)$

gatecse-2013    operating-system    resource-allocation    normal

Answer key

## 5.25.19 Resource Allocation: GATE CSE 2014 | Set 1 | Question: 31



An operating system uses the *Banker's algorithm* for deadlock avoidance when managing the allocation of three resource types  $X$ ,  $Y$ , and  $Z$  to three processes  $P_0$ ,  $P_1$ , and  $P_2$ . The table given below presents the current system state. Here, the *Allocation matrix* shows the current number of resources of each type allocated to each process and the *Max matrix* shows the maximum number of resources of each type required by each process during its execution.

| Allocation           |     |     |     | Max |     |     |  |
|----------------------|-----|-----|-----|-----|-----|-----|--|
|                      | $X$ | $Y$ | $Z$ | $X$ | $Y$ | $Z$ |  |
| <b>P<sub>0</sub></b> | 0   | 0   | 1   | 8   | 4   | 3   |  |
| <b>P<sub>1</sub></b> | 3   | 2   | 0   | 6   | 2   | 0   |  |
| <b>P<sub>2</sub></b> | 2   | 1   | 1   | 3   | 3   | 3   |  |

There are 3 units of type  $X$ , 2 units of type  $Y$  and 2 units of type  $Z$  still available. The system is currently in a **safe** state. Consider the following independent requests for additional resources in the current state:

**REQ1:**  $P_0$  requests 0 units of  $X$ , 0 units of  $Y$  and 2 units of  $Z$

**REQ2:**  $P_1$  requests 2 units of  $X$ , 0 units of  $Y$  and 0 units of  $Z$

Which one of the following is **TRUE**?

- A. Only REQ1 can be permitted.      B. Only REQ2 can be permitted.  
 C. Both REQ1 and REQ2 can be      D. Neither REQ1 nor REQ2 can be

permitted.

permitted.

gatecse-2014-set1 operating-system resource-allocation normal

Answer key

#### 5.25.20 Resource Allocation: GATE CSE 2014 | Set 3 | Question: 31



A system contains three programs and each requires three tape units for its operation. The minimum number of tape units which the system must have such that deadlocks never arise is \_\_\_\_\_.

gatecse-2014-set3 operating-system resource-allocation numerical-answers easy

Answer key

#### 5.25.21 Resource Allocation: GATE CSE 2015 | Set 2 | Question: 23



A system has 6 identical resources and  $N$  processes competing for them. Each process can request at most 2 resources. Which one of the following values of  $N$  could lead to a deadlock?

- A. 1 B. 2 C. 3 D. 4

gatecse-2015-set2 operating-system resource-allocation easy

Answer key

#### 5.25.22 Resource Allocation: GATE CSE 2015 | Set 3 | Question: 52



Consider the following policies for preventing deadlock in a system with mutually exclusive resources.

- Process should acquire all their resources at the beginning of execution. If any resource is not available, all resources acquired so far are released.
- The resources are numbered uniquely, and processes are allowed to request for resources only in increasing resource numbers
- The resources are numbered uniquely, and processes are allowed to request for resources only in decreasing resource numbers
- The resources are numbered uniquely. A processes is allowed to request for resources only for a resource with resource number larger than its currently held resources

Which of the above policies can be used for preventing deadlock?

- A. Any one of (I) and (III) but not (II) or (IV) B. Any one of (I), (III) and (IV) but not (II)  
C. Any one of (II) and (III) but not (I) or (IV) D. Any one of (I), (II), (III) and (IV)

gatecse-2015-set3 operating-system resource-allocation normal

Answer key

#### 5.25.23 Resource Allocation: GATE CSE 2017 | Set 2 | Question: 33



A system shares 9 tape drives. The current allocation and maximum requirement of tape drives for that processes are shown below:

| Process | Current Allocation | Maximum Requirement |
|---------|--------------------|---------------------|
| P1      | 3                  | 7                   |
| P2      | 1                  | 6                   |
| P3      | 3                  | 5                   |

Which of the following best describes current state of the system?

- A. Safe, Deadlocked B. Safe, Not Deadlocked  
C. Not Safe, Deadlocked D. Not Safe, Not Deadlocked

gatecse-2017-set2 operating-system resource-allocation normal

Answer key

## 5.25.24 Resource Allocation: GATE CSE 2019 | Question: 39



Consider the following snapshot of a system running  $n$  concurrent processes. Process  $i$  is holding  $X_i$  instances of a resource  $R$ ,  $1 \leq i \leq n$ . Assume that all instances of  $R$  are currently in use. Further, for all  $i$ , process  $i$  can place a request for at most  $Y_i$  additional instances of  $R$  while holding the  $X_i$  instances it already has. Of the  $n$  processes, there are exactly two processes  $p$  and  $q$  such that  $Y_p = Y_q = 0$ . Which one of the following conditions guarantees that no other process apart from  $p$  and  $q$  can complete execution?

- A.  $X_p + X_q < \text{Min}\{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- B.  $X_p + X_q < \text{Max}\{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- C.  $\text{Min}(X_p, X_q) \geq \text{Min}\{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- D.  $\text{Min}(X_p, X_q) \leq \text{Max}\{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$

gatecse-2019 operating-system two-marks resource-allocation

Answer key

## 5.25.25 Resource Allocation: GATE CSE 2022 | Question: 16



Which of the following statements is/are TRUE with respect to deadlocks?

- A. Circular wait is a necessary condition for the formation of deadlock.
- B. In a system where each resource has more than one instance, a cycle in its wait-for graph indicates the presence of a deadlock.
- C. If the current allocation of resources to processes leads the system to unsafe state, then deadlock will necessarily occur.
- D. In the resource-allocation graph of a system, if every edge is an assignment edge, then the system is not in deadlock state.

gatecse-2022 operating-system resource-allocation multiple-selects one-mark

Answer key

## 5.25.26 Resource Allocation: GATE IT 2005 | Question: 62



Two shared resources  $R_1$  and  $R_2$  are used by processes  $P_1$  and  $P_2$ . Each process has a certain priority for accessing each resource. Let  $T_{ij}$  denote the priority of  $P_i$  for accessing  $R_j$ . A process  $P_i$  can snatch a resource  $R_k$  from process  $P_j$  if  $T_{ik}$  is greater than  $T_{jk}$ . Given the following :

- I.  $T_{11} > T_{21}$
- II.  $T_{12} > T_{22}$
- III.  $T_{11} < T_{21}$
- IV.  $T_{12} < T_{22}$

Which of the following conditions ensures that  $P_1$  and  $P_2$  can never deadlock?

- A. (I) and (IV)
- B. (II) and (III)
- C. (I) and (II)
- D. None of the above

gateit-2005 operating-system resource-allocation normal

Answer key

## 5.25.27 Resource Allocation: GATE IT 2008 | Question: 54



An operating system implements a policy that requires a process to release all resources before making a request for another resource. Select the TRUE statement from the following:

- A. Both starvation and deadlock can occur
- B. Starvation can occur but deadlock cannot occur
- C. Starvation cannot occur but deadlock can occur
- D. Neither starvation nor deadlock can occur

gateit-2008 operating-system resource-allocation normal

Answer key

5.26

### Resource Allocation Graph (1)

#### 5.26.1 Resource Allocation Graph: GATE CSE 2025 | Set 2 | Question: 38



$P = \{P_1, P_2, P_3, P_4\}$  consists of all active processes in an operating system.

$R = \{R_1, R_2, R_3, R_4\}$  consists of single instances of distinct types of resources in the system.

The resource allocation graph has the following assignment and claim edges.

Assignment edges:  $R_1 \rightarrow P_1, R_2 \rightarrow P_2, R_3 \rightarrow P_3, R_4 \rightarrow P_4$  (the assignment edge  $R_1 \rightarrow P_1$  means resource  $R_1$  is assigned to process  $P_1$ , and so on for others)

Claim edges:  $P_1 \rightarrow R_2, P_2 \rightarrow R_3, P_3 \rightarrow R_1, P_2 \rightarrow R_4, P_4 \rightarrow R_2$  (the claim edge  $P_1 \rightarrow R_2$  means process  $P_1$  is waiting for resource  $R_2$ , and so on for others)

Which of the following statement(s) is/are CORRECT?

- A. Aborting  $P_1$  makes the system deadlock free.
- B. Aborting  $P_3$  makes the system deadlock free.
- C. Aborting  $P_2$  makes the system deadlock free.
- D. Aborting  $P_1$  and  $P_4$  makes the system deadlock free.

gatecse2025-set2 operating-system resource-allocation-graph multiple-selects two-marks

Answer key

5.27

### Round Robin Scheduling (1)

#### 5.27.1 Round Robin Scheduling: GATE CSE 2022 | Question: 32



Consider four processes P, Q, R, and S scheduled on a CPU as per round robin algorithm with a time quantum of 4 units. The processes arrive in the order P, Q, R, S, all at time  $t = 0$ . There is exactly one context switch from S to Q, exactly one context switch from R to Q, and exactly two context switches from Q to R. There is no context switch from S to P. Switching to a ready process after the termination of another process is also considered a context switch. Which one of the following is NOT possible as CPU burst time (in time units) of these processes?

- A.  $P = 4, Q = 10, R = 6, S = 2$
- B.  $P = 2, Q = 9, R = 5, S = 1$
- C.  $P = 4, Q = 12, R = 5, S = 4$
- D.  $P = 3, Q = 7, R = 7, S = 3$

gatecse-2022 operating-system process-scheduling round-robin-scheduling two-marks

Answer key

5.28

### Semaphore (11)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(3Q\)](#)

#### 5.28.1 Semaphore: GATE CSE 1990 | Question: 1-vii



Semaphore operations are atomic because they are implemented within the OS \_\_\_\_\_.

gate1990 operating-system semaphore process-synchronization fill-in-the-blanks

Answer key

#### 5.28.2 Semaphore: GATE CSE 1992 | Question: 02,x, ISRO2015-35



At a particular time of computation, the value of a counting semaphore is 7. Then 20 P operations and 15 V operations were completed on this semaphore. The resulting value of the semaphore is :

- A. 42
- B. 2
- C. 7
- D. 12

gate1992 operating-system semaphore easy isro2015 process-synchronization

Answer key



### 5.28.3 Semaphore: GATE CSE 1998 | Question: 1.31



A counting semaphore was initialized to 10. Then  $6P$  (wait) operations and  $4V$  (signal) operations were completed on this semaphore. The resulting value of the semaphore is

- A. 0                                      B. 8                                      C. 10                                      D. 12

gate1998    operating-system    process-synchronization    semaphore    easy

Answer key

### 5.28.4 Semaphore: GATE CSE 2008 | Question: 63



The  $P$  and  $V$  operations on counting semaphores, where  $s$  is a counting semaphore, are defined as follows:

$P(s)$  :  $s = s - 1$ ;  
          If  $s < 0$  then wait;

$V(s)$  :  $s = s + 1$ ;  
          If  $s \leq 0$  then wake up process waiting on  $s$ ;

Assume that  $P_b$  and  $V_b$  the wait and signal operations on binary semaphores are provided. Two binary semaphores  $x_b$  and  $y_b$  are used to implement the semaphore operations  $P(s)$  and  $V(s)$  as follows:

$P(s)$  :                       $P_b(x_b)$ ;  
                               $s = s - 1$ ;  
                              if ( $s < 0$ )  
                              {  
                               $V_b(x_b)$ ;  
                               $P_b(y_b)$ ;  
                              }  
                              else  $V_b(x_b)$ ;

$V(s)$  :                       $P_b(x_b)$ ;  
                               $s = s + 1$ ;  
                              if ( $s \leq 0$ )  $V_b(y_b)$ ;  
                               $V_b(x_b)$ ;

The initial values of  $x_b$  and  $y_b$  are respectively

- A. 0 and 0                                      B. 0 and 1                                      C. 1 and 0                                      D. 1 and 1

gatecse-2008    operating-system    normal    semaphore

Answer key

### 5.28.5 Semaphore: GATE CSE 2016 | Set 2 | Question: 49



Consider a non-negative counting semaphore  $S$ . The operation  $P(S)$  decrements  $S$ , and  $V(S)$  increments  $S$ . During an execution, 20  $P(S)$  operations and 12  $V(S)$  operations are issued in some order. The largest initial value of  $S$  for which at least one  $P(S)$  operation will remain blocked is \_\_\_\_\_

gatecse-2016-set2    operating-system    semaphore    normal    numerical-answers

Answer key

### 5.28.6 Semaphore: GATE CSE 2020 | Question: 34



Each of a set of  $n$  processes executes the following code using two semaphores  $a$  and  $b$  initialized to 1 and 0, respectively. Assume that `count` is a shared variable initialized to 0 and not used in CODE SECTION P.

#### CODE SECTION P

```
wait(a); count=count+1;  
if (count==n) signal (b);  
signal (a); wait (b) ; signal (b);
```

#### CODE SECTION Q

What does the code achieve?

- A. It ensures that no process executes CODE SECTION Q before every process has finished CODE SECTION P.
- B. It ensures that at most two processes are in CODE SECTION Q at any time.
- C. It ensures that all processes execute CODE SECTION P mutually exclusively.
- D. It ensures that at most  $n - 1$  processes are in CODE SECTION P at any time.

gatecse-2020 operating-system semaphore two-marks

Answer key

#### 5.28.7 Semaphore: GATE CSE 2021 | Set 1 | Question: 46



Consider the following pseudocode, where  $S$  is a semaphore initialized to 5 in line #2 and **counter** is a shared variable initialized to 0 in line #1. Assume that the increment operation in line #7 is *not* atomic.

```

1. int counter = 0;
2. Semaphore S = init(5);
3. void parop(void)
4. {
5.     wait(S);
6.     wait(S);
7.     counter++;
8.     signal(S);
9.     signal(S);
10. }

```

If five threads execute the function **parop** concurrently, which of the following program behavior(s) is/are possible?

- A. The value of **counter** is 5 after all the threads successfully complete the execution of **parop**
- B. The value of **counter** is 1 after all the threads successfully complete the execution of **parop**
- C. The value of **counter** is 0 after all the threads successfully complete the execution of **parop**
- D. There is a deadlock involving all the threads

gatecse-2021-set1 multiple-selects operating-system process-synchronization semaphore two-marks

Answer key

#### 5.28.8 Semaphore: GATE CSE 2022 | Question: 9



Consider the following threads,  $T_1$ ,  $T_2$ , and  $T_3$  executing on a single processor, synchronized using three binary semaphore variables,  $S_1$ ,  $S_2$ , and  $S_3$ , operated upon using standard **wait()** and **signal()**. The threads can be context switched in any order and at any time.

| $T_1$                                                               | $T_2$                                                               | $T_3$                                                               |
|---------------------------------------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------|
| while(true){<br>wait( $S_3$ );<br>print("C");<br>signal( $S_2$ ); } | while(true){<br>wait( $S_1$ );<br>print("B");<br>signal( $S_3$ ); } | while(true){<br>wait( $S_2$ );<br>print("A");<br>signal( $S_1$ ); } |

Which initialization of the semaphores would print the sequence BCABCABCA...

- A.  $S_1 = 1; S_2 = 1; S_3 = 1$
- B.  $S_1 = 1; S_2 = 1; S_3 = 0$
- C.  $S_1 = 1; S_2 = 0; S_3 = 0$
- D.  $S_1 = 0; S_2 = 1; S_3 = 1$

gatecse-2022 operating-system process-synchronization semaphore one-mark

Answer key

#### 5.28.9 Semaphore: GATE CSE 2023 | Question: 28



Consider the two functions **incr** and **decr** shown below.

```

incr(){      decr(){
    wait(s);  wait(s);
    X = X+1;  X = X-1;
    signal(s); signal(s);
}            }

```

There are 5 threads each invoking **incr** once, and 3 threads each invoking **decr** once, on the same shared variable **X**. The initial value of **X** is 10.

Suppose there are two implementations of the semaphore **s**, as follows:

**I-1:** **s** is a binary semaphore initialized to 1.

**I-2:** **s** is a counting semaphore initialized to 2.

Let **V1**, **V2** be the values of **X** at the end of execution of all the threads with implementations **I-1**, **I-2**, respectively.

Which one of the following choices corresponds to the minimum possible values of **V1**, **V2**, respectively?

- A. 15,7                      B. 7,7                      C. 12,7                      D. 12,8

gatecse-2023    operating-system    semaphore    two-marks

Answer key

#### 5.28.10 Semaphore: GATE CSE 2026 | Set 2 | Question: 41



Consider three processes **P1**, **P2**, and **P3** running identical code, as shown in the pseudocode below. **A** and **B** are two binary semaphores initialized to 1 and 0, respectively. **X** is a shared variable initialized to 0. Each line in the pseudocode is executed atomically.

Pseudocode of **P1**, **P2**, and **P3**

```
Wait(A);
Print(*);
X = X+1;
If (X == 2)
{
    Print($);
    Signal(B);
}
Signal(A);
Wait(B);
Print(#);
Signal(B);
```

Assume that any of the three processes can start to execute first and context switching can happen between these processes at any arbitrary time and in any arbitrary order.

Which of the following patterns is/are possible to be generated as an outcome of the execution of these three processes?

- | A.       | B.       | C.       | D.       |
|----------|----------|----------|----------|
| **\$#### | **\$*### | **\$###* | ***\$### |

gatecse-2026-set2    operating-system    semaphore    multiple-selects    two-marks

Answer key

#### 5.28.11 Semaphore: GATE IT 2006 | Question: 57



The wait and signal operations of a monitor are implemented using semaphores as follows. In the following,

- $x$  is a condition variable,
- **mutex** is a semaphore initialized to 1,
- $x\_sem$  is a semaphore initialized to 0,
- $x\_count$  is the number of processes waiting on semaphore  $x\_sem$ , initially 0,
- **next** is a semaphore initialized to 0,
- $next\_count$  is the number of processes waiting on semaphore **next**, initially 0.

The body of each procedure that is visible outside the monitor is replaced with the following:

```
P(mutex);
...
body of procedure
...
if (next_count > 0)
```

```
V(next);
else
V(mutex);
```

Each occurrence of  $x.wait$  is replaced with the following:

```
x_count = x_count + 1;
if (next_count > 0)
V(next);
else
V(mutex);
----- E1;
x_count = x_count - 1;
```

Each occurrence of  $x.signal$  is replaced with the following:

```
if (x_count > 0)
{
next_count = next_count + 1;
----- E2;
P(next);
next_count = next_count - 1;
}
```

For correct implementation of the monitor, statements  $E1$  and  $E2$  are, respectively,

- A.  $P(x\_sem), V(next)$                       B.  $V(next), P(x\_sem)$   
C.  $P(next), V(x\_sem)$                       D.  $P(x\_sem), V(x\_sem)$

gateit-2006   operating-system   process-synchronization   semaphore   normal

Answer key

## 5.29 Srtf (1)

### 5.29.1 Srtf: GATE CSE 2025 | Set 2 | Question: 16



Processes  $P1, P2, P3, P4$  arrive in that order at times 0, 1, 2, and 8 milliseconds respectively, and have execution times of 10, 13, 6, and 9 milliseconds respectively. Shortest Remaining Time First (SRTF) algorithm is used as the CPU scheduling policy. Ignore context switching times.

Which ONE of the following correctly gives the average turnaround time of the four processes in milliseconds?

- A. 22                      B. 15                      C. 37                      D. 19

gatecse2025-set2   operating-system   srtf   process-scheduling   average-turnaround-time   one-mark

Answer key

## 5.30 System Calls (1)

### 5.30.1 System Calls: GATE CSE 2021 | Set 1 | Question: 14



Which of the following standard  $C$  library functions will *always* invoke a system call when executed from a single-threaded process in a UNIX/Linux operating system?

- A. `exit`                      B. `malloc`                      C. `sleep`                      D. `strlen`

gatecse-2021-set1   multiple-selects   operating-system   system-calls   one-mark

Answer key

## 5.31 Threads (10)

**Practice Tests:** [Test 1 \(15Q\)](#) [Test 2 \(7Q\)](#)

### 5.31.1 Threads: GATE CSE 2004 | Question: 11



Consider the following statements with respect to user-level threads and kernel-supported threads

- I. context switch is faster with kernel-supported threads
- II. for user-level threads, a system call can block the entire process
- III. Kernel supported threads can be scheduled independently
- IV. User level threads are transparent to the kernel

Which of the above statements are true?

- A. (II), (III) and (IV) only
- B. (II) and (III) only
- C. (I) and (III) only
- D. (I) and (II) only

gatecse-2004 operating-system threads normal

Answer key

### 5.31.2 Threads: GATE CSE 2007 | Question: 17



Consider the following statements about user level threads and kernel level threads. Which one of the following statements is FALSE?

- A. Context switch time is longer for kernel level threads than for user level threads.
- B. User level threads do not need any hardware support.
- C. Related kernel level threads can be scheduled on different processors in a multi-processor system.
- D. Blocking one kernel level thread blocks all related threads.

gatecse-2007 operating-system threads normal

Answer key

### 5.31.3 Threads: GATE CSE 2011 | Question: 16, UGCNET-June2013-III: 65



A thread is usually defined as a "light weight process" because an Operating System (OS) maintains smaller data structure for a thread than for a process. In relation to this, which of the following statement is TRUE?

- A. On per- thread basis , the OS maintains only CPU register state .
- B. The OS does not maintain a separate stack for each thread .
- C. On per- thread basis , the OS does not maintain virtual memory state .
- D. On per- thread basis , the OS maintains only scheduling and accounting information .

gatecse-2011 operating-system threads normal ugcnetcse-june2013-paper3

Answer key

### 5.31.4 Threads: GATE CSE 2014 | Set 1 | Question: 20



Which one of the following is **FALSE**?

- A. User level threads are not scheduled by the kernel.
- B. When a user level thread is blocked, all other threads of its process are blocked.
- C. Context switching between user level threads is faster than context switching between kernel level threads.
- D. Kernel level threads cannot share the code segment.

gatecse-2014-set1 operating-system threads normal

Answer key

### 5.31.5 Threads: GATE CSE 2017 | Set 1 | Question: 18



Threads of a process share

- A. global variables but not heap
- B. heap but not global variables
- C. neither global variables nor heap
- D. both heap and global variables

## Answer key

## 5.31.6 Threads: GATE CSE 2017 | Set 2 | Question: 07



Which of the following is/are shared by all the threads in a process?

- I. Program counter
- II. Stack
- III. Address space
- IV. Registers

- A. (I) and (II) only      B. (III) only      C. (IV) only      D. (III) and (IV) only

## Answer key

## 5.31.7 Threads: GATE CSE 2021 | Set 2 | Question: 42



Consider the following multi-threaded code segment (in a mix of C and pseudo-code), invoked by two processes  $P_1$  and  $P_2$ , and each of the processes spawns two threads  $T_1$  and  $T_2$ :

```
int x = 0; // global
Lock L1; // global
main () {
    create a thread to execute foo(); // Thread T1
    create a thread to execute foo(); // Thread T2
    wait for the two threads to finish execution;
    print(x);}

foo() {
    int y = 0;
    Acquire L1;
    x = x + 1;
    y = y + 1;
    Release L1;
    print (y);}
```

Which of the following statement(s) is/are correct?

- A. Both  $P_1$  and  $P_2$  will print the value of  $x$  as 2.
- B. At least of  $P_1$  and  $P_2$  will print the value of  $x$  as 4.
- C. At least one of the threads will print the value of  $y$  as 2.
- D. Both  $T_1$  and  $T_2$ , in both the processes, will print the value of  $y$  as 1.

## Answer key

## 5.31.8 Threads: GATE CSE 2024 | Set 1 | Question: 14



Which of the following statements about threads is/are TRUE?

- A. Threads can only be implemented in kernel space
- B. Each thread has its own file descriptor table for open files
- C. All the threads belonging to a process share a common stack
- D. Threads belonging to a process are by default not protected from each other

## Answer key

## 5.31.9 Threads: GATE CSE 2024 | Set 1 | Question: 30



Consider the following two threads  $T_1$  and  $T_2$  that update two shared variables  $a$  and  $b$ . Assume that initially  $a = b = 1$ . Though context switching between threads can happen at any time, each statement of  $T_1$  or  $T_2$  is executed atomically without interruption.

T1                      T2  
 $a = a + 1;$        $b = 2 * b;$   
 $b = b + 1;$        $a = 2 * a;$

Which one of the following options lists all the possible combinations of values of a and b after both T1 and T2 finish execution?

- A.  $(a = 4, b = 4); (a = 3, b = 3); (a = 4, b = 3)$
- B.  $(a = 3, b = 4); (a = 4, b = 3); (a = 3, b = 3)$
- C.  $(a = 4, b = 4); (a = 4, b = 3); (a = 3, b = 4)$
- D.  $(a = 2, b = 2); (a = 2, b = 3); (a = 3, b = 4)$

gatecse-2024-set1   operating-system   threads   two-marks

Answer key

#### 5.31.10 Threads: GATE IT 2004 | Question: 14



Which one of the following is NOT shared by the threads of the same process ?

- A. Stack
- B. Address Space
- C. File Descriptor Table
- D. Message Queue

gateit-2004   operating-system   easy   threads

Answer key

#### 5.32

#### Translation Lookaside Buffer (2)

##### 5.32.1 Translation Lookaside Buffer: GATE CSE 2022 | Question: 28



Which one of the following statements is FALSE?

- A. The TLB performs an associative search in parallel on all its valid entries using page number of incoming virtual address.
- B. If the virtual address of a word given by CPU has a TLB hit, but the subsequent search for the word results in a cache miss, then the word will always be present in the main memory.
- C. The memory access time using a given inverted page table is always same for all incoming virtual addresses.
- D. In a system that uses hashed page tables, if two distinct virtual addresses V1 and V2 map to the same value while hashing, then the memory access time of these addresses will not be the same.

gatecse-2022   operating-system   memory-management   translation-lookaside-buffer   two-marks

Answer key

##### 5.32.2 Translation Lookaside Buffer: GATE CSE 2026 | Set 2 | Question: 44



A system has a Translation Lookaside Buffer (TLB) that has a reach of 1 MB. TLB reach is defined as the total amount of physical memory that can be accessed through the TLB entries. The paging system uses pages of size 4 KB. The virtual address space is 64 GB and physical address space is 1 GB. If each TLB entry stores a 4-bit process id, page number, frame number, and a 2-bit control field, then the size of the TLB (in bytes) is \_\_\_\_\_. (answer in integer)

Note:  $1K = 2^{10}$ ,  $1M = 2^{20}$ ,  $1G = 2^{30}$

gatecse-2026-set2   operating-system   translation-lookaside-buffer   paging   numerical-answers   two-marks

Answer key

#### 5.33

#### Virtual Memory (43)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(15Q\)](#) [Test 3 \(15Q\)](#) [Test 4 \(6Q\)](#)

##### 5.33.1 Virtual Memory: GATE CSE 1989 | Question: 2-iv



Match the pairs in the following:

|                       |                         |
|-----------------------|-------------------------|
| (A) Virtual memory    | (p) Temporal Locality   |
| (B) Shared memory     | (q) Spatial Locality    |
| (C) Look-ahead buffer | (r) Address Translation |
| (D) Look-aside buffer | (s) Mutual Exclusion    |

match-the-following gate1989 operating-system virtual-memory

Answer key

### 5.33.2 Virtual Memory: GATE CSE 1990 | Question: 1-v

Under paged memory management scheme, simple lock and key memory protection arrangement may still be required if the \_\_\_\_\_ processors do not have address mapping hardware.

gate1990 operating-system virtual-memory fill-in-the-blanks

Answer key

### 5.33.3 Virtual Memory: GATE CSE 1990 | Question: 7-b

In a two-level virtual memory, the memory access time for main memory,  $t_M = 10^{-8}$  sec, and the memory access time for the secondary memory,  $t_D = 10^{-3}$  sec. What must be the hit ratio,  $H$  such that the access efficiency is within 80 percent of its maximum value?

gate1990 descriptive operating-system virtual-memory

Answer key

### 5.33.4 Virtual Memory: GATE CSE 1991 | Question: 03-xi

Indicate all the false statements from the statements given below:

- A. The amount of virtual memory available is limited by the availability of the secondary memory
- B. Any implementation of a critical section requires the use of an indivisible machine- instruction ,such as test-and-set.
- C. The use of monitors ensure that no dead-locks will be caused .
- D. The LRU page-replacement policy may cause thrashing for some type of programs.
- E. The best fit techniques for memory allocation ensures that memory will never be fragmented.

gate1991 operating-system virtual-memory normal multiple-selects

Answer key

### 5.33.5 Virtual Memory: GATE CSE 1994 | Question: 1.21

Which one of the following statements is true?

- A. Macro definitions cannot appear within other macro definitions in assembly language programs
- B. Overlaying is used to run a program which is longer than the address space of a computer
- C. Virtual memory can be used to accommodate a program which is longer than the address space of a computer
- D. It is not possible to write interrupt service routines in a high level language

gate1994 operating-system normal virtual-memory

Answer key

### 5.33.6 Virtual Memory: GATE CSE 1995 | Question: 1.7

In a paged segmented scheme of memory management, the segment table itself must have a page table because

- A. The segment table is often too large to fit in one page
- B. Each segment is spread over a number of pages



- C. Segment tables point to page tables and not to the physical locations of the segment  
D. The processor's description base register points to a page table

gate1995 operating-system virtual-memory normal

Answer key

### 5.33.7 Virtual Memory: GATE CSE 1995 | Question: 2.16

In a virtual memory system the address space specified by the address lines of the CPU must be \_\_\_\_\_ than the physical memory size and \_\_\_\_\_ than the secondary storage size.

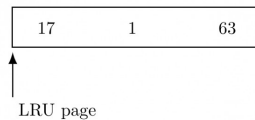
- A. smaller, smaller  
B. smaller, larger  
C. larger, smaller  
D. larger, larger

gate1995 operating-system virtual-memory normal

Answer key

### 5.33.8 Virtual Memory: GATE CSE 1996 | Question: 7

A demand paged virtual memory system uses 16 bit virtual address, page size of 256 bytes, and has 1 Kbyte of main memory. LRU page replacement is implemented using the list, whose current status (page number is decimal) is



For each hexadecimal address in the address sequence given below,  
00FF, 010D, 10FF, 11B0  
indicate

- i. the new status of the list
- ii. page faults, if any, and
- iii. page replacements, if any.

gate1996 operating-system virtual-memory normal descriptive

Answer key

### 5.33.9 Virtual Memory: GATE CSE 1998 | Question: 2.18, UGCNET-June2012-III: 48

If an instruction takes  $i$  microseconds and a page fault takes an additional  $j$  microseconds, the effective instruction time if on the average a page fault occurs every  $k$  instruction is:

- A.  $i + \frac{j}{k}$   
B.  $i + (j \times k)$   
C.  $\frac{i+j}{k}$   
D.  $(i+j) \times k$

gate1998 operating-system virtual-memory easy ugcnetcse-june2012-paper3

Answer key

### 5.33.10 Virtual Memory: GATE CSE 1999 | Question: 19

A certain computer system has the segmented paging architecture for virtual memory. The memory is byte addressable. Both virtual and physical address spaces contain  $2^{16}$  bytes each. The virtual address space is divided into 8 non-overlapping equal size segments. The memory management unit (MMU) has a hardware segment table, each entry of which contains the physical address of the page table for the segment. Page tables are stored in the main memory and consists of 2 byte page table entries.

- a. What is the minimum page size in bytes so that the page table for a segment requires at most one page to store it? Assume that the page size can only be a power of 2.
- b. Now suppose that the pages size is 512 bytes. It is proposed to provide a TLB (Transaction look-aside buffer) for speeding up address translation. The proposed TLB will be capable of storing page table entries for 16

recently referenced virtual pages, in a fast cache that will use the direct mapping scheme. What is the number of tag bits that will need to be associated with each cache entry?

- c. Assume that each page table entry contains (besides other information) 1 valid bit, 3 bits for page protection and 1 dirty bit. How many bits are available in page table entry for storing the aging information for the page? Assume that the page size is 512 bytes.

gate1999 operating-system virtual-memory normal descriptive

Answer key

#### 5.33.11 Virtual Memory: GATE CSE 1999 | Question: 2.10



A multi-user, multi-processing operating system cannot be implemented on hardware that does not support

- A. Address translation
- B. DMA for disk transfer
- C. At least two modes of CPU execution (privileged and non-privileged)
- D. Demand paging

gate1999 operating-system normal virtual-memory multiple-selects

Answer key

#### 5.33.12 Virtual Memory: GATE CSE 1999 | Question: 2.11



Which of the following is/are advantage(s) of virtual memory?

- A. Faster access to memory on an average.
- B. Processes can be given protected address spaces.
- C. Linker can assign addresses independent of where the program will be loaded in physical memory.
- D. Program larger than the physical memory size can be run.

gate1999 operating-system virtual-memory easy multiple-selects

Answer key

#### 5.33.13 Virtual Memory: GATE CSE 2000 | Question: 2.22



Suppose the time to service a page fault is on the average 10 milliseconds, while a memory access takes 1 microsecond. Then a 99.99% hit ratio results in average memory access time of

- A. 1.9999 milliseconds
- B. 1 millisecond
- C. 9.999 microseconds
- D. 1.9999 microseconds

gatecse-2000 operating-system easy virtual-memory

Answer key

#### 5.33.14 Virtual Memory: GATE CSE 2001 | Question: 1.20



Where does the swap space reside?

- A. RAM
- B. Disk
- C. ROM
- D. On-chip cache

gatecse-2001 operating-system easy virtual-memory

Answer key

#### 5.33.15 Virtual Memory: GATE CSE 2001 | Question: 1.8



Which of the following statements is false?

- A. Virtual memory implements the translation of a program's address space into physical memory address space
- B. Virtual memory allows each program to exceed the size of the primary memory
- C. Virtual memory increases the degree of multiprogramming
- D. Virtual memory reduces the context switching overhead

Answer key

## 5.33.16 Virtual Memory: GATE CSE 2001 | Question: 2.21



Consider a machine with 64 MB physical memory and a 32-bit virtual address space. If the page size is 4 KB, what is the approximate size of the page table?

- A. 16 MB                      B. 8 MB                      C. 2 MB                      D. 24 MB

Answer key

## 5.33.17 Virtual Memory: GATE CSE 2002 | Question: 19



A computer uses 32-bit virtual address, and 32-bit physical address. The physical memory is byte addressable, and the page size is 4 Kbytes. It is decided to use two level page tables to translate from virtual address to physical address. Equal number of bits should be used for indexing first level and second level page table, and the size of each table entry is 4 bytes.

- A. Give a diagram showing how a virtual address would be translated to a physical address.  
 B. What is the number of page table entries that can be contained in each page?  
 C. How many bits are available for storing protection and other information in each page table entry?

Answer key

## 5.33.18 Virtual Memory: GATE CSE 2003 | Question: 26



In a system with 32 bit virtual addresses and 1 KB page size, use of one-level page tables for virtual to physical address translation is not practical because of

- A. the large amount of internal fragmentation  
 B. the large amount of external fragmentation  
 C. the large memory overhead in maintaining page tables  
 D. the large computation overhead in the translation process

Answer key

## 5.33.19 Virtual Memory: GATE CSE 2003 | Question: 78



A processor uses 2-level page tables for virtual to physical address translation. Page tables for both levels are stored in the main memory. Virtual and physical addresses are both 32 bits wide. The memory is byte addressable. For virtual to physical address translation, the 10 most significant bits of the virtual address are used as index into the first level page table while the next 10 bits are used as index into the second level page table. The 12 least significant bits of the virtual address are used as offset within the page. Assume that the page table entries in both levels of page tables are 4 bytes wide. Further, the processor has a translation look-aside buffer (TLB), with a hit rate of 96%. The TLB caches recently used virtual page numbers and the corresponding physical page numbers. The processor also has a physically addressed cache with a hit rate of 90%. Main memory access time is 10 ns, cache access time is 1 ns, and TLB access time is also 1 ns.

Assuming that no page faults occur, the average time taken to access a virtual address is approximately (to the nearest 0.5 ns)

- A. 1.5 ns                      B. 2 ns                      C. 3 ns                      D. 4 ns

Answer key

### 5.33.20 Virtual Memory: GATE CSE 2003 | Question: 79



A processor uses 2-level page tables for virtual to physical address translation. Page tables for both levels are stored in the main memory. Virtual and physical addresses are both 32 bits wide. The memory is byte addressable. For virtual to physical address translation, the 10 most significant bits of the virtual address are used as index into the first level page table while the next 10 bits are used as index into the second level page table. The 12 least significant bits of the virtual address are used as offset within the page. Assume that the page table entries in both levels of page tables are 4 bytes wide. Further, the processor has a translation look-aside buffer (TLB), with a hit rate of 96%. The TLB caches recently used virtual page numbers and the corresponding physical page numbers. The processor also has a physically addressed cache with a hit rate of 90%. Main memory access time is 10 ns, cache access time is 1 ns, and TLB access time is also 1 ns.

Suppose a process has only the following pages in its virtual address space: two contiguous code pages starting at virtual address  $0x00000000$ , two contiguous data pages starting at virtual address  $0x00400000$ , and a stack page starting at virtual address  $0xFFFF000$ . The amount of memory required for storing the page tables of this process is

- A. 8 KB                      B. 12 KB                      C. 16 KB                      D. 20 KB

gatecse-2003   operating-system   normal   virtual-memory

Answer key

### 5.33.21 Virtual Memory: GATE CSE 2006 | Question: 62, ISRO2016-50



A CPU generates 32-bit virtual addresses. The page size is 4 KB. The processor has a translation look-aside buffer (TLB) which can hold a total of 128 page table entries and is 4-way set associative. The minimum size of the TLB tag is:

- A. 11 bits                      B. 13 bits                      C. 15 bits                      D. 20 bits

gatecse-2006   operating-system   virtual-memory   normal   isro2016

Answer key

### 5.33.22 Virtual Memory: GATE CSE 2006 | Question: 63, UGCNET-June2012-III: 45



A computer system supports 32-bit virtual addresses as well as 32-bit physical addresses. Since the virtual address space is of the same size as the physical address space, the operating system designers decide to get rid of the virtual memory entirely. Which one of the following is true?

- A. Efficient implementation of multi-user support is no longer possible  
B. The processor cache organization can be made more efficient now  
C. Hardware support for memory management is no longer needed  
D. CPU scheduling can be made more efficient now

gatecse-2006   operating-system   virtual-memory   normal   ugcnetcse-june2012-paper3

Answer key

### 5.33.23 Virtual Memory: GATE CSE 2008 | Question: 67



A processor uses 36 bit physical address and 32 bit virtual addresses, with a page frame size of 4 Kbytes. Each page table entry is of size 4 bytes. A three level page table is used for virtual to physical address translation, where the virtual address is used as follows:

- Bits 30 – 31 are used to index into the first level page table.
- Bits 21 – 29 are used to index into the 2nd level page table.
- Bits 12 – 20 are used to index into the 3rd level page table.
- Bits 0 – 11 are used as offset within the page.

The number of bits required for addressing the next level page table(or page frame) in the page table entry of the first, second and third level page tables are respectively

- A. 20,20,20                      B. 24,24,24                      C. 24,24,20                      D. 25,25,24

gatecse-2008   operating-system   virtual-memory   normal

Answer key

#### 5.33.24 Virtual Memory: GATE CSE 2009 | Question: 10



The essential content(s) in each entry of a page table is / are

- A. Virtual page number
- B. Page frame number
- C. Both virtual page number and page frame number
- D. Access right information

gatecse-2009 operating-system virtual-memory easy

Answer key

#### 5.33.25 Virtual Memory: GATE CSE 2009 | Question: 34



A multilevel page table is preferred in comparison to a single level page table for translating virtual address to physical address because

- A. It reduces the memory access time to read or write a memory location.
- B. It helps to reduce the size of page table needed to implement the virtual address space of a process
- C. It is required by the translation lookaside buffer.
- D. It helps to reduce the number of page faults in page replacement algorithms.

gatecse-2009 operating-system virtual-memory easy

Answer key

#### 5.33.26 Virtual Memory: GATE CSE 2011 | Question: 20, UGCNET-June2013-II: 48



Let the page fault service time be 10 milliseconds(ms) in a computer with average memory access time being 20 nanoseconds (ns). If one page fault is generated every  $10^6$  memory accesses, what is the effective access time for memory?

- A. 21 ns
- B. 30 ns
- C. 23 ns
- D. 35 ns

gatecse-2011 operating-system virtual-memory normal ugcnetcse-june2013-paper2

Answer key

#### 5.33.27 Virtual Memory: GATE CSE 2013 | Question: 52



A computer uses 46-bit virtual address, 32-bit physical address, and a three-level paged page table organization. The page table base register stores the base address of the first-level table (T1), which occupies exactly one page. Each entry of T1 stores the base address of a page of the second-level table (T2). Each entry of T2 stores the base address of a page of the third-level table (T3). Each entry of T3 stores a page table entry (PTE). The PTE is 32 bits in size. The processor used in the computer has a 1 MB 16 way set associative virtually indexed physically tagged cache. The cache block size is 64 bytes.

What is the size of a page in KB in this computer?

- A. 2
- B. 4
- C. 8
- D. 16

gatecse-2013 operating-system virtual-memory normal

Answer key

#### 5.33.28 Virtual Memory: GATE CSE 2013 | Question: 53



A computer uses 46-bit virtual address, 32-bit physical address, and a three-level paged page table organization. The page table base register stores the base address of the first-level table (T1), which occupies exactly one page. Each entry of T1 stores the base address of a page of the second-level table (T2). Each entry of T2 stores the base address of a page of the third-level table (T3). Each entry of T3 stores a page table entry (PTE). The PTE is 32 bits in size. The processor used in the computer has a 1 MB 16 way set associative virtually indexed physically tagged cache. The cache block size is 64 bytes.

What is the minimum number of page colours needed to guarantee that no two synonyms map to different sets in

the processor cache of this computer?

A. 2

B. 4

C. 8

D. 16

gatecse-2013 normal operating-system virtual-memory

Answer key

#### 5.33.29 Virtual Memory: GATE CSE 2014 | Set 3 | Question: 33



Consider a paging hardware with a *TLB*. Assume that the entire page table and all the pages are in the physical memory. It takes 10 milliseconds to search the *TLB* and 80 milliseconds to access the physical memory. If the *TLB* hit ratio is 0.6, the effective memory access time (in milliseconds) is \_\_\_\_\_.

gatecse-2014-set3 operating-system virtual-memory numerical-answers normal

Answer key

#### 5.33.30 Virtual Memory: GATE CSE 2015 | Set 1 | Question: 12



Consider a system with byte-addressable memory, 32-bit logical addresses, 4 kilobyte page size and page table entries of 4 bytes each. The size of the page table in the system in megabytes is \_\_\_\_\_.

gatecse-2015-set1 operating-system virtual-memory easy numerical-answers

Answer key

#### 5.33.31 Virtual Memory: GATE CSE 2015 | Set 2 | Question: 25



A computer system implements a 40-bit virtual address, page size of 8 kilobytes, and a 128-entry translation look-aside buffer (TLB) organized into 32 sets each having 4 ways. Assume that the TLB tag does not store any process id. The minimum length of the TLB tag in bits is \_\_\_\_\_.

gatecse-2015-set2 operating-system virtual-memory easy numerical-answers

Answer key

#### 5.33.32 Virtual Memory: GATE CSE 2015 | Set 2 | Question: 47



A computer system implements 8 kilobyte pages and a 32-bit physical address space. Each page table entry contains a valid bit, a dirty bit, three permission bits, and the translation. If the maximum size of the page table of a process is 24 megabytes, the length of the virtual address supported by the system is \_\_\_\_\_ bits.

gatecse-2015-set2 operating-system virtual-memory normal numerical-answers

Answer key

#### 5.33.33 Virtual Memory: GATE CSE 2016 | Set 1 | Question: 47



Consider a computer system with 40-bit virtual addressing and page size of sixteen kilobytes. If the computer system has a one-level page table per process and each page table entry requires 48 bits, then the size of the per-process page table is \_\_\_\_\_ megabytes.

gatecse-2016-set1 operating-system virtual-memory easy numerical-answers

Answer key

#### 5.33.34 Virtual Memory: GATE CSE 2018 | Question: 10



Consider a process executing on an operating system that uses demand paging. The average time for a memory access in the system is  $M$  units if the corresponding memory page is available in memory, and  $D$  units if the memory access causes a page fault. It has been experimentally measured that the average time taken for a memory access in the process is  $X$  units.

Which one of the following is the correct expression for the page fault rate experienced by the process.

- A.  $(D - M)/(X - M)$     B.  $(X - M)/(D - M)$     C.  $(D - X)/(D - M)$     D.  $(X - M)/(D - X)$

gatecse-2018 operating-system virtual-memory normal one-mark

Answer key

5.33.35 Virtual Memory: GATE CSE 2019 | Question: 33



Assume that in a certain computer, the virtual addresses are 64 bits long and the physical addresses are 48 bits long. The memory is word addressable. The page size is 8 kB and the word size is 4 bytes. The Translation Look-aside Buffer (TLB) in the address translation path has 128 valid entries. At most how many distinct virtual addresses can be translated without any TLB miss?

- A.  $16 \times 2^{10}$  B.  $256 \times 2^{10}$   
C.  $4 \times 2^{20}$  D.  $8 \times 2^{20}$

gatecse-2019 operating-system virtual-memory two-marks

Answer key

5.33.36 Virtual Memory: GATE CSE 2020 | Question: 53



Consider a paging system that uses 1-level page table residing in main memory and a TLB for address translation. Each main memory access takes 100 ns and TLB lookup takes 20 ns. Each page transfer to/from the disk takes 5000 ns. Assume that the TLB hit ratio is 95%, page fault rate is 10%. Assume that for 20% of the total page faults, a dirty page has to be written back to disk before the required page is read from disk. TLB update time is negligible. The average memory access time in ns (round off to 1 decimal places) is \_\_\_\_\_.

gatecse-2020 numerical-answers operating-system virtual-memory two-marks

Answer key

5.33.37 Virtual Memory: GATE CSE 2023 | Question: 48



Consider a computer system with 57-bit virtual addressing using multi-level tree-structured page tables with L levels for virtual to physical address translation. The page size is 4 KB (1 KB = 1024 B) and a page table entry at any of the levels occupies 8 bytes.

The value of L is \_\_\_\_\_.

gatecse-2023 operating-system virtual-memory numerical-answers two-marks

Answer key

5.33.38 Virtual Memory: GATE CSE 2024 | Set 1 | Question: 52



Consider a memory management system that uses a page size of 2 KB. Assume that both the physical and virtual addresses start from 0. Assume that the pages 0, 1, 2, and 3 are stored in the page frames 1, 3, 2, and 0, respectively. The physical address (*in decimal format*) corresponding to the virtual address 2500 (*in decimal format*) is \_\_\_\_\_.

gatecse-2024-set1 numerical-answers operating-system virtual-memory two-marks

Answer key

5.33.39 Virtual Memory: GATE CSE 2024 | Set 2 | Question: 14



Which of the following tasks is/are the responsibility/responsibilities of the memory management unit (MMU) in a system with paging-based memory management?

- A. Allocate a new page table for a newly created process  
B. Translate a virtual address to a physical address using the page table  
C. Raise a trap when a virtual address is not found in the page table  
D. Raise a trap when a process tries to write to a page marked with read-only permission in the page table

gatecse-2024-set2 operating-system multiple-selects virtual-memory one-mark

Answer key

### 5.33.40 Virtual Memory: GATE CSE 2024 | Set 2 | Question: 54



Consider a 32-bit system with 4 KB page size and page table entries of size 4 bytes each. Assume  $1 \text{ KB} = 2^{10}$  bytes. The OS uses a 2-level page table for memory management, with the page table containing an outer page directory and an inner page table. The OS allocates a page for the outer page directory upon process creation. The OS uses demand paging when allocating memory for the inner page table, i.e., a page of the inner page table is allocated only if it contains at least one valid page table entry.

An active process in this system accesses 2000 unique pages during its execution, and none of the pages are swapped out to disk. After it completes the page accesses, let  $X$  denote the minimum and  $Y$  denote the maximum number of pages across the two levels of the page table of the process.

The value of  $X+Y$  is \_\_\_\_\_.

gategse-2024-set2 numerical-answers operating-system virtual-memory two-marks

Answer key

### 5.33.41 Virtual Memory: GATE IT 2004 | Question: 66



In a virtual memory system, size of the virtual address is 32-bit, size of the physical address is 30-bit, page size is 4 Kbyte and size of each page table entry is 32-bit. The main memory is byte addressable. Which one of the following is the maximum number of bits that can be used for storing protection and other information in each page table entry?

- A. 2                                      B. 10                                      C. 12                                      D. 14

gateit-2004 operating-system virtual-memory normal

Answer key

### 5.33.42 Virtual Memory: GATE IT 2008 | Question: 16



A paging scheme uses a Translation Look-aside Buffer (TLB). A TLB-access takes 10 ns and the main memory access takes 50 ns. What is the effective access time(in ns) if the TLB hit ratio is 90% and there is no page-fault?

- A. 54                                      B. 60                                      C. 65                                      D. 75

gateit-2008 operating-system virtual-memory normal

Answer key

### 5.33.43 Virtual Memory: GATE IT 2008 | Question: 56



Match the following flag bits used in the context of virtual memory management on the left side with the different purposes on the right side of the table below.

| Name of the bit | Purpose                    |
|-----------------|----------------------------|
| I. Dirty        | a. Page initialization     |
| II. R/W         | b. Write-back policy       |
| III. Reference  | c. Page protection         |
| IV. Valid       | d. Page replacement policy |

- A. I-d, II-a, III-b, IV-c                                      B. I-b, II-c, III-a, IV-d  
C. I-c, II-d, III-a, IV-b                                      D. I-b, II-c, III-d, IV-a

gateit-2008 operating-system virtual-memory easy match-the-following

Answer key

## Answer Keys

|       |   |       |     |       |     |       |   |       |       |
|-------|---|-------|-----|-------|-----|-------|---|-------|-------|
| 5.1.1 | A | 5.1.2 | A;C | 5.2.1 | 3:3 | 5.3.1 | C | 5.3.2 | B     |
| 5.3.3 | C | 5.3.4 | A;B | 5.4.1 | 800 | 5.5.1 | 2 | 5.5.2 | A;B;C |



|         |           |
|---------|-----------|
| 5.5.3   | 06 : 06   |
| 5.7.1   | N/A       |
| 5.7.6   | 9.006     |
| 5.7.11  | D         |
| 5.7.16  | B         |
| 5.7.21  | D         |
| 5.7.26  | 30.06     |
| 5.8.1   | N/A       |
| 5.8.6   | A         |
| 5.8.11  | 346       |
| 5.8.16  | C         |
| 5.9.5   | A;C       |
| 5.10.3  | C         |
| 5.10.8  | C         |
| 5.11.5  | A         |
| 5.14.1  | 90.00     |
| 5.14.6  | D         |
| 5.17.3  | B         |
| 5.17.8  | B         |
| 5.19.3  | B         |
| 5.20.5  | C         |
| 5.20.10 | B         |
| 5.20.15 | C         |
| 5.20.20 | D         |
| 5.20.25 | B         |
| 5.20.30 | A         |
| 5.22.1  | B         |
| 5.23.1  | N/A       |
| 5.23.6  | A         |
| 5.23.11 | B         |
| 5.23.16 | B         |
| 5.23.21 | A         |
| 5.23.26 | 5.5       |
| 5.23.31 | 8.25      |
| 5.23.36 | 5.25:5.26 |
| 5.23.41 | B         |
| 5.23.46 | D         |
| 5.24.2  | N/A       |
| 5.24.7  | N/A       |
| 5.24.12 | N/A       |

|         |               |
|---------|---------------|
| 5.5.4   | A             |
| 5.7.2   | N/A           |
| 5.7.7   | D             |
| 5.7.12  | A             |
| 5.7.17  | C             |
| 5.7.22  | 99.55 : 99.65 |
| 5.7.27  | C             |
| 5.8.2   | N/A           |
| 5.8.7   | D             |
| 5.8.12  | 85            |
| 5.9.1   | C             |
| 5.9.6   | 153           |
| 5.10.4  | 31            |
| 5.11.1  | A             |
| 5.11.6  | B             |
| 5.14.2  | D             |
| 5.15.1  | 4096          |
| 5.17.4  | B             |
| 5.17.9  | B             |
| 5.20.1  | N/A           |
| 5.20.6  | A             |
| 5.20.11 | A             |
| 5.20.16 | A             |
| 5.20.21 | 6             |
| 5.20.26 | A;C           |
| 5.20.31 | B             |
| 5.22.2  | C             |
| 5.23.2  | N/A           |
| 5.23.7  | D             |
| 5.23.12 | D             |
| 5.23.17 | A             |
| 5.23.22 | C             |
| 5.23.27 | 12            |
| 5.23.32 | 3             |
| 5.23.37 | 12 : 12       |
| 5.23.42 | A             |
| 5.23.47 | B             |
| 5.24.3  | N/A           |
| 5.24.8  | N/A           |
| 5.24.13 | D             |

|         |             |
|---------|-------------|
| 5.6.1   | 0.6         |
| 5.7.3   | B           |
| 5.7.8   | N/A         |
| 5.7.13  | C           |
| 5.7.18  | C           |
| 5.7.23  | 14020       |
| 5.7.28  | D           |
| 5.8.3   | N/A         |
| 5.8.8   | B           |
| 5.8.13  | B           |
| 5.9.2   | D           |
| 5.9.7   | B           |
| 5.10.5  | C;D         |
| 5.11.2  | B           |
| 5.11.7  | C           |
| 5.14.3  | A           |
| 5.16.1  | 65468:65468 |
| 5.17.5  | 10000       |
| 5.18.1  | 11:11       |
| 5.20.2  | B           |
| 5.20.7  | B           |
| 5.20.12 | C           |
| 5.20.17 | A           |
| 5.20.22 | A           |
| 5.20.27 | 4108 : 4108 |
| 5.21.1  | N/A         |
| 5.22.3  | B           |
| 5.23.3  | N/A         |
| 5.23.8  | A           |
| 5.23.13 | A           |
| 5.23.18 | B           |
| 5.23.23 | B           |
| 5.23.28 | D           |
| 5.23.33 | 29          |
| 5.23.38 | A;C;D       |
| 5.23.43 | 9.5 : 9.5   |
| 5.23.48 | D           |
| 5.24.4  | N/A         |
| 5.24.9  | N/A         |
| 5.24.14 | N/A         |

|         |           |
|---------|-----------|
| 5.6.2   | A         |
| 5.7.4   | N/A       |
| 5.7.9   | N/A       |
| 5.7.14  | B         |
| 5.7.19  | B         |
| 5.7.24  | 6.1 : 6.2 |
| 5.7.29  | B         |
| 5.8.4   | C         |
| 5.8.9   | 3         |
| 5.8.14  | C         |
| 5.9.3   | D         |
| 5.10.1  | C         |
| 5.10.6  | 14        |
| 5.11.3  | C         |
| 5.12.1  | 21:21     |
| 5.14.4  | C         |
| 5.17.1  | 3.2       |
| 5.17.6  | A         |
| 5.19.1  | A         |
| 5.20.3  | B         |
| 5.20.8  | C         |
| 5.20.13 | B         |
| 5.20.18 | B         |
| 5.20.23 | 1         |
| 5.20.28 | 6:6       |
| 5.21.2  | N/A       |
| 5.22.4  | B;C;D     |
| 5.23.4  | C         |
| 5.23.9  | N/A       |
| 5.23.14 | B         |
| 5.23.19 | C         |
| 5.23.24 | 7.2       |
| 5.23.29 | C         |
| 5.23.34 | 2         |
| 5.23.39 | C;D       |
| 5.23.44 | B         |
| 5.23.49 | C         |
| 5.24.5  | N/A       |
| 5.24.10 | C         |
| 5.24.15 | B         |

|         |           |
|---------|-----------|
| 5.6.3   | D         |
| 5.7.5   | N/A       |
| 5.7.10  | D         |
| 5.7.15  | A         |
| 5.7.20  | B         |
| 5.7.25  | 4096      |
| 5.7.30  | D         |
| 5.8.5   | B         |
| 5.8.10  | 10        |
| 5.8.15  | B         |
| 5.9.4   | 4.0 : 4.1 |
| 5.10.2  | B         |
| 5.10.7  | 4 : 4     |
| 5.11.4  | A         |
| 5.13.1  | B         |
| 5.14.5  | C         |
| 5.17.2  | N/A       |
| 5.17.7  | C         |
| 5.19.2  | C         |
| 5.20.4  | C         |
| 5.20.9  | C         |
| 5.20.14 | A         |
| 5.20.19 | 7         |
| 5.20.24 | D         |
| 5.20.29 | C         |
| 5.21.3  | N/A       |
| 5.22.5  | B         |
| 5.23.5  | B         |
| 5.23.10 | 19        |
| 5.23.15 | A         |
| 5.23.20 | D         |
| 5.23.25 | 1000      |
| 5.23.30 | A         |
| 5.23.35 | C         |
| 5.23.40 | B;C       |
| 5.23.45 | D         |
| 5.24.1  | D         |
| 5.24.6  | N/A       |
| 5.24.11 | C         |
| 5.24.16 | N/A       |

|         |         |
|---------|---------|
| 5.24.17 | N/A     |
| 5.24.22 | N/A     |
| 5.24.27 | A       |
| 5.24.32 | A       |
| 5.24.37 | C       |
| 5.24.42 | D       |
| 5.24.47 | A       |
| 5.24.52 | D       |
| 5.25.5  | N/A     |
| 5.25.10 | C       |
| 5.25.15 | A       |
| 5.25.20 | 7       |
| 5.25.25 | A;D     |
| 5.28.1  | N/A     |
| 5.28.6  | A       |
| 5.28.11 | D       |
| 5.31.3  | C       |
| 5.31.8  | D       |
| 5.33.1  | N/A     |
| 5.33.6  | A       |
| 5.33.11 | A;C     |
| 5.33.16 | C       |
| 5.33.21 | C       |
| 5.33.26 | B       |
| 5.33.31 | 22      |
| 5.33.36 | 155:156 |
| 5.33.41 | D       |

|         |       |
|---------|-------|
| 5.24.18 | B     |
| 5.24.23 | N/A   |
| 5.24.28 | B     |
| 5.24.33 | A     |
| 5.24.38 | 3     |
| 5.24.43 | C     |
| 5.24.48 | B     |
| 5.25.1  | N/A   |
| 5.25.6  | N/A   |
| 5.25.11 | N/A   |
| 5.25.16 | A     |
| 5.25.21 | D     |
| 5.25.26 | C     |
| 5.28.2  | B     |
| 5.28.7  | A;B;D |
| 5.29.1  | D     |
| 5.31.4  | D     |
| 5.31.9  | A     |
| 5.33.2  | N/A   |
| 5.33.7  | C     |
| 5.33.12 | B;D   |
| 5.33.17 | N/A   |
| 5.33.22 | C     |
| 5.33.27 | C     |
| 5.33.32 | 36    |
| 5.33.37 | 5     |
| 5.33.42 | C     |

|         |       |
|---------|-------|
| 5.24.19 | N/A   |
| 5.24.24 | B     |
| 5.24.29 | B     |
| 5.24.34 | B     |
| 5.24.39 | A     |
| 5.24.44 | 80    |
| 5.24.49 | A     |
| 5.25.2  | N/A   |
| 5.25.7  | C     |
| 5.25.12 | C     |
| 5.25.17 | B     |
| 5.25.22 | D     |
| 5.25.27 | B     |
| 5.28.3  | B     |
| 5.28.8  | C     |
| 5.30.1  | A;C   |
| 5.31.5  | D     |
| 5.31.10 | A     |
| 5.33.3  | 99.99 |
| 5.33.8  | N/A   |
| 5.33.13 | D     |
| 5.33.18 | C     |
| 5.33.23 | D     |
| 5.33.28 | C     |
| 5.33.33 | 384   |
| 5.33.38 | 6596  |
| 5.33.43 | D     |

|         |       |
|---------|-------|
| 5.24.20 | B     |
| 5.24.25 | C     |
| 5.24.30 | D     |
| 5.24.35 | D     |
| 5.24.40 | A     |
| 5.24.45 | B;C   |
| 5.24.50 | C     |
| 5.25.3  | A     |
| 5.25.8  | N/A   |
| 5.25.13 | B     |
| 5.25.18 | B     |
| 5.25.23 | B     |
| 5.26.1  | C;D   |
| 5.28.4  | C     |
| 5.28.9  | C     |
| 5.31.1  | A     |
| 5.31.6  | B     |
| 5.32.1  | C     |
| 5.33.4  | B;C;E |
| 5.33.9  | A     |
| 5.33.14 | B     |
| 5.33.19 | D     |
| 5.33.24 | B     |
| 5.33.29 | 122   |
| 5.33.34 | B     |
| 5.33.39 | B;C;D |

|         |       |
|---------|-------|
| 5.24.21 | N/A   |
| 5.24.26 | D     |
| 5.24.31 | A     |
| 5.24.36 | C     |
| 5.24.41 | C     |
| 5.24.46 | D     |
| 5.24.51 | C     |
| 5.25.4  | D;E   |
| 5.25.9  | B     |
| 5.25.14 | C     |
| 5.25.19 | B     |
| 5.25.24 | A     |
| 5.27.1  | D     |
| 5.28.5  | 7     |
| 5.28.10 | A;B;C |
| 5.31.2  | D     |
| 5.31.7  | A;D   |
| 5.32.2  | 1536  |
| 5.33.5  | A     |
| 5.33.10 | N/A   |
| 5.33.15 | D     |
| 5.33.20 | C     |
| 5.33.25 | B     |
| 5.33.30 | 4     |
| 5.33.35 | B     |
| 5.33.40 | 1028  |